Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Duy Khanh Tran

Master's Thesis
Espoo, November 5, 2026

Supervisor: Prof. Petri Vuorimaa
Major: Computer Science

Abstract

A user-interface (UI) transition graph describes the screens of an application and the transitions a user can cause between them. For single-page web applications they are usually built by running the application, though routing frameworks now declare routes in the source. How well static extraction works on real applications, and why it fails, has not been measured construct by construct.

This thesis builds uigraph, a static extractor for Angular, and makes that measurement with it on six open-source applications. uigraph reads nested and parameterised route trees, which the stated restrictions of an earlier Angular extraction exclude in every one of them. Five methods check the graphs: four read the source, one the running application's route configuration. Each construct the extractor does not recover is assigned to a loss category. Under the order of the loss rules, the largest loss is navigation that belongs to no single screen, such as a link in a shared header. For this loss the thesis proposes scope attribution, a dataflow analysis that computes the set of screens from which each navigation site can fire.

Three results stand out. First, almost no navigation can be extracted as a deterministic transition. The corpus yields 150 such edges from 25 constructs. Most of them come from one toolbar whose screen set is untested, and most rest on an assumption about route order that the extractor does not read. Second, almost every cause of loss traced in the source is a construct a static reader can read, so the loss was the tool's; the final version reads most of them, and two lie beyond any static reader of a route table. Third, scope attribution recovers 64.3 % of the constructs the loss rules assign to it, most of them on one application and almost all as possible rather than certain transitions. On the one application whose screen sets were read by hand, it adds almost nothing to a reimplementation of earlier mechanisms. That reference applies the analysis's own relations, was partly completed from the graph's output and guided two repairs. Mutation testing on three applications shows that the graph responds to injected changes in constructs it already represents; its harness was corrected after its misses were read, so the test is not independent.

The conclusion is that a declared route table is a usable starting point for static navigation extraction, but not a sufficient one. Scope attribution adds a modest gain over earlier mechanisms, and its screen sets were checked in both directions on one application only. The corpus is small and Angular-only, and the tool was developed on it.

Keywords: static analysis; UI transition graph; single-page application; Angular; dataflow analysis; program analysis; software testing

Use of AI tools

Aalto University asks that the use of artificial intelligence (AI) tools in a thesis be declared. An AI coding assistant was used in this work in the ways set out below, and I am responsible for the whole of the result.

The extractor built in this thesis (uigraph) and the evaluation harness were written with the help of an AI coding assistant. The result tables and plots of Chapter 5 are generated from the harness's result files, except the two tables marked as compiled by hand. So are the figures the text states for the final version. All of these can be reproduced from the commits recorded with them. Four kinds of figure cannot:

- those of superseded instruments, copied by hand from the outputs of those runs (Appendix A);
- a few counts taken at named earlier commits whose output was not committed;
- a few counts from the history of the structural reference and of the instruments, stated in words;
- the corpus-selection counts of Section 4.1, read from the candidate repositories.

The diagrams in Chapters 1 to 4 were drawn by hand.

The assistant was also used to read the application sources, in three roles. As a reviewer it read the constructs behind an edge and reported where the extractor's reading disagreed with them. In red-team panels it attacked the extractor, the harness and this text. Several of the demotion rules of Section 5.5 were written in response to its objections, among them the rule by which the extractor decides which guards a navigation runs. Every defect it reported in either role was reproduced against the code before anything was changed, and the corrections that followed are listed in Appendix A.1. Its readings are also part of the evaluation itself. The two readings of a development version's must tier in `results-must/review.json` and the reading of the final tier in `results-must/tier-reading.json` were made by the assistant. Chapter 5 says what the reading of the final tier can and cannot show, and Table A.1 what the review of the development version's tier found. The two readings are two runs of one model, so their agreement measures consistency, not correctness.

The assistant was also used during the literature search, to locate candidate sources and to summarise them. No source is cited on the strength of such a summary.

Every entry in the bibliography was checked field by field against the publisher page, Crossref metadata, arXiv metadata or DBLP before it was cited, and sources that could not be checked in this way are kept in a separate block in the bibliography file and are not cited anywhere in the text.

Finally, the assistant was used to draft and edit prose, which I then revised. The research questions and the interpretation of the results are my own; the analysis and the evaluation protocol were revised in the course of the work in response to the reviews and panels described above, as Appendix A.1 records, and the responsibility for every claim made here is mine.

Abbreviations

AI	artificial intelligence
API	application programming interface
DOM	Document Object Model
GUI	graphical user interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ICU	International Components for Unicode (message format)
JSON	JavaScript Object Notation
JSP	JavaServer Pages
JSX	JavaScript XML, the markup syntax of React
MVC	model–view–controller
RQ	research question
RxJS	Reactive Extensions for JavaScript
SPA	single-page application
UI	user interface
URL	Uniform Resource Locator

Contents

Abstract **1**

Use of AI tools **2**

Abbreviations **4**

1 Introduction **8**

1.1 Motivation 8

1.2 Research questions and scope 10

1.3 Thesis structure 11

2 Background **13**

2.1 User-interface transition graphs 13

2.2 Static extraction for Android 14

2.3 Navigation models for web applications 15

2.4 Dynamic state models for single-page applications 15

2.5 Static analysis of JavaScript and TypeScript source 17

2.5.1 Analyses over a source-level syntax tree 18

2.5.2 Abstract interpretation of JavaScript 18

2.5.3 What TypeScript's types contribute 18

2.5.4 Frameworks, not the language, are the obstacle 19

2.6 Formal foundations 19

2.6.1 Modal transition systems 20

2.6.2 Soundiness 20

2.6.3 Dataflow analysis and abstract interpretation 20

2.7 Angular routing, dependency injection and reactive state 21

2.7.1 Routes 21

2.7.2 Components and templates 22

2.7.3 Navigation sites 22

2.7.4 Dependency injection 22

2.7.5 Reactive state . . . 24
2.8 Reactive programming and the limits of static dataflow . . . 24
2.8.1 Propagation order and glitch freedom . . . 25
2.8.2 Reactive code and the tools developers have . . . 25
2.9 Mutation testing . . . 26
2.9.1 The coupling effect . . . 26
2.9.2 Mutation for JavaScript and web applications . . . 26
2.9.3 Reporting a mutation experiment . . . 27
2.10 Summary and the gap . . . 27

3 Static navigation analysis and scope attribution 31
3.1 Graph model and soundness . . . 31
3.2 Scope attribution . . . 36
3.2.1 The defect being addressed . . . 37
3.2.2 Scope . . . 37
3.2.3 Input relations . . . 40
3.2.4 Guards as program points . . . 41
3.2.5 Boundary . . . 42
3.2.6 Anchored edges . . . 42
3.3 Implementation . . . 43
3.4 Summary . . . 46

4 Evaluation setup and research material 48
4.1 Corpus selection and validity . . . 48
4.1.1 Selection . . . 48
4.1.2 Validity of an open-source corpus . . . 49
4.2 Metrics . . . 50
4.3 Evaluation methods . . . 55
4.3.1 Preconditions . . . 58
4.3.2 Corrections made to the protocol . . . 58
4.3.3 Reproducibility . . . 59
4.4 Summary . . . 60

5 Results 64
5.1 The two levels of loss (RQ1) . . . 64
5.2 Which failure is worth an analysis (RQ2) . . . 66
5.3 Scope attribution (RQ3) . . . 68
5.3.1 Checking the scopes against the running interface . . . 72

5.3.2 What the attribution discriminates . . . 74
5.3.3 What each mechanism contributes . . . 75
5.3.4 Agreement with a structural reference . . . 76
5.3.5 Agreement with the applications' own suites . . . 77
5.3.6 Against earlier work . . . 79
5.3.7 What the RQ3 evidence supports . . . 79
5.4 Change sensitivity (RQ4) . . . 80
5.5 What the losses were (RQ5) . . . 83
5.5.1 What the losses were . . . 83
5.5.2 Construct accounting . . . 85
5.5.3 The soundness property of the final tier . . . 87
5.5.4 Nodes at addresses the application does not mount . . . 91
5.6 Dead links, found from the source . . . 93
5.7 Summary . . . 95

6 Discussion **96**
6.1 Key findings . . . 96
6.2 Limitations and threats to validity . . . 99
6.3 Declared approximations . . . 104
6.4 Transfer to other frameworks . . . 110
6.5 Future work . . . 112
6.6 Summary . . . 115

7 Conclusion **116**

A Validating the measurement **119**
A.1 History of the corrections . . . 119
A.2 Nodes the two static readers disagreed about . . . 127
A.3 How the declared-screen set moved . . . 128
A.4 Every remaining site names a reason . . . 130
A.5 The four dead-link corrections . . . 131
A.6 Asking the running framework . . . 134
A.6.1 How the framework was asked . . . 134
A.6.2 What the framework mounts and the extractor does not . . . 135
A.6.3 Nothing is missing that was not declared . . . 136
A.7 What a route table's evidence does not establish . . . 137
A.8 The intermediate must tier, construct by construct . . . 141

Bibliography **143**

Chapter 1

Introduction

A user interface (UI) transition graph is a model of an application in which the nodes are the screens a user can see and the edges are the transitions a user action can cause between them. For each edge, the graph records the event that triggers it, the condition under which the transition is available, and the screen it leads to. A graph of this kind makes it possible to answer questions that are difficult to answer from the code alone, for example which screens are reachable from a given screen, or which parts of the user interface are affected when a particular component changes.

Transition graphs have been used in automated testing for a long time. In the Android domain they are used for test generation and for guiding exploration [80, 40, 14], and in web testing they form the basis of crawling-based test derivation [51, 52]. They are also useful for change impact analysis [4, 12], because each transition can be traced back to the code that implements it, and from there to the tests that cover it.

1.1 Motivation

There are two general ways to obtain a transition graph. A dynamic approach executes the application and records the states and transitions it observes. A static approach reads the source code and derives the graph without running anything. Their limitations differ. A dynamically built graph contains only the states the crawler managed to reach, which depends on the available test data, authentication, and the exploration budget, and the graph has to be rebuilt for every new version. A statically built graph covers everything the source declares and can be recomputed on each commit without a running system, but it is limited to what can be decided from the source, and it must over-approximate wherever a target cannot be resolved.

The two limitations complement each other. Applications of the kind studied here are mostly used signed in, so much of their navigation runs behind an authentication guard whose outcome only a running system decides, while a crawler reaches only the states its data and credentials open. A static graph can name the screens and candidate transitions a crawler should look for and the guards each waits on, and a run can confirm which of them occur. This thesis measures the static side; the combination is discussed in Section 6.5 and is not measured.

For Android, static extraction is a well-established technique. The window transition graph work derives windows and the transitions between them from bytecode, and resolves the control flow of callbacks to the widget that registered each handler [80, 79, 78]. For the web, navigation models have been extracted statically from server-rendered frameworks, in which pages and links are declared in templates and configuration files [63, 9, 70]. Modern single-page applications, however, have almost always been modelled by executing them. Crawljax builds its state flow graph by firing events against the Document Object Model (DOM) of a running application [51], and later tools in the same line improve the exploration strategy without changing where the model comes from [52, 75]. A survey of web testing covering 2014 to 2025 collects the model-based tools of the past decade [37]. Of the tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application. Static analyses of a modern client-side framework are few. One builds a component dependency graph for React and does not address routing [39]. Three extract navigation, each for a different consumer:

- one from Angular in order to model-check it, accepting only applications that declare no nested routes, no route parameters and no data-bound navigation [57];
- one from Angular sources in order to enumerate the workflows a generated end-to-end suite should cover [65];
- one from the compiled JavaScript bundle of a single-page application in order to check an access-control policy against the screens it can reach [83].

Section 2.4 compares them with the work reported here.

When the dynamic approach was developed, a single-page application had no declarative route table, so its navigation had no explicit representation in the source and was hard to recover from it, and the DOM was the observable the tools used (Section 2.4). Client-side routing frameworks changed this. Angular, React Router and Vue Router all declare routes as data in the source code, connect navigation to those routes through template attributes and service calls, and attach guards to routes as named functions [27]. The artefact needed for static analysis now exists, but how well static extraction works on real applications built with these frameworks, and where it fails, has been measured only against hand-built references on the applications each work evaluated, and not construct by construct.

A statically extracted graph also has consumers outside model-based testing. Tools that drive a graphical user interface (GUI) with a large language model either explore an application without a model of its navigation structure or build one by exploring it. GPTDroid treats mobile GUI testing as a question-and-answer loop, passing the current screen to the model and replaying the action it returns, and reports higher activity coverage and more bugs than the best baseline on 93 applications [47]. AutoDroid gives the model application-specific knowledge in addition to its commonsense knowledge, held as a UI transition graph that it builds by automated dynamic exploration offline [74]. LLMDroid uses the model to guide exploration towards coverage the test run has not reached [71]. The approach has been applied

to web interfaces with a vision-language model and curiosity-driven exploration [73], and a survey of large language models in software testing covers the wider area [72]. Coding agents working on a repository may also benefit from structure of this kind: the SWE-bench issues typically require coordinated edits across functions, classes and files [32], and agent performance depends on the interface through which the repository is navigated [77]. Whether a navigation graph helps such tools is not tested here; these uses motivate the extraction but do not depend on the results of this thesis.

This thesis builds a static extractor for Angular, carries out that measurement with it, and proposes an analysis for the largest loss it finds under the order of its loss rules. The analysis, scope attribution, is stated as a dataflow analysis over a lattice of screen sets, and nothing in its formulation depends on this extractor.

A method of this kind cannot be assessed without being instantiated, because the questions asked here are empirical: how much of the navigation in real applications it recovers, where it fails, and whether the resulting graph responds to a change in the source. The extractor is *uigraph*, an open-source static analyser for Angular built in this thesis. It is part of the contribution, not an existing tool: it reads nested, parameterised and lazily loaded route trees, which an earlier Angular extraction excludes by its stated restrictions (Section 5.3.6), and it implements scope attribution. Its development versions and the corrections between them are listed in Appendix A.1. Because the same person built the tool and measured it, on the same corpus, the figures describe what this extractor recovers on these applications; Section 6.2 discusses what that means for them, and Chapter 6 separates the limits of the method from those of this implementation.

1.2 Research questions and scope

The thesis addresses five research questions (RQs). This section states what each asks and how it is answered; the exact measures are defined in Section 4.2, and the answers are given in Chapter 5.

RQ1. *How much of the navigation in Angular applications can static analysis recover as deterministic transitions?*

This is measured at two levels, because a transition cannot be recovered without the screens it runs between. The first level is node coverage: the share of the screens an application declares that the graph holds a node for. The second is the share of navigation constructs in the source that yield at least one deterministic (*must*) transition.

RQ2. *Where does static extraction fail, and why?*

The observed failures are grouped into defect classes. A class is accepted only if at least two lines of evidence confirm it, and at least one of them must come from an evaluation method of Section 4.3. Each construct that is not recovered is assigned to one loss category, so the categories can be compared by size.

RQ3. *How does scope attribution affect the recovered share and precision of static*

navigation analysis?

Scope attribution is the analysis this thesis proposes. For a navigation site that belongs to no single screen, it computes the set of screens from which the site can fire. The same extractor is run with the analysis disabled and enabled, so that nothing else differs between the two runs.

RQ4. *Does the extracted graph respond to source changes in the constructs it represents, as change impact analysis requires?*

This is measured with mutation testing: known changes are inserted into the source, and the graph is checked for the expected difference.

RQ5. *Are the losses a property of the applications or of the tool?*

A defect class names where extraction failed, but not whether the navigation was unreadable or merely unread. Each class is therefore traced to its cause in the source, and for each cause the thesis records whether the final version reads it. If it does, the information was in the source and the loss was the tool's. The causes that are still not read, and those beyond a static reader, are named. Every site that still yields no transition must carry a recorded label.

The questions are answered on a corpus of six open-source Angular applications pinned to fixed commits (Chapter 4). Five evaluation methods are used, and RQ3 adds four further measurements; the methods are chosen so that their blind spots differ. The figures in Chapter 5 are those of the final version of the extractor. Its development versions and the corrections between them are listed in Appendix A.1.

The thesis has the following limits of scope.

- It does not claim completeness. Where a navigation target is computed at run time, the transitions cannot in general be determined statically. The thesis instead states a soundness property over a declared scope and lists what it approximates, following the soundiness position (Section 2.6.2). The property is tested, not proved.
- It does not measure the effect of the graph on coding agents or any other consumer.
- The screen sets that scope attribution computes are compared in both directions on one application only, against a reference that applies the analysis's own relations.
- uigraph was developed on this corpus, and no held-out application tests how far its results generalise.
- All measurements concern Angular. The graph model does not depend on the framework, but no other framework is evaluated.

1.3 Thesis structure

Figure 1.1 shows how the parts of the study fit together.

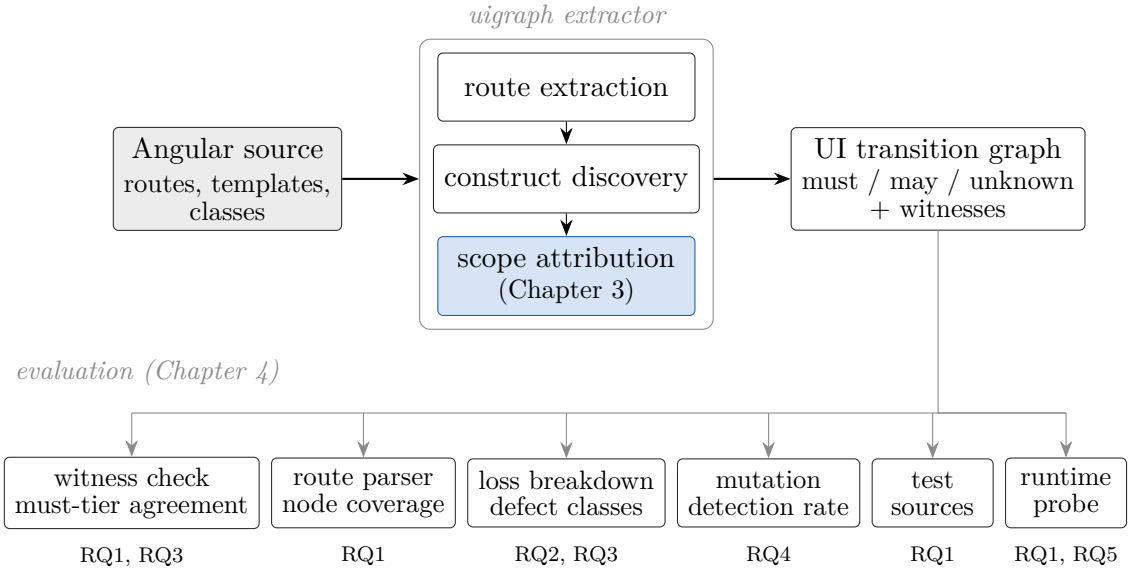


Figure 1.1: Overview of the study. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. The row below shows the main measurements, each tied to the research questions it answers; the loss breakdown classifies what the other methods find missing rather than being one of the five methods itself.

Chapter 2 presents the background. It introduces the terminology used for transition graphs in different research communities, reviews static extraction for Android, navigation models for web applications and dynamic state models for single-page applications, and describes the Angular mechanisms the analysis has to handle together with the program analysis ideas it builds on. Chapter 3 presents the graph model and its soundness claim, the scope attribution analysis proposed as a modest extension, and its implementation; the measurement, which is the main contribution, is set up in Chapter 4 and reported in Chapter 5. Chapter 4 describes the evaluation setup: the corpus and why it can be considered valid, the metrics, and the five evaluation methods. Chapter 5 presents the results for each research question: the two levels of loss (RQ1), the defect classes (RQ2), scope attribution and what it recovers (RQ3), change sensitivity (RQ4) the causes of the losses and the soundness of the must tier (RQ5), followed by the dead-link candidates the graph finds statically, each checked against the route configuration the runtime route oracle recorded. Appendix A lists the development versions of uigraph and the corrections between them, and holds the validation of the measuring instruments: the disagreements between the two static readers, how the declared-screen set moved under the corrections, the case-by-case account of asking the running framework, and the record that every remaining site names a reason, recorded rather than verified. It is kept apart from the results because it records the checks made on the instruments, not what the analysis recovers. Chapter 6 discusses the results, the limitations of the study, the constructs the analysis approximates, how the approach transfers to other frameworks, and future work. Chapter 7 concludes the thesis.

Chapter 2

Background

This chapter gives the background needed for the rest of the thesis and ends by stating the gap the thesis addresses. Section 2.1 fixes the terminology, since the same kind of graph is called by different names in different communities. Sections 2.2 to 2.4 review the three lines of prior work closest to this thesis: static extraction for Android, navigation models for web applications, and dynamic state models for single-page applications (SPAs). Section 2.5 reviews static analysis over a source-level syntax tree, the tooling style of Chapter 3, and what the type annotations of a TypeScript program do and do not contribute to it. Section 2.6 summarises the three formal ideas the analysis builds on. Section 2.7 introduces the Angular mechanisms the analysis has to deal with, dependency injection at length because it hides the most control flow. Section 2.8 reviews reactive programming and the boundary it puts on a static analysis. Section 2.9 reviews mutation testing, the method Chapter 4 uses to answer RQ4. Section 2.10 states the gap.

2.1 User-interface transition graphs

A UI transition graph is a directed graph in which the nodes represent user-visible states of an application and the edges represent transitions that a user action can cause between those states. Figure 2.1 shows a small example in the model used in this thesis.

Several research communities have studied this kind of graph, and each uses its own terminology. In Android static analysis it is called a *window transition graph* [80], or, at a finer level of detail, an *activity transition graph* [14] or a *screen transition graph* [40]. In the crawling literature for Ajax (asynchronous JavaScript and XML) applications the term is *state flow graph*, where a state is an abstraction of the DOM [51]. In web engineering the usual terms are *navigation model* and *navigation state machine* [63, 18]. This thesis uses the term *UI transition graph* and treats the other terms as equivalent for its purposes, while noting that they differ in what is counted as a state and in how the graph is constructed.

More important than the naming is how the graph is obtained: *statically*, from source code or binaries without executing the application, or *dynamically*, by running and observing it. It also matters whether the graph represents *guards*, that is, whether each edge records the condition under which the transition is available or

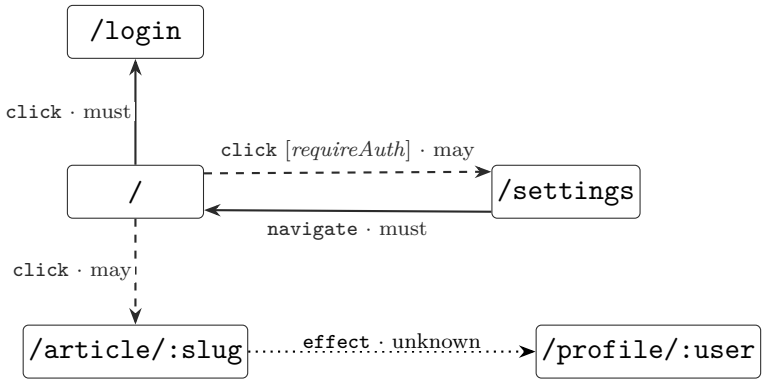


Figure 2.1: A small UI transition graph in the model of this thesis. Each edge carries an event, an optional guard in brackets, and a modality. A *must* edge always fires when its code runs, and a *may* edge is conditional, here on a route guard or on a target that is only partly known. The dotted *unknown* edge shows the model's treatment of a navigation inside a signal effect, whose firing condition cannot be determined statically; the prototype does not implement it (Section 3.2.5). In the measured graphs the unknown modality marks only edges into a sink for destinations that only the run time knows.

only that the transition exists. Table 2.1 classifies the work reviewed in the following sections according to both distinctions.

2.2 Static extraction for Android

The largest body of static work on transition graphs concerns Android. Yang et al. introduced the window transition graph, in which the nodes are windows (activities, dialogs and menus) and the edges are transitions caused by callbacks that run on the UI thread; the analysis models the stack of active windows, so that the effect of the back button is captured [80], and a journal version extends the work [78]. A related analysis by the same group constructs a callback control-flow graph whose nodes are pairs of a handler and the widget it belongs to [79]. StoryDroid builds an activity transition graph from `startActivity` calls and uses a simple lexical rule to attribute code inside an inner class to the enclosing activity [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration and evaluates the result against manually produced ground truth for 36 applications [40]. UI-CTX identifies the problem of finding the correct code context for a widget's behaviour as its first challenge and solves it with a backward data-flow analysis from the handler to the call that retrieves the widget and its identifier [45].

These approaches have in common the way a navigation is attributed to a screen. In Android, an event handler is registered on a widget object, and at run time that object belongs to exactly one window. Attribution can therefore be treated as an object resolution problem: the widget identifier is resolved, usually by constant propagation of the `getId()` call, and the window is then known [79].

The limits of this approach become visible in ProMal. Its authors report that a purely static window transition graph contains infeasible transitions, because

imprecision in the reference analysis makes several widgets aliases of one another and the over-approximated data flow then attaches a handler to the wrong window. They address this by combining the static analysis with runtime traces and a learned model [46]. Where the owning object cannot be resolved precisely, ProMal therefore turns to dynamic evidence rather than refining the static attribution.

2.3 Navigation models for web applications

Navigation models for the web were first extracted from server-rendered sites, well before single-page applications became common. Ricca and Tonella modelled a web application as a set of pages, frames and forms connected by links and form submissions. Their model includes conditional edges that carry a boolean condition on input values, and it was derived semi-automatically using a crawler together with annotations by an analyst [63]. WebUml recovers a state diagram from Hypertext Markup Language (HTML), script and server-side pages by combining static source analysis with mutation-driven execution, and derives guards by control-flow analysis of the server code [9]. A patent assigned to Fujitsu describes a fully static method for extracting a navigation model from Struts and JSP sources, including pages that are referenced but do not exist [70]. On the verification side, Haydar et al. model a web application as a system of communicating automata derived from recorded browsing sessions [29], and SiteHopper builds abstracted navigation state machines on the fly for model checking [18].

This work shows that navigation models, including guards, have been extracted statically from web applications. The models reviewed in this section all concern server-rendered frameworks, where a page corresponds to a file and a link to an anchor element, and none of them deals with a client-side router, a component tree, or dependency injection; Section 2.4 turns to the work that does.

2.4 Dynamic state models for single-page applications

When web applications moved to Ajax and later to client-side frameworks, work on model construction moved to run time. Crawljax executes user events against a running application, compares the DOM before and after each event, and builds a state flow graph whose nodes are equivalence classes of DOM states [51]. Later work in this line changed the exploration strategy but not the source of the model. FeedEx uses feedback from the graph built so far to direct further exploration [52], and FRAGGEN abstracts states as hierarchies of DOM fragments in order to generate better tests [75]. The survey by Kertusha et al. reviews web testing research from 2014 to 2025 [37]. Of the model-based tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application.

The reason is largely historical. In an Ajax application there was no declarative route table, and one uniform resource locator (URL) could represent many states,

because navigation was implemented by handlers that modified the DOM. The DOM was therefore the observable that most directly distinguished one state from another. The source was there to analyse, and static analysis of JavaScript was developed for it [30], but with dynamic dispatch, handlers constructed at run time and the DOM itself to model, recovering navigation from it was hard, and nothing in the source represented the routes explicitly. The dynamic approach was used because it observed directly what the source represented only implicitly.

Client-side routing frameworks changed this situation. The route table is now a declared data structure in the source, a navigation is an attribute or a method call that names a route, and a guard is a named function attached to a route. Four static analyses of such a framework were found during the literature search for this thesis, three of them reading the framework's own sources and one a compiled bundle. Kurant's component dependency graph for React stops at the component rendering graph without addressing routing [39].

The other three do address routing. Oshima and Kishi extract page transitions from an Angular application in order to model-check them with SPIN [57]. Their extraction reads the transition information from the route declarations in `RouterModule`, reads the navigable paths from every component rather than only from the routed ones, and then resolves the parent–child relationships among the root component, the routed component and the custom child components in order to attribute each navigable path to the routed component that contains it. That attribution step is scope attribution: a navigable path is attributed to the routed component that contains it, and the root component's own paths to every page, computed statically from the router configuration. Section 5.3.3 measures that combination, as the *rooted* level of the analysis presented here (the levels are defined in Section 4.3).

What distinguishes the present work is the input the method accepts and what it claims about its output. The authors state that the applications they handle must not use the `children` attribute or route parameters in the routing declarations, and that changes arising from data binding are not treated as page transitions. Those exclusions remove nested route trees and parameterised addresses; how the method treats a navigation whose destination is an expression rather than a literal is not stated in the paper, and is not assumed here. A screen is identified by the name of its routed component rather than by an address, so path composition does not arise, and guards, lazy loading and named outlets are not part of the model. The attribution follows component containment together with the root component's own paths, so a navigation site that reaches a screen through a method call or through a dispatch lies outside the mechanism. The model carries no modality, so a navigation that is always available and one that is available only if a guard admits it are represented identically. The evaluation applies the tool to four constructed applications and to two published ones of 1824 and 6893 lines, with six and twenty-three pages, and reports whether the intended model was produced and whether transition faults, injected by hand into constructed variants of the first sample application, were detected; the published applications were checked without injection. What the extraction loses on an application it cannot fully read is not measured, which is the

question this thesis takes as its subject.

Two recent works read a client-side router configuration statically for other ends. Rima et al. parse Angular sources and templates to build a graph of navigation structure, interaction sites and the conditions (guards among them) that decide whether an interaction is executable, enumerate workflows over that graph, and compile each workflow into a browser test; the graph is a coverage denominator for generated end-to-end tests [65]. This account is taken from three sources: the publisher's abstract, the operational specification in the project's replication package (`softscanner-sw/softscanner_fa`, `docs/paper/approach.md`), and the anonymised manuscript in the same repository (`docs/paper/main.pdf`). The published chapter was not available to this review and may differ from both documents. Both documents describe attribution beyond containment. In the manuscript the components active at a route are those activated by the route and its ancestor routes, closed under composition, and the specification adds the bootstrap components to that seed, which covers what the *rooted* and *outlet* levels of Section 5.3.3 place. A handler is followed into helper methods of the same class to depth one and into the first callback of a `subscribe`, and the service a call names is resolved by the type of the constructor parameter. The specification excludes router-outlet modelling and provider resolution beyond module and root providers, and it does not follow a call into a service, so a navigation inside a service member is not placed on the screens of its callers. Zhang et al. parse the compiled JavaScript bundle rather than the sources, take declared routes as vertices and navigation calls as edges, and check an access-control policy against the screens reachable from an authorised one. Their graph is declared an over-approximation and is not narrowed, and no navigation site is attributed to the screens it can fire from. The authors note that the graph may contain paths a guard would block, because guards are not represented [83]. Rima et al. compare their output with a ground truth built by hand for the applications they study, which measures what their extraction misses there; neither work classifies the losses by construct or measures them across a corpus. The problem addressed in this thesis is therefore neither new nor settled; Section 2.10 states what none of these works does.

The work reported here is located between these analyses and the dynamic tools described above.

2.5 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 reads Angular source with the TypeScript compiler application programming interface (API), so it works on a source-level abstract syntax tree rather than on a compiled artefact. This section reviews that tooling style, the abstract interpretation work that put lattice-based analysis of JavaScript on a formal footing, what the type annotations of a TypeScript program contribute, and the evidence that frameworks rather than the language are what limits such analyses in practice.

2.5.1 Analyses over a source-level syntax tree

Analysing the syntax tree of the source, rather than a bytecode or an intermediate representation, is a long-established style. Baxter et al. detect exact and near-miss code clones by comparing abstract syntax trees, using a variant of the common-subexpression detection used in compilers, and apply the technique to a C system of about 400 000 lines [8]. The approach keeps the analysis close to what the developer wrote, which matters when the result has to be reported back at a source location.

SPOON makes this style into a reusable library. It exposes a metamodel of Java source and lets an analysis or a transformation be written in plain Java, without modifying a compiler and without learning a separate formalism [59]. The TypeScript compiler API occupies the same position for TypeScript: it gives a typed syntax tree, a symbol table and a type checker over an ordinary program, which is what allows the Angular adapter to resolve a constant reference in a route table or the declared type of an injected field. The cost of the style is that the analysis sees the program as written, including the parts that a compiler would later rewrite or erase.

2.5.2 Abstract interpretation of JavaScript

Lattice-based analysis of JavaScript itself has a formal treatment. Jensen et al. present a static analysis based on abstract interpretation that infers detailed and sound type information for JavaScript, covering the full language including its object model and the built-in functions, and propose it as a foundation for developer tools [30]. The work shows that the framework of Section 2.6 applies to JavaScript at the level of individual values, which is a considerably harder problem than the one addressed in this thesis. Scope attribution (Section 3.2.2) uses the same machinery over a much coarser domain than heap objects.

2.5.3 What TypeScript's types contribute

Angular applications are written in TypeScript, so an analysis can read declared types instead of inferring them. The type system is deliberately unsound. Bierman et al. give a precise definition of it [10], and Richards et al. describe the annotations as unsound by design [64]. That deliberate unsoundness means a declared type is evidence about the program rather than a guarantee about its execution.

The empirical picture is consistent with that. Gao et al. took bugs that had been fixed in the histories of JavaScript projects, added type annotations to the buggy versions, and checked whether a type checker would have reported them; both Flow and TypeScript detected about 15% of these bugs [25]. Bogner and Merkel mined 604 GitHub projects, 299 in JavaScript and 305 in TypeScript, together more than 16 million lines of code, and found better code quality and understandability for the TypeScript projects, but no lower bug proneness and no shorter bug resolution time; heavier use of the `any` type correlated weakly with worse measurements [11]. The declarations that describe untyped libraries can themselves be wrong: TSCheck compares a declaration file against the library it describes by capturing the runtime

object state after initialisation and adding a light static analysis of function code, and found 142 errors in the declarations of 10 libraries [22].

What follows for an analysis of Angular code is a division of labour. A declared type narrows the set of candidate implementations of a call, which is useful and is what the call graph construction in Section 2.7 relies on. It does not establish that the call reaches exactly one of them, so an unresolved or ambiguous type has to widen the result rather than be trusted. In the graph model this appears as the difference between the must tier and the may tier.

2.5.4 Frameworks, not the language, are the obstacle

The practical obstacle to analysing a client-side application is the framework around the code, not the language. Madsen et al. observe that JavaScript applications depend on large frameworks and libraries whose behaviour a static analysis has to know. Hand-written models of them are tedious to produce, incomplete and sometimes wrong, and the alternative is to infer library behaviour from the way the application uses it. They combine pointer analysis with such a *use analysis* and evaluate it on 25 Windows 8 JavaScript applications [49]. A later analysis by Madsen et al. addresses implicit control flow directly. Their *event-based call graph* for Node.js represents listeners that run when an event fires, and the analyses that build it find event-handling defects such as events that no listener handles and listeners registered too late [50]. An Angular application has the same shape of problem, with the router, template event bindings and observable subscriptions in place of the Node.js event emitter.

Two further static analyses of a framework in the Angular family bear on this work. The one that addresses navigation is Oshima and Kishi's, discussed in Section 2.4. The other, which does not, is Aurebesh, by Ocariza et al. [56]. It defines a consistency model for web applications built on a model–view–controller (MVC) framework and detects identifier and type inconsistencies between models, views and controllers, reporting a recall of 96.1% and a precision of 100%, and finding 15 real bugs. The work concerns AngularJS 1.x, which is a different framework from the Angular versions this thesis analyses, with a different template language, a different component model and a different injector. What the two have in common is the problem of relating an identifier in a template to the code that defines it, which Aurebesh solves for a consistency check.

2.6 Formal foundations

The analysis builds on three ideas from the literature: a transition system in which edges carry a modality, a stated position on what soundness can mean for a practical analysis, and dataflow analysis as the computational framework.

2.6.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required (*must*) or merely permitted (*may*) [43]. The graph model in Chapter 3 adopts this distinction and adds a third label, *unknown*, for edges whose modality could not be determined. The distinction has practical consequences: the thesis makes a soundness claim about the must tier only, and the evaluation treats the tiers differently, since the evidence behind a may edge establishes less than the evidence behind a must edge.

Carrying a modality on the edge also changes what an unresolved construct does to the graph. An analysis with a single edge kind has two options when it cannot decide whether a transition occurs: emit the edge and lose precision, or drop it and lose recall. With a modality the same situation produces an edge whose label records how much was established, and a consumer can select the tier it needs. The tiers are reported separately in Chapter 5 for this reason.

2.6.2 Soundiness

Livshits et al. argue that practical static analyses are sound only with respect to a subset of the language, that this is unavoidable in practice, and that the appropriate response is to state explicitly which language features are approximated rather than to claim soundness in general [48]. The thesis states its own position in Section 3.1. The literature reviewed in Sections 2.5 to 2.7 gives the specific reasons why a general claim is not available for this setting. Type annotations in TypeScript are deliberately not guarantees, framework configuration decides which implementation a call reaches, and the firing condition of a reactive computation is not written in the program text.

2.6.3 Dataflow analysis and abstract interpretation

Kildall showed that many program analyses can be stated as the computation of a fixpoint over a lattice of abstract values, where information is combined with a join operation and propagated along the edges of a graph until nothing changes [38]. If the lattice has finite height and the transfer functions are monotone, the computation terminates and its result does not depend on the order in which nodes are visited. Cousot and Cousot gave the general theory behind this approach as abstract interpretation [16], and Nielson et al. give a textbook account [55]. Scope attribution in Chapter 3 is an instance of this framework; its abstract values and join are defined in Section 3.2.2.

The framework is independent of the language being analysed, and it has been applied to JavaScript as well; Section 2.5 describes that work. What a particular instance needs is a lattice of bounded height and a propagation graph. Where the propagation graph is incomplete, because a call could not be resolved, the result is incomplete in a way the framework does not detect.

2.7 Angular routing, dependency injection and reactive state

The analysis presented in Chapter 3 operates on Angular applications, and most of its difficulties arise from particular Angular mechanisms. This section introduces those mechanisms and relates each of them to the program analysis problem it creates. The Angular documentation is the reference for the framework details [27]. Listings 2.1 to 2.3 show the constructs discussed below in a small example, which later chapters return to.

Listing 2.1: A route table with the constructs the analysis must handle: a guarded route, a parent route whose children render in its outlet, a lazily loaded route table and a path held in a constant.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root component's template. The header sits next to the outlet, so its links are present on every screen but belong to none of them.

```
<app-header />                <!-- shell: rendered on every screen -->
<router-outlet />             <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Listing 2.3: Navigation from an injected service. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  private session = inject(SessionStore);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

2.7.1 Routes

An Angular application declares its screens as a tree of route objects. Each route has a `path` pattern and, in addition, either a `component` to render, a `redirectTo` target,

or a list of `children`. A route that has children but no component acts as a path prefix and does not itself correspond to a screen. Routes can be loaded lazily using `loadChildren` or `loadComponent`; in that case the child routes or the component are defined in another module or file, which the bundler resolves when the application is built and the router fetches when a navigation first matches the route, or earlier if a preloading strategy is configured. Routes may also have guards, namely `canMatch`, `canActivate`, `canActivateChild` and `canDeactivate`, with `canLoad` still accepted but deprecated in favour of `canMatch`. Guards are functions or injectable classes that return a boolean, a `UrlTree` or `RedirectCommand` representing a redirect, or a Promise or Observable of one; a `canMatch` that returns false makes the router try the next matching route rather than cancel the navigation. Routes may also have resolvers, which fetch data the router awaits before it activates the route; `runGuardsAndResolvers` controls whether they run again when a navigation keeps the route active. Route paths are strings, but they are not necessarily string literals. Applications that keep their route table in one place often write `path: routes.about.path`, and resolving such a path requires following the constant to its definition.

2.7.2 Components and templates

Each screen is rendered by a component, which is a class together with a template. A template can instantiate other components by their selectors, so the components of an application form a nesting graph. A template can also contain a `<router-outlet>` element, which marks the position where the router inserts the component of the currently active child route. Components that are above the outlet in the nesting graph make up the application shell and are present on every screen, whereas components below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make the nesting conditional. Templates can additionally be passed around as values of type `TemplateRef` and rendered in a different component, and components can be created programmatically at run time.

2.7.3 Navigation sites

In a template, navigation is expressed with the `routerLink` directive, whose value is either a literal path or an array of path segments. In class code, navigation is performed by calling `Router.navigate` or `Router.navigateByUrl`; the target may be given relative to the current route through the `relativeTo` option. Both kinds of navigation site can appear in any component, in any injectable service, and in guards, interceptors and resolvers. The source code does not state anywhere which screens a given site can be triggered from; this information has to be derived.

2.7.4 Dependency injection

Angular creates services and passes them to the components and services that declare them as dependencies. This is dependency injection, a form of inversion of control:

a class does not construct or look up its collaborators, but receives them from a container [33, 24]. A dependency is requested by type or by an injection token, and the same token can be bound to different implementations in different injector scopes, using `useClass`, `useExisting` or a factory function.

The pattern is not specific to Angular and is widely used. Yang, Tempero and Melton developed operational definitions of dependency injection and techniques for detecting its use from source code, and applied them to 34 open-source Java applications [76]. Their motivation is the usual argument for the pattern, that it improves extensibility, modifiability, testability and reusability. The configuration that dependency injection introduces is itself an object of static analysis: Laigner et al. catalogue 12 dependency injection anti-patterns together with refactorings, and detect them with a static analyser at a relative recall of 92.19%, reporting that at least 9 of the anti-patterns occur frequently in open-source and industrial projects [41]. Dependency injection configuration is therefore readable statically; it does not state which binding a given call site will see.

For a static analysis this turns a simple method call into a dispatch problem. A call such as `this.nav.goHome()` can only be linked to the method it invokes after the type of `nav` and the binding of that type in the relevant injector have been resolved. This is the call graph construction problem known from object-oriented languages, where a virtual call must be resolved to its possible targets. Class hierarchy analysis resolves such a call using the declared type and its subclasses [17]. Rapid type analysis narrows the result to classes that are actually instantiated [5], and points-to analysis narrows it further by tracking which objects each variable may hold [2]. Grove and Chambers place these algorithms in a common framework that trades precision against cost [28]. For JavaScript, Feldthaus et al. show that an approximate call graph, which tracks functions only by the property names they are stored under, is precise enough for practical tools [23].

Dependency injection adds a further difficulty, because the binding of a type to an implementation is not written in ordinary code but in framework configuration. The Java literature has measured what this costs a general analysis and what recovers it, and Table 2.2 lists the analyses that model framework behaviour explicitly. Sridharan et al. report that the heavy use of reflection in frameworks makes static taint analysis ineffective, and that adding support for one framework at a time does not scale. Their tool instead generates a specification of framework behaviour from the application code and its configuration files, which an unmodified taint engine then consumes. This found 525 new issues across nine benchmarks [68]. Antoniadis et al. report that major Java analysis frameworks offer almost no support for enterprise applications, capture the concepts those frameworks share, such as components and entry points, in a declarative and largely framework-independent form, and measure about four times higher completeness and about a sixfold speedup against the most precise conventional analysis [3]. Landman et al. put a bound on what can be hoped for: in a literature review and corpus study of Java reflection they find that reflection cannot be ignored in 78% of projects and that resolving it requires unsound assumptions [42].

The closest analogue to resolving Angular's injectors is Jasmine, by Chen et al. [13].

Spring's core mechanisms, dependency injection and aspect-oriented programming, rely on dynamic language features, and the consequence is that a conventional call graph of a Spring application is missing edges. Jasmine extends Soot so that the call graph of a Spring program becomes more complete, and was evaluated on micro-benchmarks and on 18 real Spring programs; it improves on tools built with Soot and with Doop, and in combination with FlowDroid it exposed 12 sensitive-data leak paths. The structural problem is the one that occurs in Angular: an injected field has a declared type, the binding of that type lives in framework configuration, and an analysis that does not read the configuration cannot connect the call to a body. The difference is in where the configuration lives and how far its effect reaches. Angular binds tokens in provider arrays that belong to a module, a route or a component, so the same token can resolve to different implementations depending on where the injection happens. Resolution is relative to an injector rather than global, which makes the provider hierarchy part of the input to the analysis.

The analysis follows the advice this literature converges on, which is to model the framework explicitly rather than to treat its configuration as ordinary code (Section 3.2.3).

2.7.5 Reactive state

Recent versions of Angular include signals and effects. A signal is a value that changes over time, and an effect is a computation that runs again whenever a signal it read changes [26]. This is an instance of reactive programming, which goes back to functional reactive programming [21] and has been surveyed by Bainomugisha et al. [6]. In reactive systems the dependencies between values are recorded at run time, and the runtime propagates updates without explicit calls in the program text. So when a navigation call is placed inside an effect, the condition under which it fires is a data dependence in a reactive graph and not a control-flow condition. A static analysis can find which signals the effect reads; when the effect runs is not determined by the source in general, and the analysis of this thesis does not attempt to bound it. Angular applications also use observables from Reactive Extensions for JavaScript (RxJS) for the same purpose, in router event subscriptions, Hypertext Transfer Protocol (HTTP) responses and state containers. Section 2.8 reviews what the reactive programming literature establishes about this boundary.

2.8 Reactive programming and the limits of static dataflow

The reactive style gives the runtime, rather than the program text, control over when a computation runs. This section reviews the propagation discipline that reactive runtimes implement, and the evidence about how well reactive code can be reasoned about with tools.

2.8.1 Propagation order and glitch freedom

A reactive program is a graph of values in which an update to one value has to be propagated to the values that depend on it. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language, so that an expression evaluates to a node in a dataflow graph [15]. Their account names the central hazard: a *glitch* occurs when a signal is recomputed before all of the signals it depends on have been brought up to date, which makes an intermediate value observable that no consistent state of the program ever had. Their runtime prevents glitches by assigning each signal a height and updating signals in order of increasing height, which is a topological order and therefore requires the dependency graph to be acyclic. Drechsler et al. carry the property into a distributed setting with a change-propagation algorithm that needs no central knowledge of the dependency topology, avoids propagating updates that cannot change a result, and preserves glitch freedom [20]. An Angular signal graph, as described in Section 2.7.5, records its dependencies as they are read, so it is the kind of structure that these propagation algorithms are defined over.

The consequence for a static analysis is that the order in which reactive computations run is a property of the graph that the runtime builds, and the graph is built from the reads that actually happen. The set of signals a given effect may read can be over-approximated from the source. The number of times the effect runs, and the state of the application when it runs, depend on which writes occur and in what order, so they cannot in general be determined exactly from the source; a static analysis could still over-approximate them, and the one presented here does not try.

2.8.2 Reactive code and the tools developers have

The empirical literature on reactive programming reports a split between reading code and reasoning about its behaviour. Salvaneschi et al. ran an experiment with 127 subjects comparing reactive programming against object-oriented code using the Observer pattern, and found comprehension significantly better in the reactive version [67]. The same group reports that the tool support is the weak side. Reactive programs, they argue, need debugging concepts of their own, because a conventional debugger shows a call stack while the question is about a dataflow graph; their Reactive Inspector implements such concepts and outperformed conventional debugging in a controlled experiment [66].

The same picture holds for the Rx family of libraries, which Angular applications use for asynchronous data. Banken et al. report that conventional debuggers have no support for Rx and that developers fall back on inserting console logging; their RxFiddle visualises the structure and the values of an Rx dataflow, and in an experiment with 111 developers it let them finish debugging tasks faster [7]. Alabor and Stolze interviewed 10 professional engineers and observed 4 of them debugging RxJS code, and found that they rely on manually added trace logging rather than on specialised tools; the study is a workshop paper with a small sample, so it indicates rather than establishes the practice [1]. The surface that a tool would have to cover is large: Zimmerle et al. mined RxJava, RxJS and RxSwift usage on GitHub and Stack Overflow and found that 95.2% of the Rx operators are used in practice, with the

stream abstraction itself the largest of the 23 Stack Overflow topics they identified, at 36.4% of the posts [84].

An analysis that aims to recover navigation from an Angular application inherits this boundary. A navigation placed in an effect or in a subscription has a firing condition that is a data dependence in a reactive graph. The read set is visible in the source; the firing is not determined by it exactly, and this analysis does not approximate it. Chapter 3 therefore specifies the modality *unknown* for such edges and keeps the read set as the symbolic guard, which is the treatment the soundiness position of Section 2.6 prescribes for a construct that cannot be decided. No pass in this version emits that modality for an effect or a subscription; the unknown edges it does emit are those whose destination it cannot resolve to a screen.

2.9 Mutation testing

RQ4 asks whether the extracted graph responds to source changes in the constructs it represents, as change impact analysis requires. The method used to answer it is mutation testing, applied in an unusual direction: the mutants are inserted into the application source, and the artefact whose reaction is measured is the extracted graph rather than a test suite. This section reviews the technique, the evidence for the assumption it rests on, its use on JavaScript and web applications, and the reporting practice that Chapter 4 follows.

2.9.1 The coupling effect

Mutation testing inserts small syntactic changes into a program and measures how many of them a test suite detects. The original argument for it is the *coupling effect* stated by DeMillo et al., namely that tests which uncover simple errors are often also effective at uncovering much more complex errors [19]. Without that assumption a mutant would say nothing about the faults a developer actually writes.

Jia and Harman give the standard survey of the field, covering approaches, tools and empirical results; their trend analysis suggests that the techniques and the tools are reaching maturity and applicability [31]. Papadakis et al. survey the following decade of work on the fundamental problems of the technique and list the problems that remain open [58]. The evidence that mutants are a usable proxy comes from Just et al., who used 357 real faults in 5 open-source applications totalling 321 000 lines of code and found that mutant detection correlates significantly with real-fault detection, independently of code coverage, while also discussing the limitations of the analysis [34].

2.9.2 Mutation for JavaScript and web applications

Mutation analysis has been adapted to JavaScript and to web applications, where the cost of generating and running mutants is a central concern. Mirshokraie et al. guide mutant generation with a combination of static and dynamic analysis, directing it at

code that is error-prone or likely to affect the output, in order to reduce the cost and the number of equivalent mutants, and they propose mutation operators specific to JavaScript and the web [53]. The journal version adds a ranking of functions and reports that more than 93% of the generated mutants are non-equivalent and that more than 75% of the surviving non-equivalent mutants fall in the top 30% of the ranked functions [54]. Praphamontripong and Offutt define operators for faults in the way web components interact with each other, covering client-to-server and server-to-server interaction, implement them in a tool called webMuJava, and report a small case study suggesting that the resulting tests find web application faults; the work appeared at a workshop [60].

On the tooling side, Stryker is a mutation testing framework for JavaScript and TypeScript. No peer-reviewed publication describing it was found, and the project does not cite one; its documentation describes implementations for JavaScript and TypeScript, for C# and for Scala, more than 30 supported mutations, a runner-agnostic design and an Apache-2.0 licence [69]. The absence of a paper is the reason the mutation procedure in Chapter 4 is specified in the thesis rather than delegated to a tool's defaults.

2.9.3 Reporting a mutation experiment

Papadakis et al. also give best-practice advice on the use of mutation in empirical studies of testing [58]. Chapter 4 follows it by stating the operator set, how the mutants were produced and sampled, which mutants were excluded and why, and the detection rate for each operator separately instead of one aggregate figure.

This use of mutation puts the thesis in the change impact analysis tradition rather than in test adequacy measurement. Arnold and Bohner define a framework for comparing impact analysis approaches along three dimensions, namely how an approach is used, how it works internally, and how effective it is [4], and the collection edited by Bohner and Arnold is the standard reference for the area, covering source-code dependency analysis and traceability analysis [12]. Chianti is the classic tool of this kind: it decomposes the difference between two versions of a program into atomic changes and reports, for each affected test, the changes that affect it. On a year of version history from one project it found that an average of 52% of the unit tests were affected, each by only 3.95% of the atomic changes [62]. Lehnert's taxonomy classifies impact analysis approaches along more detailed criteria than the earlier frameworks. It is useful for placing a new one, although it appeared at a workshop [44]. A transition graph whose edges carry source witnesses supports the same kind of query at the level of screens, and the mutation experiment measures whether the graph changes when the source it was derived from changes.

2.10 Summary and the gap

Static extraction of navigation models from web applications has been directed largely at server-rendered frameworks, in which a page is a file and a link an anchor;

client-side routing frameworks have mostly been modelled dynamically instead. The literature search for this thesis found three static extractions of a client-side navigation graph, two from a framework's own sources and one from a compiled bundle, each building its graph for a different consumer. Oshima and Kishi's is the one that fixes the terms of the problem addressed here (Section 2.4). It does so, however, only for applications that declare no nested routes, no route parameters and no data-bound navigation, with attribution computed over component containment and the root component's own paths, with no modality on the transitions, and with no measurement of what the extraction fails to read. The other two put the graph to work without narrowing it: one as the coverage denominator of a generated end-to-end suite [65], one as the reachability relation of an access-control check, declared an over-approximation [83]. Rima et al. already read nested and lazily loaded route trees and attach guards to edges as constraints. What this review found no earlier work combining is attribution that follows calls across classes and into services under their provider bindings, a modality on each transition that separates what is unconditionally available from what a guard or an unresolved scope makes uncertain, and a measurement of the resulting loss, classified by construct, on applications large enough for the difficult constructs to occur. The Android literature is the nearest static relative. It attributes navigation by resolving the owning object, and when that resolution is imprecise it falls back on runtime traces. A navigation site in a web application frequently has no owning object that could be resolved. In Android the owning object of a handler usually identifies the window, so attribution reduces to object resolution in the common case; fragments, background tasks and aliased widgets are the exceptions, and the Android work handles them with dynamic evidence. That is why an Android technique cannot simply be reused.

The literature reviewed in Sections 2.5 to 2.9 fills in why the problem is open and what is available for addressing it. The analysis machinery is not missing: abstract interpretation of JavaScript is established [30], source-level analysis over a typed syntax tree is a standard tooling style [8, 59], and the Java literature has shown repeatedly that framework behaviour can be modelled explicitly to recover call edges that a general analysis loses [68, 3, 13]. The other peer-reviewed static analysis in the Angular family targets AngularJS 1.x and checks template and controller consistency rather than navigation [56], and the type information that TypeScript adds narrows a dispatch without settling it, because the type system is unsound by design [10, 64]. The reactive literature marks where the boundary of the approach lies, since the firing of a reactive computation is decided by a graph the runtime builds and not by the program text [15, 7]. Mutation testing supplies a method for asking whether an extracted model reacts to source changes [31, 58], and for a statically extracted navigation graph of a client-side framework it has been applied only to faults injected by hand into constructed applications [57]; an operator-level mutation study of such a graph was not found in this review.

Table 2.1: Prior work closest to this thesis, classified by target, extraction mode, guard representation, and the evaluation basis each reports, which is a reference, a metric or the tool it was compared with, as the work states it.

Work	Target	Static	Guards	Evaluation basis
Yang et al. [80, 78]	Android windows	yes	no	20 apps; manual on 6
Yang et al. [79]	Android callbacks	yes	context	manual vs. dynamic
StoryDroid [14]	Android activities	yes	no	hand-built, 10 apps
GoalExplorer [40]	Android screens	hybrid	no	manual, 36 apps
ProMal [46]	Android windows	hybrid	no	40 apps, F1
UI-CTX [45]	Android widgets	yes	branches	2000 labelled widgets
Ricca and Tonella [63]	server-rendered pages	semi	yes	manual
WebUml [9]	server-rendered pages	hybrid	yes	case study
Tkachuk and Rajan [70]	Struts, JavaServer Pages (JSP)	yes	outcome	not stated
Crawljax [51]	Ajax DOM states	no	no	manual
FeedEx, FRAGGEN [52, 75]	DOM states	no	no	coverage
Kurant [39]	React components	yes	n/a	F1, no routing
Oshima and Kishi [57]	Angular pages	yes	no	6 apps, faults
Rima et al. [65]	Angular workflows	yes	conditions	7 apps, tests run
Zhang et al. [83]	SPA routes, from bundles	yes	no	4 apps, vs. ZAP

Table 2.2: Static analyses that model framework behaviour explicitly, and the mechanism each of them resolves.

Work	Framework	Mechanism modelled
F4F [68]	Java web frameworks	reflective calls, from a specification generated from code and configuration
JackEE [3]	Java enterprise frameworks	components and entry points, declaratively
Jasmine [13]	Spring	dependency injection and aspect-oriented programming
Aurebesh [56]	AngularJS 1.x	identifiers shared between template and controller

Chapter 3

Static navigation analysis and scope attribution

This chapter defines the graph model the measurement uses and the analysis the thesis proposes. Section 3.1 defines the graph model and states the one soundness property claimed for it. Section 3.2 presents scope attribution, the analysis proposed in answer to RQ3, and says which of its parts the prototype implements. Section 3.3 describes the implementation in the extractor. The evaluation is in Chapter 4.

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, namely the first and the last component of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge. That edge carries the strongest modality among them, with* M *ordered* $unknown < may < must$*, so* E *holds at most one modality for each* (s, σ, g, s')*. The paragraph on witnesses below says how the constructs are traced.*

Nodes outside the model. The extractor emits two node kinds that are not screens, and the edge relation does not range over them. An *unplaced origin* stands

Table 3.1: What the published graphs are made of: nodes by kind, where an anchor stands for a whole subtree of screens and *other* counts unplaced origins and dynamic targets together, and edges by modality. These are the attributed graphs; the node column of Table 5.1 counts the graphs of the same snapshot without attribution, which hold no anchors and fewer *other* nodes.

	nodes			edges		
Application	screens	anchors	other	must	may	unknown
realworld	10	2	0	9	56	0
alfresco	27	1	26	0	36	19
ghostfolio	84	6	96	0	266	137
k8s dashboard	67	1	76	125	81	103
bitwarden	98	4	147	1	373	155
mifos	606	14	204	15	10590	206
All	892	28	549	150	11402	620

for a component, with the navigation sites in it, that the analysis could not place under any screen. A *dynamic target* stands for a destination the extractor records as unresolved. Both are kept, because an unplaced site is a loss Chapter 5 has to count and a destination the analysis cannot name is one the run time could still take. Table 3.1 reports how much of each graph they are: 549 of the 1469 nodes in the corpus, 37.4 % of them, and 620 of the 12172 edges reach a dynamic target. A further 73 edges leave an unplaced origin and are held at *may*, so they sit inside a tier Chapter 5 reports while their source is not a screen. An edge into a dynamic target asserts that its source screen has an outgoing navigation whose destination the analysis could not name, and nothing about which screen is reached. Claim 3.1 and must conversion (RQ1) range over the edges whose source and target are both screens.

Modalities. A *must* edge is claimed to fire for every input on which the enclosing code executes. A *may* edge is possible but conditional. *Unknown* marks an edge whose modality could not be determined, and carries the destinations the analysis cannot name. In the materialised graph it labels only edges outside E, namely edges into the `u_` sink of Section 5.5, which records that a screen is left by an event whose target the extractor records as unresolved. The pinned extractor produces 620 such edges of the 12172 in the corpus, in 5 of the 6 applications. Carrying this modality and reaching a node that is not a screen coincide on every edge of every corpus graph, and the census refuses to report a graph in which they come apart. Must conversion (RQ1) and Claim 3.1 are defined over the other two tiers, so the unknown tier is counted in the edge totals and in no rate.

The model also assigns *unknown* to navigations inside signal effects (Section 3.2.5), which may have a screen as target and so lie in E. No pass of this version emits one, so in the measured graphs the two still coincide.

Guards are stored as source text and are never evaluated by the tool. A consumer can read a guard but is not expected to decide whether it holds. This keeps G opaque, so the model does not need to represent application state.

The artefact also records a numeric *confidence* on each edge, a finer version of the same ordering (1 for a must edge, at most 0.6 for a may edge, 0 for an edge into a dynamic target). It is not part of the model and carries no claim this thesis makes. Every figure in this thesis is reported over the modalities, never over the confidence.

Screens and routes. The set S is derived from the route configuration, and not every route contributes a screen. Write *Routes* for the set of routes the application declares and $component(r)$ for the component that route r renders. Then

$$S \;=\; \{\, screen(r) \;\mid\; r \in \mathit{Routes} \text{ and } component(r) \text{ is defined} \,\},$$

where $screen(r)$ is the node for route r, identified by the full path that concatenates the paths of r and of its ancestors. Two kinds of route contribute no element of S:

- A route that declares `children` but no `component` is only a path prefix (Section 2.7). Its path is carried into the full paths of its children.
- A route whose only effect is a `redirectTo` renders nothing.

In Angular a redirecting route is only a redirect: the router refuses `redirectTo` together with `children` or with a `component`. This was read off the router's own configuration validator in Angular 18.2.14, the version vendored with this work, and holds only for that version. It is not true of every client-side router: Vue Router applies a record's redirect only when that record is the last one matched, so a redirecting parent does render its component for a matching child. Section 6.4 returns to this.

Angular also permits two component-bearing routes with the same full path, told apart at run time by `pathMatch` or a `canMatch` guard. The model assumes this does not occur: the extractor keeps the route declared first under a full path and drops the later ones. For Vue and React the drop is declared as a note carrying the file and line of the discarded record; the note is over-stated about the graph, because where both records name the same component the address still renders it. Angular's extractor drops silently (Section 6.4). Two routes with different paths that render the same component give two screens, which is the case that makes the attribution of Section 3.2 necessary.

Witnesses. Every edge of static or runtime origin carries a witness: the file and line of a construct that produced it, or an observation identifier. Where several constructs produce the same edge, one is kept as the witness and the others are recorded as merged witnesses, so every construct behind an edge can be traced (Section 4.2). No edge is added to the base graph without a witness, and edges added manually are kept in a separate overlay. The tool's specification states this as an invariant, and the first evaluation method checks it mechanically.

The soundness claim. The soundiness position that frames such claims is argued in Section 2.6.2. The thesis claims only the following.

Claim 3.1 (Soundness of the must tier)**.** *Every must edge corresponds to a navigation that occurs, for every input on which the enclosing code executes, over the declared route set, under the route-order assumption of Section 6.3, and for a site whose*

component is not placed by a join over an ambiguous selector, within the following execution model: the event is a plain primary activation that navigates the current browsing context, no other listener the event reaches starts a navigation, and no call the reader cannot resolve to a body in the application, nor any work the handler schedules for later, starts one.

The execution model is part of the claim and not a detail of the implementation. Section 6.3 lists the cases it excludes: a call into a library that cancels the navigation by starting another, a second listener on the element or an ancestor, and an activation with a modifier key or into another browsing context. On such an execution a must edge can name a navigation that does not complete, and the claim says nothing about it.

The route-order assumption of Claim 3.1 carries much of the tier: without it the must tier holds 24 of its 150 edges, on 22 of 25 constructs.

What the evaluation tests. The evaluation tests a necessary condition of this claim rather than the claim itself, and the distinction decides how the figures of Chapter 5 may be read. The witness check of Section 4.3 re-reads the source at every location an edge cites and asks whether the construct found there names the destination the edge names. A must edge cannot satisfy the claim if the construct that makes it must names a different destination, so a failure of the check refutes the claim there. Agreement does not establish it. The other constructs a must edge cites are evidence for may and are not judged against the claim.

Demotion conditions. Whether the navigation occurs *for every input on which the enclosing code executes* is decided by the rules that place an edge in the must tier instead of the may tier. A site is demoted to may in each of the following cases.

- **Guards and resolvers.** The navigation runs a guard or resolver, whatever it would decide, because the rules read the route configuration and never evaluate a guard. A navigation runs the guards the router runs: the `canMatch` of every route its address is matched through; the `canActivate`, `canLoad` and resolvers of every route it enters (`canLoad` is taken as run every time); the `canActivateChild` of every route above those; and the `canDeactivate` of every route it leaves. A route the source and the target share is re-run only if its parameters can change or its `runGuardsAndResolvers` policy directs. A resolver demotes the edge as an `await` does, and a guard list the reader cannot bring back to an array demotes it as a guard does.

- **Target and source addresses.** The edge is demoted when any of the following holds:

  - the target is the address the source already displays, which the router ignores or follows only when a parameter or the query differs (held at may as a matter of policy, the safe side);
  - the source is never displayed at its own address because a redirect answers it;
  - two declared routes, or a route and a redirect, can answer the address;

  - the base a relative target resolves against cannot be read;
  - the URL names an auxiliary outlet, whose route the adapter does not read.

  A shell or anchor edge is judged per screen and drawn per screen where the screens disagree.

- **Conditional execution.** The site runs only on some executions of its own handler. This covers a site:

  - under a template control-flow block, a structural directive or a container that renders its content lazily;
  - inside an `<ng-template>` or an International Components for Unicode (ICU) case;
  - under a branch, in a loop or a `catch` clause, in an optional call or a default value, or inside a callback;
  - after an `await`, an early `return` or `throw`, or a `break` out of its block;
  - in a function declared at module level, or a member of its class, that no handler calls unconditionally;
  - in a component mounted under a template condition on that screen, or in a route component behind a conditional outlet.

  A handler is a member the framework calls itself (a lifecycle hook, or a member `@HostListener` registers), or one the template binds to an event as a bare call outside any template condition.

- **A later navigation may cancel it.** The router cancels a navigation when another starts before it has finished. A navigation is therefore demoted when code that runs after it, before the event is handled, may navigate. That code is:

  - the rest of the expression that holds the navigation (the body of a call it is an argument of, a right operand, a branch of a conditional whose condition it is);
  - the statements after it in its block and in every block around it, and a `finally` block around it;
  - what runs once its function returns: what each caller in the class does after calling that function, handler or not, and for a constructor or lifecycle hook the hooks Angular calls after it.

  Such code may navigate when it holds a navigation call, in a branch or a callback as much as at the top, or a call to a member of the class, a function of the file or a method of an injected service whose body does. An awaited navigation has finished before the rest of its function runs, so what follows it there does not count, but its callers go on as soon as the function first awaits, so what they do still counts. A call the reader cannot resolve to a body in the application is taken not to navigate (Section 6.3).

Two things do not count as a later navigation. An exception from an earlier synchronous call is not counted, since every call can throw; an `await` is counted, because a rejected request is an ordinary outcome of the code that awaits it. The router's own fetch of a lazily loaded route is not counted either: a failed chunk load is a fault of the deployment, and the extractor takes the code as deployed.

How far the rules are checked. Unit tests cover each rule in both directions, and a mutation sweep deletes each rule in turn and checks that some test fails (Section 5.5). The tests share the extractor's reading of each construct, and no second classifier re-derives them. The constructs of the final must tier were read against the source by the assistant the declaration describes, with the extractor's decisions in view; this checks that each decision can be defended and is not an independent second reading (Section 5.5). Some defects in the condition rules were found by an earlier review of a development version's tier and not by the witness check (Appendix A.1). What the witness check establishes is therefore narrower than the claim: that every cited site exists in the source and agrees on its target. The stronger half of the claim rests on the demotion rules and is not independently verified.

Scope of the claim. This is the only soundness property claimed in the thesis, and it holds only relative to the declared route set. No such property is claimed for the may and unknown tiers. The witness of a may edge shows that the corresponding call site exists in the source, not that the transition is ever taken. This difference matters for the evaluation. Whether the target found at a witness location agrees with the target of the edge is a meaningful question for must edges, and for may edges whose target is a literal. It is not a meaningful question for edges the extractor produced by over-approximation, and treating it as one leads to spurious failures (Appendix A.1, Table A.1).

3.2 Scope attribution

What is specified and what is implemented. Scope attribution is specified here as a fixpoint over four *input relations*:

- component nesting (N): which component's template instantiates which;
- router outlets: which components lie on the path from the root to an outlet, and below a route's component;
- the call graph with dependency injection (CG);
- dispatch (D): the relations that carry NgRx effects (NgRx is the Angular state-management library) and router event subscriptions.

The fixpoint also uses two derived relations: *Mem* links each component to its members, and T links each callable that holds a navigation site to the entry guards that navigation runs. The prototype implements three of the four input relations: nesting, outlets and the call graph. The dispatch relation D is specified but not

implemented. The guard relation T is not implemented either, and a stand-in replaces it (Section 3.3). The evaluation of Chapter 4 therefore measures this three-relation prototype and not the full model.

Three further parts belong to the model only: guards that return a `UrlTree`, the injector hierarchy that decides which binding of a token applies where, and navigation inside signal effects and observable subscriptions, which the model labels *unknown* (Section 3.2.5). They are specified to define the analysis. The extractor approximates or omits them as Section 6.3 lists. A navigation site that no implemented relation reaches contributes no edge, and the evaluation counts it in the loss category "Attribution, service" of Table 4.3.

3.2.1 The defect being addressed

Without scope attribution, a navigation site is attributed to a screen only if it occurs directly in the component that the route for that screen renders. Navigation sites anywhere else are attributed to no screen. This one cause explains three symptoms:

- shared components contribute no edges, because the application shell places the header next to the router outlet and not inside the subtree of any screen;
- a component mounted at several routes contributes one edge instead of one per route;
- navigations from injected services yield no edges.

These are the symptoms of the baseline, the extractor with the attribution pass disabled (Section 3.3). On it no application is without edges.

3.2.2 Scope

This subsection is the definition of scope attribution that the rest of the thesis points to.

The lattice. Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is an element of the powerset $\mathcal{P}(S)$. The abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y \;=\; X \cup Y,$$

the powerset lattice over the screens. Its least element is the empty scope $\bot = \varnothing$, a site that fires from no screen at all. Its greatest element is $\top = S$, every screen.

True, computed and reference scope. Two different things are called a scope, and the rest of this thesis keeps them apart.

- $\mathit{scope}^*(x)$ is the *true scope* of a site x, the set of screens from which x really can fire. It is a property of the application and not of any analysis.

- *scope*(x) is the *computed scope*, the set of screens the extractor emits edges from for x. In the model it is the fixpoint of Section 3.2.3. The prototype widens it with fallbacks that are not constraints of the model (Section 3.3).

A screen in $\mathit{scope}^*(x) \setminus \mathit{scope}(x)$ is a scope too small, and a screen in $\mathit{scope}(x) \setminus \mathit{scope}^*(x)$ is a scope too large. No true scope is established anywhere in this thesis. For one application a third set is read off the sources by hand, the *structural reference* $\mathit{scope}^R(x)$, which applies the analysis's own relations without its implementation (Section 5.3.4). Agreement between *scope* and scope^R shows that the relations are implemented as stated. It does not show that either equals scope^*. Everywhere else the thesis reports the computed scope and says so.

The empty scope. The empty computed scope is likewise two situations, and conflating them would hide a defect inside a property of the application.

- A site is *unreachable* when $\mathit{scope}^*(x) = \varnothing$: the code exists and no screen can reach it, so contributing no edge is the correct answer.
- A site is *unresolved* when $\mathit{scope}(x) = \varnothing$ because no rule of the analysis reaches it (a navigation in a service nothing is known to inject, or behind a construct the reader could not follow). The true scope is simply unknown.

Both yield $\bot$, so $\bot$ alone does not say which holds. The extractor therefore does not let an unresolved site vanish silently. A site in a component that no rule places is anchored at an unplaced origin, a node that carries the component's file, and its edges are held at *may*. A site in any other component whose scope is empty contributes no edge, and the extractor records a note naming its file and the reason. Either trace makes the unresolved sites countable in Chapter 5 rather than indistinguishable from unreachable code.

Named elements. Three elements of the domain are named, because attribution anchors edges at them. Write $r' \succeq r$ for the reflexive ancestor relation on routes: r' is r itself or an ancestor of r in the route tree. The three elements are

$$\{\mathit{screen}(r)\}, \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

and they form the chain

$$\bot \;\sqsubseteq\; \{\mathit{screen}(r)\} \;\sqsubseteq\; \mathit{subtree}(r) \;\sqsubseteq\; \top.$$

The chain is stated for a route r with $\mathit{component}(r)$ defined. For a prefix route, $\mathit{screen}(r)$ is undefined and only $\mathit{subtree}(r)$, the set of the screens of its descendants, is defined. The chain then reads $\bot \sqsubseteq \mathit{subtree}(r) \sqsubseteq \top$.

These names are a representation device for anchoring edges, not a restriction of the domain. The join of two scopes is their union, not the least named element above both: a component mounted at two unrelated leaf routes has the scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, which is none of the three named elements. The names matter for storage, where an edge anchored at $\top$ or at $\mathit{subtree}(r)$ is kept once rather than once per screen. Table 3.2 assigns a scope to each Angular construct. Figure 3.1

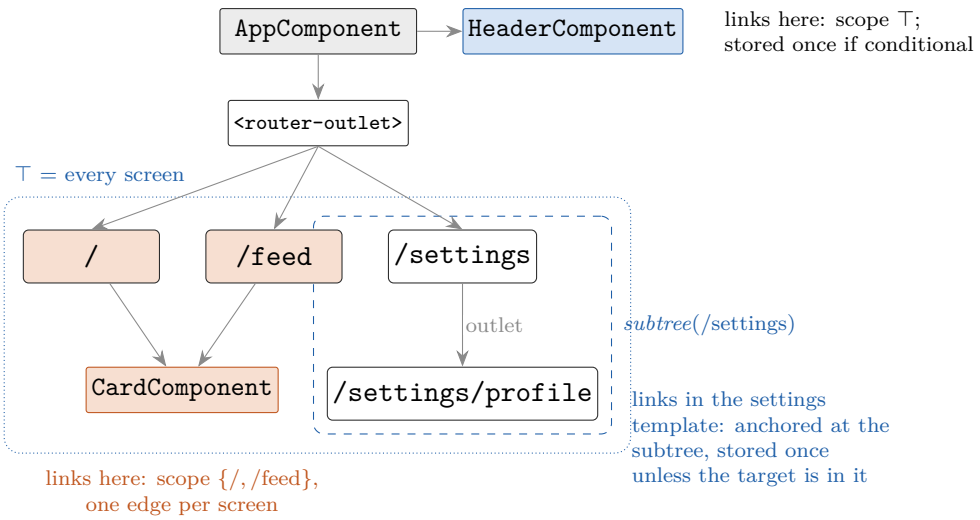


Figure 3.1: Scopes in a small application. The header is rendered next to the outlet, so its links fire from every screen. The card is rendered inside two unrelated screens, so its scope is the union of the two, which is none of the named elements. The settings template is rendered on its own screen and on its child screen, which is exactly *subtree*(/settings). Grey arrows show component nesting and outlets.

shows the three kinds of scope in a small application, and Figure 3.2 shows the domain for three of its screens.

Computing the scopes. The row for services is why a simple traversal of the component tree is not enough. A service has no scope of its own: its scope is the join of the scopes of the places that call it, which may themselves be services. Attribution is therefore computed as a fixpoint over the relations of Section 3.2.3. The outlet relation gives the initial scopes. Scopes are propagated along the nesting relation N, from each component to its own members along Mem, along the call graph CG, along the dispatch relation D, and from each navigation site to the entry guards of the routes its target lies under along T. This is a monotone dataflow problem in the sense of Kildall [38, 55], and two properties of it do separate work.

- Finite height gives termination. The height of $\mathcal{P}(S)$ is $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$.
- Monotonicity gives the result. The iteration reaches the least solution of the constraint system, independent of the order in which nodes leave the worklist. The transfer functions are monotone because the only operation they apply to a scope is a join, and union is monotone in both arguments.

A worklist solver re-examines a node once for each rise of its own scope, and the work for a node is proportional to its out-degree in $N \cup Mem \cup CG \cup D \cup T$. With scopes held as bit sets, so that a union costs $O(|S|)$, the solver for the full model therefore runs in $O(k \cdot |S| + |S|^2 \cdot (|N| + |Mem| + |CG| + |D| + |T|))$, where k is the number of seeded nodes (the routed components, the shell, the `canDeactivate` guards and the interceptors). The bound covers the solver only: building the relations, which includes parsing the sources and resolving calls, is a separate cost not bounded

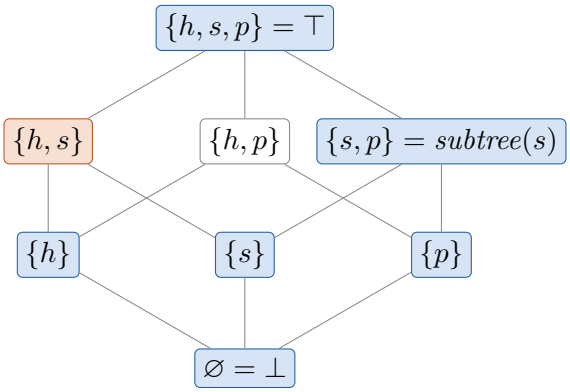


Figure 3.2: The scope domain $\mathcal{P}(S)$ for three screens: h for `/`, s for `/settings` and its child p for `/settings/profile`. Shaded elements are the named scopes at which edges are anchored. The join is set union: a component rendered in both h and s gets $\{h\} \sqcup \{s\} = \{h, s\}$, marked in orange, which is not a named element, so its edges are stored per screen. The height of the lattice is $|S|$.

here. The implemented part has D and T empty, and the restrictions, fallbacks and stand-ins of Section 3.3 are computed outside the worklist. Propagation is needed because scopes travel transitively and the scope of one node can rise once for each screen that reaches it. Cycles are not the reason: the relations may contain cycles, since mutually recursive templates are legal, and the worklist handles them.

3.2.3 Input relations

Four relations, in order of increasing difficulty, provide the input to the fixpoint.

- *Component nesting.* $A \to B$ when A's template instantiates B's selector, resolved within the correct import scope. Matching selector text globally is unsound, since two components may share a selector string in different scopes.
- *Router outlets.* Components on the path from the root to the outlet form the shell and take $\top$. Components below the component of route r take its scope $subtree(r)$. A named outlet carries a second mount, which makes auxiliary routes representable without separate treatment.
- *Call graph with dependency injection.* A call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17], with Angular's provider declarations as the source of bindings, as recommended for framework-heavy code [3]. Where a token admits several implementations the analysis joins over all of them, which is one source of *may* modality.
- *Dispatch relations.* A dispatch is treated as a call-graph edge from every dispatcher to every handler, which admits NgRx effects and router event subscriptions under the same fixpoint.

Table 3.2: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$\mathit{subtree}(r)$
component mounted at routes $r_1, \ldots, r_k$	$\mathit{subtree}(r_1) \sqcup \cdots \sqcup \mathit{subtree}(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
member (method, property initialiser, lifecycle hook) of a component	the scope of the component
method of an injectable service	join of the scopes of its callers
`canActivate`, `canActivateChild` or `canMatch` on route r	join of the scopes of the navigation sites with a known target in $\mathit{subtree}(r)$, an approximation of the screens a navigation into r starts from
`canDeactivate` on route r	$\mathit{subtree}(r)$
HTTP interceptor	$\top$
NgRx effect for action a	join of the scopes of every dispatcher of a

Algorithm 1 gives the computation, and its propagation steps are the transfer-function applications bounded above. A step adds a screen to a scope only when an edge of N, Mem, CG, D or T carries it there, and scopes only grow. What the algorithm computes is therefore the least solution of the constraint system that Table 3.2 states. In the model a component that is both mounted by a route and rendered in the template of another component holds the join of the two scopes. Section 3.3 reports where the prototype departs from this: it does not propagate into a component that a route mounts, and it adds fallbacks that are not constraints of the table, so its result is not the least solution of this system or of any restriction of it.

3.2.4 Guards as program points

A guard that returns a `UrlTree` is itself a navigation, typically to a login screen. Its source is not the guarded route. A `canActivate` or `canMatch` guard on r runs while the router leaves the screen the navigation started from, so the redirect edge leaves that screen, the origin of an edge into $\mathit{subtree}(r)$. Only a `canDeactivate` guard redirects from a screen of $\mathit{subtree}(r)$ itself. Treating guards only as labels on inbound edges misses this edge class entirely.

Further, the guard of an edge in the model is the conjunction of the conditions on the containment path from screen to call site (structural directives) and the guards the router runs on that navigation (Section 3.1). The prototype does not materialise that conjunction in the measured graph. A structural condition in a template, or an enclosing `if` or conditional expression, demotes the edge to may but is not written

into its guard, so the guard of a screen-to-screen edge holds only the router's guards. Only the optional mode that anchors navigations at the controls that trigger them, which no table of Chapter 5 uses, keeps the enclosing condition. The evaluated graph therefore records that a navigation is conditional without recording the condition. In the baseline, for example, a link rendered only inside `@if (isUser())` was labelled with the authentication guard of its target route, which any logged-in user satisfies, including users who never see the link.

3.2.5 Boundary

A navigation inside a signal effect has a firing condition that is a data dependence on a reactive graph. As Section 2.7 notes, only the read set is statically visible. The model gives such edges the modality *unknown* and records the set of signals read by the effect as the symbolic guard. No pass in this version emits one. The analysis does not try to determine when these edges fire. It marks this limit explicitly instead of giving a result that looks more precise than it is.

3.2.6 Anchored edges

If the result of attribution were stored explicitly, a single link in the application shell would produce one edge for every screen. Where every site has a single target, the size of the graph would then be bounded by $|S| \cdot |\mathit{sites}|$, where *sites* is the set of navigation sites. For the largest application in the corpus that bound is the product of its screens (Table 5.1) and its 1343 constructs. A site whose target is only partly known can reach several screens, so in general

$$|E_{\mathit{expanded}}| \;\leq\; \sum_{x \in \mathit{sites}} |\mathit{scope}(x)| \cdot |\mathit{Targets}(x)|,$$

which is $O(|S|^2 \cdot |\mathit{sites}|)$ when a site of scope $\top$ reaches every screen. Anchoring removes the factor that comes from the source scope, not the one that comes from the targets. Each edge is stored once, anchored at its scope, and the set of edges leaving a screen is computed when requested:

$$\mathit{edgesFrom}(r) \;=\; E_{\top} \;\cup\; \bigcup_{r' \succeq r} E_{\mathit{subtree}(r')} \;\cup\; E_{\{\mathit{screen}(r)\}},$$

where E_X is the set of edges anchored at scope X. The term for $r' = r$ contributes $E_{\mathit{subtree}(r)}$ itself. The query is made for a screen, so $\mathit{component}(r)$ is defined, while r' ranges over all ancestors of r, prefix routes included. An edge whose scope is some other screen set is stored at each of its screens and reaches $\mathit{edgesFrom}(r)$ through the $E_{\{\mathit{screen}(r)\}}$ term. Figure 3.3 contrasts the two representations.

Claim 3.2 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set.*

The claim is tested rather than proved: Section 5.3 compares the two sets on the corpus and on fixtures.

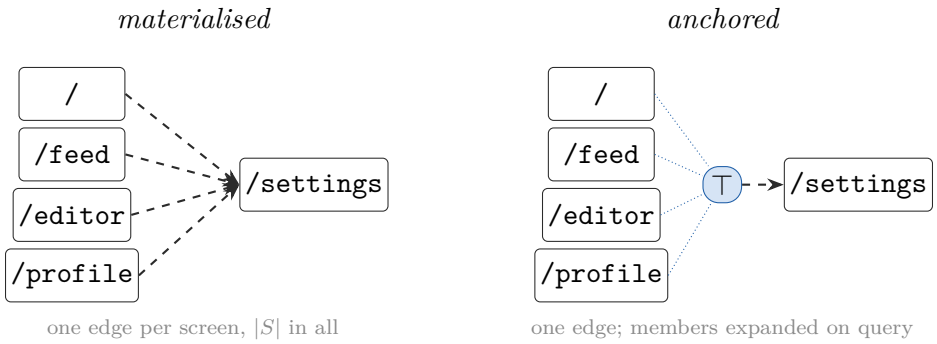


Figure 3.3: A header link to `/settings`, rendered under a condition and so held at may on every screen, stored two ways. Materialised, it becomes one edge per screen, including one from `/settings` itself, which is not drawn. Anchored, it is stored once at the scope node $\top$, and *edgesFrom*(r) adds it to the edges of any screen r when asked. Claim 3.2 states that both forms describe the same transitions. Only a reason for may that every screen shares keeps the anchored form: an unconditional link is demoted from `/settings`, the screen it already displays, but not from the others, so the extractor stores it as one edge per screen.

3.3 Implementation

The extractor, *uigraph*, was built for this thesis and is available as open source. It consists of a core that is independent of any particular framework and a set of adapters, one per framework. The core defines the graph model, its representation on disk, and the operations over graphs. Each adapter maps the source constructs of one framework onto the shared model, following a fixed interface. The Angular adapter reads route configuration, templates and class code with the TypeScript compiler API, and emits nodes for screens and edges for `routerLink` attributes and `Router.navigate` calls, each with the rule identifier and source location that produced it as witness.

Scope attribution is a separate pass in the Angular adapter, enabled with the option `--attribution`. With the option off, the extractor produces exactly the baseline edge set; this was checked on every application in the corpus. The pass implements three of the four input relations of Section 3.2: component nesting, router outlets and, as the second departure below describes, the call graph. The dispatch relation D is not implemented, and neither is the guard relation T, which a stand-in replaces (below).

Component nesting. Every component in the application, excluding test hosts and stories, is read with its element selectors and template. An edge $A \to B$ is added when A's template contains an element with B's selector. When a selector belongs to more than one component, the candidates are narrowed to those that A's file imports. If none is imported, the analysis joins over all of them and records a note. Edges from such a join are not capped at *may*, so a site in one candidate can be placed on a screen that renders another, and Claim 3.1 excludes such edges (Section 6.3). The corpus has such joins in k8sdash and bitwarden, and no must edge of the final graphs cites a construct placed through one. An attribute selector

is matched the same way, so a directive has a scope as a component does.

Router outlets. The route tree is recorded during route extraction. A route component's template is rendered on its own screen and on every descendant screen, so its navigation sites get the scope $subtree(r)$. A component that is not routed, is not placed on a screen by the nesting relation and holds the primary `<router-outlet>` is the application shell, and everything it renders gets the scope $\top$. A component that is not routed and not referenced, but whose subtree contains the outlet, is most likely mounted by a route the extractor cannot see, for example through a helper function that wraps a route table. It is also treated as shell, but its edges are capped at *may*, so that Claim 3.1 is not put at risk by a scope that may be too large.

Departures from Algorithm 1. The pass departs from the algorithm in two respects.

The first is that it does not propagate a scope into a component that a route mounts. A routed component keeps $subtree(r)$ and does not also receive the scope of a component whose template renders it. Traversal does pass through a routed component, so a component nested beneath it still receives the scopes of the components above. Taken alone, this restriction can only shrink a routed component's scope relative to the full system, so a navigation site in a routed component can be attributed to fewer screens than the model allows, never to more. Two other rules can enlarge a scope beyond the model's:

- components that nothing references and no route renders, but whose subtree holds the primary outlet, are treated as shell and given $\top$, with their edges capped at may;
- an ambiguous selector that no candidate's import resolves joins over all candidates, which Claim 3.1 excludes.

A scope that is too small loses edges and cannot invent them, which is why the restriction cannot put Claim 3.1 at risk. It costs recall, and the constructs it strands are counted in Chapter 5.

The second departure is in how the call graph of Section 3.2.3 is read. The pass ranges over the members of classes, and a member of a component runs on that component's screens. A call `this.f.g()` is resolved through the declared type of `f` (a constructor parameter, typed field, `inject(T)` field or `@Inject(TOKEN)` parameter) to the class that type names, to the classes a provider binds it to with `useClass` or `useExisting`, or, for an abstract class no provider binds, to its subclasses in the application. This is class hierarchy analysis [17] restricted to the declared type. Bindings are read across the whole application rather than relative to the injector that resolves them, so where two injectors bind a token differently the join covers both, which can only widen a scope. A factory or value provider is not read, and a call on a type declared outside the application contributes no edge.

The seeded call-graph iteration on its own reaches the least solution of its constraints. The scopes emitted for service sites add two fallbacks to it:

- a member no caller reaches falls back to the join over every component that

injects its class, the per-class approximation that the *class* level of the ablation applies to every service (Section 5.3.3);

- a member the iteration does reach receives more screens where a caller the graph does not see reaches it: the join of that caller's class is added, or this class's own join if nothing places the caller.

Neither fallback is a constraint of Table 3.2, and the first is not monotone in the call graph: adding a call edge can narrow a scope. The emitted service scopes are therefore the least call-graph solution widened by these fallbacks, and not the least solution of the constraints in Table 3.2. Service sites are held at *may* either way, because the calls on the way carry conditions the analysis does not follow.

Anchored edges. Edges with scope $\top$ are stored once at a node of kind `scope`. Edges whose screen set is exactly $\mathit{subtree}(r)$ for a screen r with at least one descendant are stored once at a scope node listing the members of that subtree. A prefix route that is not itself a screen is not anchored, and other screen sets are stored per screen. The core provides the query $\mathit{edgesFrom}(r)$ and a function that materialises all anchored edges, used to test Claim 3.2.

Stand-ins for D and T. A navigation in an NgRx effect has no scope from the model itself, because D is not implemented. Two approximations stand in where the model has a relation the pass does not implement. Section 5.3.3 withholds them below the *full* level, so no lower level counts their sites.

- *Guards (for T).* A guard is given the screens of the routes that name it, as a bare identifier, in `canActivate`, `canActivateChild`, `canMatch` or `canDeactivate`: the screen of each such route, not $\mathit{subtree}(r)$, and nothing for a componentless prefix. For a `canDeactivate` guard that is narrower than the model's scope. For an entry guard it is not the model's rule, since the redirect leaves the screen a navigation starts from, so the stand-in can be too small where navigations into the route start elsewhere and too large where they do not start at the route itself.

- *Effects (for D).* An effect, being registered for the lifetime of the application, is given $\top$.

Edges from both are capped at *may*. A class the call graph does not reach and nothing injects still has no scope, and the evaluation counts those in the loss category "Attribution, service" of Table 4.3. Unit tests cover each case above, including that the baseline is unchanged when the option is off.

One change was made to the baseline itself. For components with an inline template, the witness of an edge pointed at the line where the template starts rather than at the construct. It now points at the construct's own line. This can change witness locations but no edge. On the corpus, the baseline graphs with and without this change are identical, including witnesses.

3.4 Summary

The chapter defined a graph of screens and edges with three confidence labels, and one soundness claim for the must tier. The claim is relative to the declared route set, the route-order assumption and an execution model, and the evaluation tests only a necessary condition of it. Scope attribution assigns each navigation site a set of screens in a powerset lattice and computes the sets as a least fixpoint over the input relations. The prototype implements three of the four input relations, with the dispatch relation specified but not implemented and a stand-in for the guard relation, and it widens the scopes with fallbacks that are not part of the model. Chapter 4 describes how the analysis is measured.

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$. The relations marked *model only* belong to the full analysis and are not implemented in the prototype evaluated in Chapter 5; the prototype's other departures, among them the stand-ins for guards and effects and the rule for routed components, are listed in Section 3.3.

Require: routes R with route tree; for each route r its entry guards $\mathit{entry}(r)$, named in `canActivate`, `canActivateChild` or `canMatch`, and its `canDeactivate` guards $\mathit{leave}(r)$; components C; nesting $N \subseteq C \times C$; HTTP interceptors I; call graph CG; dispatch relation D; guard relation T, linking a callable x to $g \in \mathit{entry}(r)$ when x holds a navigation site whose target lies in $\mathit{subtree}(r)$

Ensure: $\mathit{scope}(x) \in \mathcal{P}(S)$ for every component and callable x; the callables include the members of components and of services

for all x **do** $\mathit{scope}(x) \leftarrow \varnothing$
end for
for all $r \in R$ and c the component of r or a layout wrapping it **do**
    $\mathit{scope}(c) \leftarrow \mathit{scope}(c) \cup \mathit{subtree}(r)$ ▷ own screen and outlet descendants
end for
for all c on the nesting path from the root component to the holder of the root-level primary outlet **do**
    $\mathit{scope}(c) \leftarrow S$ ▷ application shell, $\top$
end for
for all $r \in R$ and $g \in \mathit{leave}(r)$ **do**
    $\mathit{scope}(g) \leftarrow \mathit{scope}(g) \cup \mathit{subtree}(r)$ ▷ *model only*: runs as the user leaves r
end for
for all HTTP interceptor $i \in I$ **do** ▷ *model only*: registered for the application
    $\mathit{scope}(i) \leftarrow S$
end for
$W \leftarrow$ every x with $\mathit{scope}(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        PROPAGATE(x, y)
    end for
    for all members m of x, if x is a component **do** ▷ a member runs where its component is shown
        PROPAGATE(x, m)
    end for
    for all y with $(x, y) \in CG \cup D \cup T$ **do** ▷ CG: member level, bindings read application-wide; D, T: *model only*
        PROPAGATE(x, y)
    end for
end while
procedure PROPAGATE(x, y)
    if $\mathit{scope}(x) \not\subseteq \mathit{scope}(y)$ **then**
        $\mathit{scope}(y) \leftarrow \mathit{scope}(y) \cup \mathit{scope}(x)$; add y to W
    end if
end procedure

Chapter 4

Evaluation setup and research material

This chapter describes how the analysis of Chapter 3 is evaluated. Section 4.1 presents the corpus and argues that it is a valid basis for the measurements. Section 4.2 defines every ratio and measure in one place. Section 4.3 describes the five evaluation methods, what each can and cannot show, and how the results are kept reproducible. Chapter 5 reports the results.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria, applied in order of increasing cost so that a candidate could be rejected cheaply. The criteria were:

1. a real application rather than a library or template;
2. a nested, lazily loaded route tree;
3. guards present, since guards produce conditional transitions;
4. end-to-end test configuration for a browser driver;
5. retrievable history in which tests and source change together;
6. an active repository.

Six candidates were retained (Table 4.1), each fixed to a specific commit so that the measurements can be repeated on the same code.

Criterion (4) was checked as configuration, not as a usable suite. At the pinned commits only alfresco has specifications the test-source cross-check can read, so that method covers alfresco alone. Ghostfolio and bitwarden carry no browser-test configuration at all, so for them the criterion was not met. It was therefore not applied uniformly: `gravitee` was rejected for a thin UI suite while two applications with none were kept.

Four candidates were rejected. They are listed because the rejected candidates are part of the selection protocol. The counts below were read from the candidate repositories and are not generated.

Table 4.1: The corpus. Routes are the route entries reached by the independent route parser of Section 4.3, as it stands in the final version (Section 5.5). A count marked ≥ is a lower bound: the independent route parser records a blind spot for that application, so the entries behind it are not in the figure, and Table 5.2 gives how many blind spots each application has. The marks are on alfresco, ghostfolio, bitwarden, mifos, on these grounds: children not resolvable to a route array; matcher route: path not statically knowable; spread route source: not followed.

Application	Commit	Routes	Role in the study
gothinkster/angular-realworld-example-app	`dd99ed2c`	14	size floor; all literal paths
alfresco/alfresco-content-app	`b88d47f4`	≥57	87 Playwright specs including a navigation suite
ghostfolio/ghostfolio	`9884fa0b`	≥135	constant-valued route paths; self-hostable
kubernetes/dashboard	`0ba796d9`	140	mid-size control
bitwarden/clients (web)	`e40ce6f0`	≥168	many guards, guard factories among them
openMF/web-app	`7f781796`	≥909	size ceiling

- `angular/components` is a component library with no product router.
- `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none of which drive the user interface.
- `gravitee` declares 618 routes but has 14 UI specifications.
- `SAP/spartacus` has a large Cypress suite, but its routing is configuration-driven in a way the extractor does not support, so it was set aside for later work. That reason is tool support, which is not one of the six criteria, so this rejection rests on a ground added after screening began.

The screening also showed that route-tree size and end-to-end test density were inversely related: the largest applications had few or no tests that exercise the user interface, and the application with the richest navigation suite was of medium size. No application therefore provides both a large route tree and execution-based ground truth. The methods that need test suites are applied only to the applications that have them, and the other methods to all six.

4.1.2 Validity of an open-source corpus

Kalliamvakou et al. show that many GitHub repositories are inactive, personal or not software projects, that stars do not measure quality, and that forks inflate counts

[35, 36]. They recommend filtering candidates by explicit criteria and reporting them, which the selection above does. The remaining threat is external validity: open-source applications may differ from proprietary ones in size, routing style and navigation mechanisms. In addition, I selected the corpus and wrote the extractor, which introduces a possible selection bias. Three measures address these threats:

- The selection protocol is reported in full, rejected candidates included. The criteria were fixed before any results were obtained, but one candidate was rejected on tool support and two retained applications do not meet the test-suite criterion.
- The corpus spans nearly two orders of magnitude in the number of routes, so an effect of size is visible and not averaged away.
- The commits are fixed and the harness is released with the thesis, so the measurement can be repeated rather than taken on trust.

None of these addresses a further threat: uigraph was developed on this corpus and is measured on it, with no held-out application (Section 6.2), so the figures are fitted to it. A case study on a proprietary Angular application, reporting only aggregate figures, is planned as the most direct response (Chapter 6). There is no standard benchmark corpus for single-page applications either; evaluations in this area use collections chosen for each study [37]. The corpus and its manifest are therefore an output of the thesis.

4.2 Metrics

Each research question is answered with one main measure. This section is the home of every ratio the thesis reports. Table 4.2 lists them, and the paragraphs after it define the terms the table uses.

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code, in the application source excluding tests. Constructs are found by a separately written parser that shares no code with the extractor. Table A.1 qualifies what that independence covers: for the route-path forms added during development, the two resolve the same constructs by the same rules, and three of the independent route parser's corrections were made because it disagreed with the extractor. Attributes such as `routerLinkActive` are not constructs.

Must conversion (RQ1). Must conversion is the fraction of in-scope constructs that are witnessed by at least one must edge. Four rules make it precise.

- A construct is matched to a witness on its own line or on a line either side, to absorb offsets in the extractor's locations. Two constructs on adjacent lines can therefore both be credited by one witness, and the ratio is an upper bound at line granularity.

- Query-only navigations and the two exclusions of Table 4.3 are left out of the denominator as well as the numerator.
- When several constructs produce the same edge, the extractor keeps the most certain as the edge's witness and records the others as merged witnesses, each with the modality it earns on its own. A construct counts as converted only where it earns must itself. The tables of Chapter 5 count this way.
- The ratio is computed over constructs and not edges, because the number of edges depends on representation: one construct can be stored as one anchored edge or as one edge per screen (Section 3.2).

The other ratios. More than one of these measures has been called coverage in the literature this thesis builds on, and they are not interchangeable: on this corpus they differ by more than an order of magnitude (98.8 % of sites accounted for against 1.4 % converted to must). Four points complete Table 4.2.

- Node coverage rises if less of an application is read, so the blind-spot count is printed beside it. Over the addresses the running framework mounts it is a different and smaller figure (Section A.6).
- The recovered share widens the must-conversion numerator to an edge of any tier. A construct that a must edge cites only as evidence for may does not count towards must conversion.
- On an application whose screens sit behind guards the two are far apart: almost every transition is conditional, so a corpus can be mostly recovered while almost none of it is deterministic.
- The accounted share alone measures whether the losses are declared, not how much is recovered.

Coverage without a qualifier means node coverage. The gap in recovered constructs is called the recovery gap, and the loss tables measure it.

Agreement between a scope and the rendered interface (RQ3). The measures above cannot be asked the question the contribution turns on: is the set of screens that attribution computes for a site the set that site can really fire from? The *rendered-interface check*, run on the harness of the runtime route oracle (Section 4.3), answers one direction. The harness navigates to each screen the graph holds, lets it render, reads the in-app links the DOM carries, and compares them with the edges the graph's per-screen query yields there.

The two directions are not equally decidable.

- A link rendered at a screen where the graph has no such edge is unambiguous: the site was attributed to too few screens or never read.
- An edge the graph has and the DOM does not show is weaker. A template may render links conditionally and this harness signs in to nothing, so an absent

link may be correctly attributed. And where a target could not be resolved to one address, every address the static prefix admits is recorded at reduced confidence, which is not a claim that the screen links there.

This direction is therefore judged only over predictions that name a destination exactly, and their number is published so that no share is read off a denominator that cannot support one. Three bounds apply.

- Only links with a readable target are seen: an anchor's `href` or a static `routerLink`. A `Router.navigate` call in an event handler leaves nothing in the DOM, so the sites judged are the template-sourced ones.
- A screen whose guard redirects to a sign-in page without a backend renders another screen's links and cannot be judged. Such screens are counted apart; on this corpus they are most screens (Section 5.3.1).
- The harness serves one built directory at the server root, which ghostfolio's per-locale build does not fit. Ghostfolio is measured by every other method and by none of this one.

The ablation and the structural reference (RQ3). Two measurements ask whether the attribution computes a set of screens that agrees more closely with a structural reference than a simpler analysis does, or only a larger one.

The ablation withholds the mechanisms of Section 3.2 one at a time and extracts the whole corpus at each level:

- *Owned*: a site belongs to the screen whose routed component's own file holds it, the direct ownership of the Android literature.
- *Nesting*: adds component containment through template element selectors.
- *Rooted*: adds the shell rule, so a component the root renders is attributed to every screen. Oshima and Kishi assemble their page information from the root component as well (Section 2.4), so this level measures how far the prior work reaches here.
- *Outlet*: adds router-outlet descent.
- *Full*: adds the member-level call graph, with its fallback join and the stand-ins for guards and effects.

Each level adds to the one before it, so a step measures what a mechanism adds given those below it, and the *full* step is not one mechanism alone. A sixth level, *class*, runs beside the nested ones and replaces the call graph with the per-class rule, which joins a navigating service over every component that injects its class. The nesting holds because the implementation withholds results rather than computing them differently. The harness checks this on the real graphs: a (site, screen) pair present at one level and absent at the next is a defect in the switch, and no table is written.

The structural reference is scored in both directions, which no other method allows. For one application, the screens each navigation site fires from were read off the sources by hand, under the rules in the file's method field, and recorded with a derivation per site in `route-oracle/ground-truth/realworld.json`. It is the structural reference of Section 3.2.2, called the ground truth below. A pair the reference has and a level does not is a scope too small by the reference; the converse is a scope too large. Four limits apply.

- It is one application, chosen because it is small enough to read exhaustively.
- It is the author's reading, so it is not independent of the tool's author. It is published with its reasoning so a reader can check it.
- Its rules are the analysis's own relations: a component is on every screen whose routed component renders it through template selectors, the shell's components are on every screen, and a navigation in a service member is on the screens of its callers. A template condition or guard that admits a screen only in one sign-in state does not narrow a scope under these rules. The ground truth therefore tests whether the implementation applies the relations correctly, not whether they match what the running application renders.
- It is checked in one direction only: the scorer reports every site the graph places that the ground truth omits.

The end-to-end navigation check (RQ3). The structural reference applies the analysis's own relations, so it cannot show that they miss a navigation the application performs. This check takes its reference from execution. Two applications ship an end-to-end suite that can be run here: realworld, against the public demonstration backend, which is not pinned, and mifos, against a local Fineract server started from the repository's own compose file with the image pinned by digest. A small module added to each application logs every navigation the router completes while the suite runs, with the route patterns of the screen left and the screen reached. Nothing else is changed.

An observed pair is such a source and target pattern for a navigation the application itself triggered. Back-button navigations and the first screen after a full page load are excluded, so no pair spans a reload. A pair is *contained* if any edge, must or may, joins the two screens once scope nodes are expanded to their members, with parameter names ignored; an edge to an unknown target never counts. A hit is any such edge, not the edge of the site the test used, so the check does not test the scope of an individual site. Redirected navigations are compared on the address they asked for, as preregistered. The scorer's first departure from this rule is listed in Appendix A.1, and its figure is not used.

A missed pair is classified from the graph alone, by rules over the graph so that no assignment is made by hand. The classes are tested in this order and the first match wins: *redirect* (the navigation was redirected and the graph holds the pair on the address it landed on; the preregistered redirect class), *base* (the target is reached from a screen above the source but not from the source, as a relative link resolved

against the wrong route appears), *dynamic* (the source screen's routed component, or one above it, holds a site whose target the extractor records as unresolved), and *other* (none of these). The base and dynamic classes were defined after the misses had been read. Because of the order, the dynamic count is what is left after base. On mifos every miss meets the dynamic test whatever the order, so that count says little alone, and Section 5.3.5 names the sites.

The counting rules, three validity checks and predictions for each application were written down before any log was read, except for that departure. The validity checks are that the instrumented and the plain suite pass and fail the same tests, that at least ten distinct pairs are observed, and that a second instrumented run observes the same pair set. The check measures recall on the transitions the suite exercises and nothing else. The graph is an over-approximation, so that recall is containment, and its weight depends on how many (screen, screen) pairs the graph holds, which is reported beside it. A graph edge no test takes is not an error, so the check cannot measure precision; and one run cannot refute a must edge, so it says nothing about the soundness property.

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct that is in scope, cited by no must or may edge and not accounted for is *lost*. It is assigned to exactly one loss category by the first rule in Table 4.3 that applies, so the categories partition the lost constructs and their sizes can be compared within an application. Named outlets are removed before the assignment, and guards are measured outside it. The categories Constant target, Node missing and Other were not put through the admission rule that maps a category to a defect class (Chapter 5).

Guard defects do not appear in the table, since a wrong guard mislabels an edge rather than removing it. No method here checks a guard label directly: the witness check compares the construct, its target and the destination of a must edge, not whether the transition is unconditional. Guard handling is exercised only by the guard-removal mutation operator and by the source review of Section 5.5. The guard defect was found by red-team review of the graph and the source, not by any check above.

The category *other* is reached in unrelated ways: a template site whose owner cannot be resolved, a route component whose route has a node elsewhere, and a construct no rule matches. Its size counts constructs left for inspection by hand, not one cause. The nested branch decides a category from the *owning* route component, so a construct can be assigned to a node category and still be converted once attribution gives it a scope through the shell or a subtree (Section 5.3).

Change in recovery and must-tier precision (RQ3). Must conversion and the loss breakdown are measured on one commit of the extractor, run twice on identical inputs with the attribution pass disabled and enabled. One commit rules out any difference between the runs other than the pass itself. Section 3.3 reports that with the pass off the extractor reproduces the baseline edge set exactly on every application.

Must-tier precision is the fraction of must edges whose witness line contains a construct of the claimed kind with the claimed target, *and* whose destination node

sits at an address the independent route set declares. The second half is needed because checking only the witness against the edge's destination takes the destination for granted, so a node minted at an address the application never mounts would pass.

An edge whose destination the check cannot read is neither a pass nor a failure, so precision is stated as bounds: the share of must edges that pass, over all of them, and the share that do not fail. RQ3 is answered positively only if the recovered share and must conversion, over the same construct denominator, do not fall, the recovered share rises, and must-tier precision does not fall. This is read as no detected failure in either run. It is not a precision that did not fall where the check reads only part of the tier, and the bounds of the two runs cover different sets of must edges, so they are not compared as rates.

Node-level recall. Node-level recall is the fraction of distinct URLs named in an application's own end-to-end test sources that have a matching node in the graph. It is a cross-check on node extraction, which all edges depend on.

Detection rate (RQ4). The detection rate is the fraction of injected source changes that produce the predicted change in the graph, reported per mutation operator with its n. A change whose construct the graph does not contain is excluded and not counted as a miss, since it can only repeat a recovery gap that the loss tables measure. The rate is therefore conditional on recovery, and the number of excluded mutants is reported alongside it.

Cause trace (RQ5). Each defect class of Table 5.3 is traced to its cause in the source of the applications. For each cause the analysis records one of three outcomes.

- The final version reads the cause. The information was in the source, so the loss was the tool's and not the application's.
- The final version does not yet read the cause, although a static reader could.
- The cause lies beyond a static reader of a route table.

The result is a list of causes by outcome, not a ratio, and every site that yields no transition still carries a recorded reason (Section 5.5).

4.3 Evaluation methods

Five methods are used; RQ3 adds four further checks (the rendered-interface check, the ablation, the structural reference and the end-to-end check), which Table 4.4 does not list. None establishes correctness alone, so Table 4.4 states what each can show and where it is blind. Figure 4.1 shows which part of the graph each method checks and where its ground truth comes from.

Four of the five read the application's sources. That shared footing is their common weakness: two readers of the same text can be wrong in the same way, and agreement between them is then no evidence. The fifth, the runtime route oracle, was added for that reason.

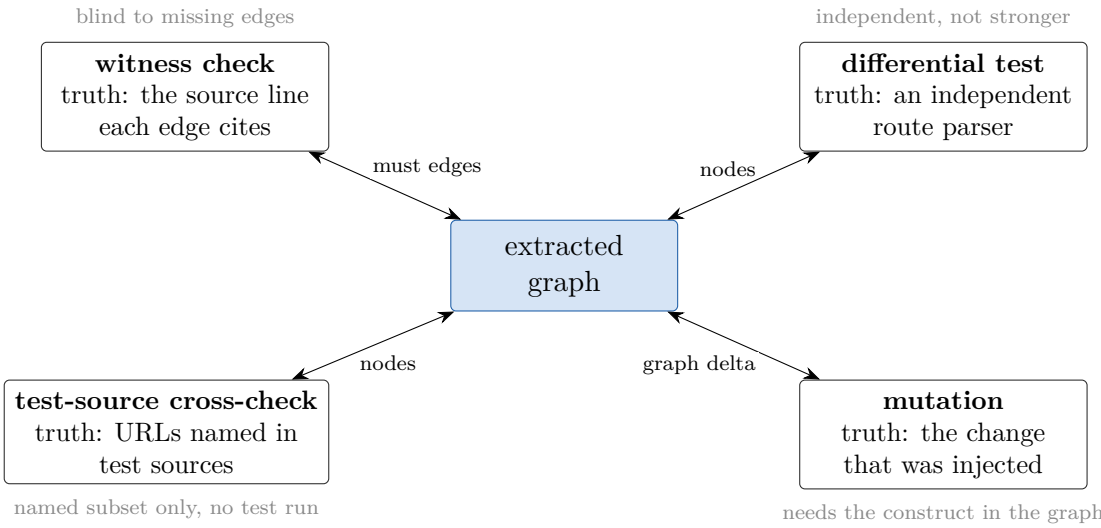


Figure 4.1: The four evaluation methods that read the applications' sources, the part of the graph each one checks, and where its ground truth comes from. The fifth, the runtime route probe, is not drawn here because its ground truth is the running application rather than any part of the graph. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.4.

Table 4.4: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-tier precision. For every must edge, the witness file is re-parsed at the claimed line, the construct kind and target are compared, and the destination node's address is tested against the independent route set.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists. The destination test is only as good as the route set: where both tools read a table at the wrong prefix, they agree on a wrong address; Section A.6 records such addresses.
Differential test	Disagreements with a separately written route-configuration parser.	Diagnostic, not a metric. The independent route parser is independent, not stronger, and for the path forms added during development (Table A.1) it is no longer independent of the extractor at all.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Test-source cross-check	Node-level recall against a fourth reader of the source. The URL literals in the application's end-to-end specifications are collected; a URL its own test suite names is an address its authors expect to exist, and the extractor is asked whether it minted a node there.	Reads test source text and runs nothing, so a collected string is an address a test *names*, not one a test was observed to reach, and whether the suite passes is not established. Tested subset only. A direct `goto` is not a UI transition, so visit pairs are not counted as edges. Textual extraction under-samples click-driven navigation.
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a recovery gap and is excluded, as is a guard mutant on a screen that no edge enters. A component several routes render maps to every screen those routes declare, which errs towards scoring a mutant rather than dropping it. Detection is conditional on recovery, and the attributed and unattributed graphs do not exclude the same mutants, so they are also compared over the mutants both can score, paired by operator, file, line and subject. The sample is drawn from the graph, so it is not independent of the artefact under test. Sampling is seeded and stratified by operator.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Runtime route oracle	Node-level soundness against the framework itself. Each application is built in its development configuration, loaded in a headless browser, and asked for the route configuration the Angular router holds, with every lazily loaded subtree expanded through the application's own `loadChildren`. The addresses the extractor mints are compared with the addresses the framework mounts. Nothing re-reads the route files. Which addresses count as mounted is itself a reading, so the harness navigates the running application to a list of addresses and records what the recogniser made of each, before any `canActivate` guard runs and after the `canLoad` and `canMatch` guards that take part in matching.	Requires the application to build and boot without its server. It reads the configuration and not the rendered screen, so a route that a guard always refuses still counts as mounted, and a route whose component fails to render is not distinguished. An address claimed through a `UrlMatcher` cannot be decided from the configuration. A workspace application that composes the measured library mounts the shell's own routes as well, so part of any difference is scope and not reading. Its result files record no extractor commit or runtime version (Section 4.3.3).

4.3.1 Preconditions

Two properties of the extractor were checked before any measurement. *Determinism*: two extractions from the same unchanged source give identical nodes and edges, compared on identifier, route, event, guard and modality after sorting; without it, differences seen in mutation testing could not be told from noise. *No accumulated state*: extracting into an existing database gives the same graph as extracting after deleting it; without it, a mutant's result could be affected by the earlier baseline run.

4.3.2 Corrections made to the protocol

The final figures are those of the final version of the protocol. The development versions of uigraph and the corrections between them are listed in Appendix A.1. The figures of superseded instruments cannot be regenerated with the shipped harness, and none is cited as a result. A surprising figure is checked against its inputs before it is written down, because a figure that looked implausibly good or bad has more than once proved a property of the measurement and not of the extractor.

The mutation harness was corrected during development after its misses had been read, so the final run is not an independent test. The final run used the same seed as the run whose misses were read but a partly different sample: 46 of its 73 mutants are at the same location (operator, file and line) as one of the 70 of that run, though the destinations may differ. 6 of the 9 misses that were read are at no location of the final sample.

4.3.3 Reproducibility

The four methods that read the sources are re-run by three scripts at one commit of the extractor. The first two refuse to start while the extractor's packages carry uncommitted changes.

- `regen-round.sh` runs `run-all.mjs` with the attribution pass disabled and then enabled. It refuses an application whose checkout is not at its fixed commit, and it writes a results file with the raw output of each method. The ratio, loss, before-and-after and repair tables are generated from those two runs.
- `remeasure.sh` maps the corpus again and generates what is read off the graphs: the census, the expansion check, the witness check, the guard-only run, the call-graph counts, the ablation, the ground truth and the mutation run.
- `mutation-sweep.mjs` records the sweep of the demotion rules apart, because it runs the adapter's test suite once per rule (about forty minutes). It is refused if recorded on other rules than the commit's.

The scripts need the dependencies of the harness directory, installed with `npm ci`, and a checkout of the extractor at the evaluated commit. The extractor is run from its TypeScript source, so that the checkout being measured is not edited.

The runtime route oracle is re-run by its own documented command, because it needs each application built in its development configuration and served to a browser. Its result files record no extractor commit or runtime version, so the guarantee below does not extend to them.

Each figure from the first four methods is recorded with the extractor commit, the corpus commits, the harness version and the runtime version. Every result table is generated from those files, and prose that states a measured figure takes it from the same file as the table. Two of the scripts that produce tables stop rather than print a figure they cannot account for: the census, unless every route entry falls in exactly one bucket and the buckets sum to the route total; and the before-and-after comparison, unless every attribution-category construct is either recovered or classified among the remaining losses. That ensures the accounting closes, not that each category explains the construct's cause. Two limits remain. The evaluated commit must be one a reader can fetch, so the figures are reproducible only once it is published. And the harness version decides several figures independently of the extractor, so a result file is comparable only with one made by the same version.

uigraph was developed within this thesis. The version evaluated throughout is the final version: extractor commit `c42f220`, harness 1.9.0 and mutation harness 1.9.1. The source of the Angular adapter is identical at `920e1aa`, where the sweep of the demotion rules was recorded. The tables of Section 5.5 were regenerated at that commit. Its development versions and the corrections between them are listed in Appendix A.1.

Table 4.5 gives, for each group of generated tables, the instrument and data it comes from. The table itself, the corpus table, the counts of the rejected candidates and the figures of superseded instruments are written by hand. Two groups are historical by design and labelled so where printed: the rendered-interface check and the runtime route oracle. They were not re-measured on the final version. The runtime route oracle was run on 29 and 30 October against a graph whose extractor commit was not recorded; its node total equals the final screen count, and that agreement is its only link to the final graph. The construct prevalence counts read the sources only.

4.4 Summary

This chapter fixed the material and the measures for Chapter 5. The corpus is six applications at pinned commits, selected by explicit criteria that were not applied uniformly, and uigraph was developed on it. Every ratio has one name and one definition in Table 4.2. The five methods are each blind where the others see, and the fifth, the runtime route oracle, does not read the route files at all. The reproducibility arrangements cover the first four methods fully and the fifth only in part, and the corrections that led to the final protocol are listed in Appendix A.1.

Table 4.2: The ratios and measures of the thesis. Each has one name, used throughout. A construct is *in scope* when it is neither a query-only navigation nor excluded by the first rows of Table 4.3. Node coverage is the only measure called coverage in this thesis.

Name	Numerator	Denominator	RQ	Reported in
Node coverage	declared screens that have a node	screens the independent route parser declares	RQ1	5.1
Must conversion	in-scope constructs witnessed by a must edge	in-scope constructs	RQ1	5.1
Recovered share	in-scope constructs witnessed by an edge of any tier	in-scope constructs	RQ1, RQ3	5.3, 5.5
Accounted share	recovered constructs plus in-scope constructs recorded as dynamic or unplaced	in-scope constructs	RQ2	5.2
Must-tier precision	must edges whose witness passes (lower bound) or does not fail (upper bound)	all must edges	RQ3	5.5.3
Node-level recall	distinct URLs named in an application's end-to-end test sources that have a node	distinct URLs so named	RQ1	5.1
Structural-reference recall and precision	(site, screen) pairs the reference and a level share	pairs in the reference (recall) or pairs the level places (precision)	RQ3	5.3.4
End-to-end containment	observed screen pairs joined by an edge of the graph	observed screen pairs	RQ3	5.3.5
Scope width	mean screens a site is attributed to	the same when every site is assigned to every screen	RQ3	5.3.2
Detection rate	injected changes that produce the predicted change in the graph	injected changes the graph can score (excluded mutants reported)	RQ4	5.4
Cause trace	defect classes traced to a cause in the source, by kind (read by the final version, still not read, beyond a static reader); no ratio	defect classes	RQ5	5.5

Table 4.3: Rules that classify a construct, in the order the harness tests them. The first three rows take a construct out of the denominator, the next counts it as recovered in the tier that cites it, and the two after that as accounted for. Only a construct that reaches the rows below is lost. The indented rules are the nested decision taken when the construct's owning route component has no node.

Category	The construct ...
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
No commands	names no destination at all
Converted, must or may	is cited as witness by an edge of that tier
Accounted, dynamic	is cited by an edge into a sink: the extractor records its destination as unresolved
Accounted, unplaced	is recorded by the extractor as a site it read but could not place
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

Table 4.5: Provenance of the generated tables. *Final* means measured on the instrument every other figure describes; *historical* means measured on an earlier version of uigraph or its graph and printed as a record of it.

Tables	Status	Measured with	Regenerated from
Loss ledger, before and after, repair tables, plots	final	`c42f220`, harness 1.9.0	`results-round2/final14.tgz`; the plots also `results-rq4/`
Census, must tier, witness check, guard-only, route order, call graph, expansion	final	`c42f220`	the corpus graphs, via `remeasure.sh`
Ablation, structural reference	final	`c42f220`; generated at later commits with identical `packages/`	`results-ablation/ablation.tgz`
Mutation (RQ4)	final	`c42f220`, mutate 1.9.1	`results-rq4/`
End-to-end navigation check	final	`c42f220` graphs; suites run 3 Nov	`e2e-nav/results/`, via `score.mjs`
Demotion-rule sweep	final	`920e1aa`, same adapter source[a]	`results-must/mutation-sweep.json`
Rendered-interface check (Table 5.10)	historical	an earlier graph, before the call graph; check 1.3.0	`route-probe/results/`
Runtime route oracle (Tables 5.19, 5.20)	historical, 29–30 Oct	oracle 1.0.0, comparison 1.1.0; extractor commit not recorded	`route-probe/results/`
Construct prevalence (Table 6.1)	no extractor	pinned corpus commits	`approx-counts.mjs` over the sources

[a] `packages/core` differs between the two commits; the adapter tests the sweep runs import from it, so the sweep result is carried over on the adapter source alone.

Chapter 5

Results

This chapter reports what the harness of Section 4.3 measured on the corpus of Section 4.1, one research question at a time. Section 5.1 answers RQ1, Section 5.2 RQ2, Section 5.3 RQ3 and Section 5.4 RQ4. Section 5.5 asks what the losses were and what the cause of each says about the claim (RQ5), and Section 5.6 reports the dead links the method found as a by-product.

The numerical tables are generated from the harness result files. The summaries compiled by hand are marked as such. The main must-conversion, loss, before-and-after and graph-composition tables describe the final extractor snapshot. The baseline is that code without scope attribution, and the attributed version is the same code with the pass enabled. The development versions of uigraph and the corrections between them are listed in Appendix A.1.

Two conditions of the measurement hold for every figure below.

- The corpus source trees are extracted without their dependencies installed. The TypeScript compiler therefore cannot resolve types from the framework or third-party packages, and type-directed steps are weaker than in an installed project.
- The corpus was selected with the tool in view: `SAP/spartacus` was rejected because its configuration-driven routing is not supported (Section 4.1).

The figures therefore describe this configuration on these six applications. They are likely to be optimistic for Angular applications in general (Section 6.2).

5.1 The two levels of loss (RQ1)

RQ1 is answered at two levels. The first is whether the graph holds the screens the application declares. The second is how much of the application's navigation becomes a deterministic transition between them (the ratios are defined in Section 4.2). The first is a precondition of the second, because a transition cannot start or end at a screen the graph does not have. It is not a numerical bound on the second: one ratio counts screens and the other counts constructs, so a missing screen costs only the constructs that name it.

Table 5.1: Must conversion per application, baseline. Must conversion is the fraction of in-scope constructs whose own witness earns must; the edge columns count edges. *Nodes* counts every node of the baseline graph: the screens of Table 5.2 together with its unplaced origins and dynamic targets (Table 3.1 counts the attributed graphs, which add anchors and more of the latter). The baseline and attributed graphs are extracted from the same final snapshot on identical corpus inputs.

Application	Constructs	In scope	Nodes	Must	May	Must conversion
realworld	40	39	10	1	24	2.6%
alfresco	34	27	37	0	7	0.0%
ghostfolio	198	186	112	0	60	0.0%
k8s dashboard	72	68	74	0	3	0.0%
bitwarden	131	112	113	1	89	0.9%
mifos	1343	1343	684	15	5572	1.4%

Table 5.1 gives the second measure on the final snapshot. The constructs that earn must number 21 in the whole corpus without attribution and 25 with it. They lie in 4 applications with attribution, and the other 2 have none. Edge counts must be read per tier and not summed as distinct constructs, because one unresolved site can produce many over-approximated may edges.

Table 5.2 gives the first measure. It compares the nodes the extractor mints with the screens the independent route parser finds declared. Against that denominator node extraction is complete: on all six applications every declared screen has a node.

Section 5.5 gives the causes of the node-extraction defects the final version reads, and Appendix A.2 the handful of addresses the two readers still disagree about.

Two qualifications apply, because the figure invites a reading that neither supports.

- The denominator is the screens a *static reader* can enumerate. Against the addresses the running framework mounts, the same node set is 892 of 1146 (Section A.6). Almost all of the difference is one construct in one application that the extractor declares unread.
- The two static readers reached their agreement partly by being corrected against each other. Their agreement is therefore weaker evidence than if they had stayed independent. The runtime method is the external check for that reason, and it finds no node at an address where the framework mounts nothing. That check ran on 29 and 30 October against a graph whose extractor commit was not recorded. Its only link to the final graph is the equal node total (Table 4.5), and the graph had been corrected using earlier findings of the same check on these applications.

Completeness here is a claim about the screens the readers can *see*. The blind-spot column marks where it is incomplete but does not bound it: one route entry neither reader can resolve can hide many screens (Section A.6). Such an entry is outside the denominator, and the applications that have such entries say so in their own soundiness notes. Refusing to read more of an application would raise this column, not lower it. It is printed beside the coverage figure for that reason.

Table 5.2: Node extraction against the independent route parser, in both directions: declared screens that have a matching node, and nodes for which the independent route parser holds no declared screen. The blind-spot column counts route entries the independent route parser could not follow, which marks where either direction is incomplete. A blind spot is a route the independent route parser knows is there and cannot address: a lazy boundary whose route array it could not find, a path expression it could not resolve to a literal, a route whose path is computed by a custom matcher, or a route source spread into a table from a call it does not follow; the last kind now accounts for most of the column. The node set is the same with scope attribution off and on, since attribution mints no nodes.

Application	Declared	Nodes	Matched	Coverage	Unmatched	Blind spots
realworld	10	10	10	100.0%	0	0
alfresco	26	27	26	100.0%	1	10
ghostfolio	84	84	84	100.0%	0	2
k8s dashboard	67	67	67	100.0%	0	0
bitwarden	98	98	98	100.0%	0	1
mifos	605	606	605	100.0%	1	1
All		892			2	

5.2 Which failure is worth an analysis (RQ2)

Table 5.3 lists the defect classes and the evidence for each, separating the named evaluation methods from the other observations. Four of the five accepted classes rest on one named method each. Attribution has three observations on top of it, which gives four lines of evidence. Guards, named outlets and constant paths have one observation each, which gives two. The lazy-boundary class rests on three methods. The attribution class leans most on observations, and it is the class the analysis of Chapter 3 addresses. Its observations are not weak in themselves, but they are not independent instruments.

The five classes were found during development, and each has at least two confirmations. The quantitative breakdown revealed a sixth candidate: a node that exists but is not linked to the component that renders it. It is confirmed only by the comparison between the independent route parser and the graph, so it is reported as a candidate until a second method confirms it. It is empty on the final version.

Table 5.4 and Figure 5.1 assign every navigation construct to one outcome by the rules of Table 4.3. Of the 1818 constructs in the corpus, they show how many convert onto each tier and why the rest do not. A construct counts on the must tier only when its own witness earns that modality. The rows of Table 4.3 are loss categories, not the defect classes of Table 5.3. Guards and named outlets are measured outside this assignment. The categories that match no admitted defect class were not put through the admission rule. Guard defects are measured separately (Section 4.2).

Under the order in which the loss rules test a construct, attribution is the largest loss category in every application. Of the 1818 constructs, the two attribution categories hold 454 between them, and no other loss category holds a tenth as many. The constructs whose destination the extractor records as unresolved are counted

Table 5.3: Defect classes and the evidence for each. A class is admitted on two lines of evidence, at least one from a named evaluation method; the methods column counts only those. This summary is compiled by hand; it is not harness output.

Class	Evidence	Methods	All
Attribution: navigation outside route components is invisible	mutants excluded on the baseline graph; multi-mount edge absent; manual audit; an application with nodes and no edges on a development version (k8s dashboard; Appendix A.1)	1	4
Lazy module boundary not followed	independent route parser; test-source cross-check; mutation	3	3
Guards unreliable in both directions	red-team panels and a source survey of the guard factory and the call-form `canActivate` (Section 5.5.3)	1	2
Named router outlets not implemented	test-source cross-check; source survey	1	2
Constant-valued route paths not resolved	independent route parser; node-count collapse	1	2
Candidate: node not linked to its component	independent route parser against graph	1	1

apart; some are run-time values and some are references the extractor declines to follow. They come to about a third as many. Most of the attribution categories come from components that several screens render, not from services.

This ranking depends on the order in which the loss rules test a construct. Ownership is tested before the target, so a construct outside a route component is filed as attribution whatever its target. Of the attribution constructs, 101 have a target that the loss rules' literal test does not accept. If the target were tested first, the target category would be larger than attribution on alfresco and k8s dashboard.

Constant route paths, lazy module boundaries and nodes with no link to the component that renders them are empty in Table 5.4, because the final extractor reads their causes (Section 5.5.1 explains all three once). The ranking justifies spending an analysis on attribution: once the extractor reads what the applications declare, the navigation it still cannot place is mostly navigation with no single owning screen, under the order caveat above.

The categories also interact, in both directions. A construct in a shared component used only inside screens with no node is counted as attribution, although attribution alone cannot recover it. A construct whose own route component has no node is counted as a node defect, although attribution could reach it through a shell or subtree scope. In the final baseline the lazy-boundary, constant-path and component-unlinked categories are empty. A residual node-missing category still holds a few constructs (Table 5.4), and attribution converts none of them (Table 5.8); that it could is a possibility the rules allow and this instrument does not show. The 454 attribution constructs are therefore neither a floor nor a ceiling on what scope attribution recovers (Section 5.3).

Table 5.5 reports the witness check on the final attributed tier and the test-source cross-check. Section 5.5 treats the must tier separately.

Table 5.4: Outcome of every navigation construct, baseline. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos. All rows are measured on the final snapshot with attribution disabled.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	1	0	0	0	1	19	21
Converted, may	16	3	55	3	53	956	1086
Attribution, shared component	21	10	63	51	37	224	406
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	0	0	0
Constant path	0	0	0	0	0	0	0
Component unlinked	0	0	0	0	0	0	0
Constant target	0	0	0	0	0	5	5
Node missing, other	0	2	3	1	0	34	40
Other	0	0	6	0	2	3	11
Dynamic destination	0	4	50	8	4	92	158
Source the analysis cannot place	0	0	0	0	0	0	0
Query only (not a transition)	1	0	11	4	19	0	35
Excluded: names no destination	0	0	1	0	0	0	1
Excluded: navigates a named outlet	0	7	0	0	0	0	7
All constructs	40	34	198	72	131	1343	1818

5.3 Scope attribution (RQ3)

The analysis under evaluation is the one Chapter 3 states. Every navigation site is anchored at a scope, the set of screens from which it can fire, and scopes form the powerset lattice defined in Section 3.2.2. The rules that compute them are in Section 3.2, and the anchored representation in Section 3.2.6.

The effect of the pass is measured by running one commit of the extractor twice over the same inputs, with the pass disabled and enabled, so that nothing but the pass differs. Three quantities decide the answer: how much navigation the pass recovers, whether it costs the deterministic tier anything, and what it still cannot place. The third shows where the analysis as implemented stops.

Table 5.6 and Figure 5.2 compare the extractor with and without scope attribution on identical inputs. The comparison is made construct by construct. A construct is *recovered* if it was lost to attribution in the baseline and is witnessed by an edge after attribution.

Attribution recovers 292 of the 454 constructs the loss rules assign to it, 64.3 % of them, which this thesis calls the attribution recovery. Most of them are on mifos: without mifos the share is 37.3 %, and 4 of the six applications recover under half. It takes the graphs from 5919 edges to 12172.

Measured by RQ1's must conversion, the gain is far smaller. Of the recovered constructs, 2 earn must and 290 earn may, and 2 of the six applications gain any

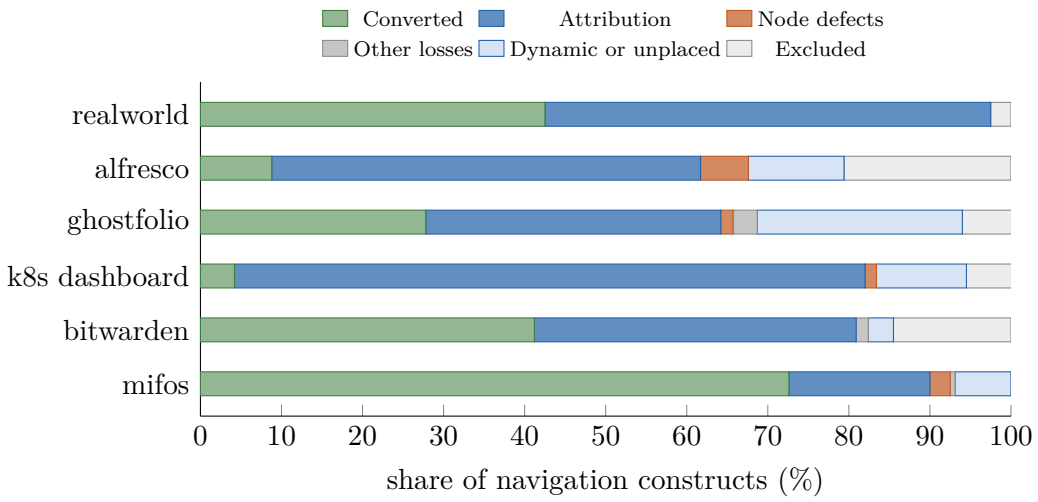


Figure 5.1: What happens to the navigation constructs of each application without the attribution pass. Under the order of the loss rules, attribution losses are the largest category in every application; of the node classes, the lazy-boundary, constant-path and component-unlinked ones are empty on this instrument, and the remaining one holds few constructs, most of them in mifos. The exact counts per class are in Table 5.4. *Converted* counts constructs witnessed at must or may, as that table does; the must conversion of Figure 5.2 counts must only.

Table 5.5: Results of the witness check, on the final attributed tier, and of the test-source cross-check. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

Method	Result	*n*
Witness check, final attributed must tier	0 detected failures; 62 edges have a destination the checker cannot read. Unread edges are not passes, and target agreement does not establish Claim 3.1.	150
Test-source cross-check, alfresco	10 of 14 URLs the specifications name have a node; one negative test excluded.	14

must conversion at all. The must constructs rise from 21 to 25. Two of the rise are those recoveries, both on realworld. The other two are k8s dashboard constructs that the baseline already converted at may and attribution raises to must; the recovery count does not include them.

The two are different measures of recovery, and both are reported. A gain in represented navigation need not be a gain in deterministic transitions when targets or execution conditions remain uncertain. Counted in edges, the must tier grows from 17 to 150. The edges added are the links of two application shells, drawn once for each screen the shell renders (Section 5.5). That is why the ratio is computed over constructs.

The attributed tier has 0 detected witness-check failures and 62 edges whose cited destination the check cannot read, and unread edges are not agreements. Following each construct by its identifier across the two runs finds 0 that leave the converted

Table 5.6: Before and after scope attribution on the final evaluated snapshot. Must conversion in per cent and its change in percentage points; must-tier edges whose destination the witness check read and passed; and constructs assigned to attribution in the baseline that recover onto must or may. A construct's tier is the modality its own witness earns. The remaining must edges are unread destinations, all 62 of them on k8sdash, which are neither passes nor failures; no edge failed. Constructs converted from other baseline categories are in Table 5.8.

	Must conversion			Must, read and passed		Attribution losses, recovered		
Application	Before	After	pp	Before	After	Before	must	may
realworld	2.6%	7.7%	+5.1	1/1	9/9	22	2	20
alfresco	0.0%	0.0%	+0.0	–	–	18	0	4
ghostfolio	0.0%	0.0%	+0.0	–	–	72	0	32
k8s dashboard	0.0%	2.9%	+2.9	–	63/125	56	0	8
bitwarden	0.9%	0.9%	+0.0	1/1	1/1	52	0	16
mifos	1.4%	1.4%	+0.0	15/15	15/15	234	0	210
All						454	2	290

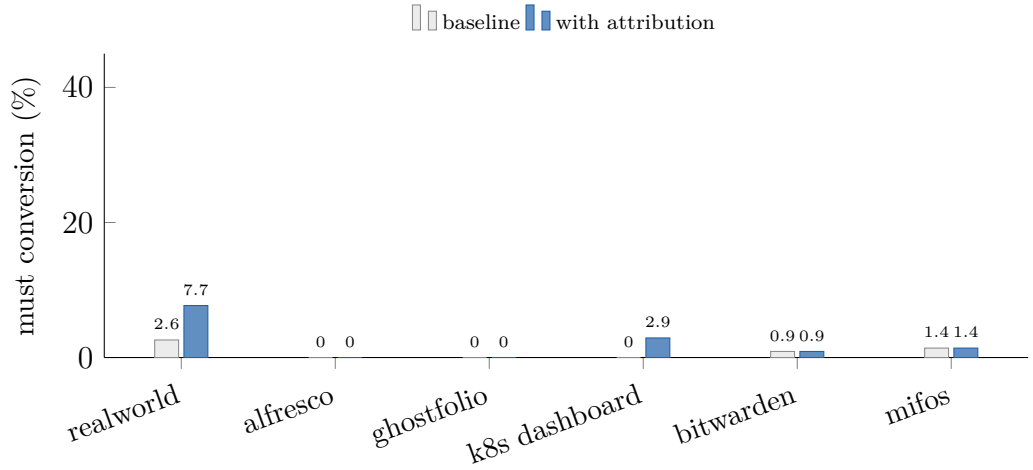


Figure 5.2: Must conversion before and after scope attribution, on identical inputs.

categories, so no must conversion of the baseline was lost. This measures recovery of represented constructs and says little about must-tier precision. The evidence for that tier, and the edges outside it, are treated in Section 5.5.3.

The anchored representation was checked on the same graphs. Claim 3.2 requires the expansion of the anchored edge set to equal the materialised edge set. The anchored edges are therefore materialised independently, by fanning each scope-anchored edge over the screens its anchor stands for with its own deduplication, because the library's own materialisation function is a loop over the query and could only disagree about which nodes it iterates. The two agree on every application, and Table 5.7 is not produced otherwise. The table is also the only measurement of what the representation saves: on mifos the materialised edge set is 3.1 times the size of what is stored.

The check exercises the member walk and the deduplication key. The definition

Table 5.7: The anchored edge set against its expansion. *Anchored* is what the graph stores, *expanded* what the per-screen query yields, and *independent* a second materialisation written for this check. Claim 3.2 requires the last two to be equal; the check refuses to emit this table if they differ on any application.

Application	Screens	Anchored	Expanded	Independent
realworld	10	65	97	97
alfresco	27	55	100	100
ghostfolio	84	403	1166	1166
k8sdash	67	309	694	694
bitwarden	98	529	714	714
mifos	606	10811	33418	33418

Table 5.8: Where recovered constructs came from on the final snapshot. Rows are their baseline loss categories; columns give the modality each construct's own witness earns after attribution. Constructs are followed by identity, not matched by source location.

Baseline class of the construct	must	may	total
Attribution, shared component	2	271	273
Attribution, service	0	19	19
Other, no class rule applied	0	2	2
Out of the attribution classes	2	290	292
Out of the node classes	0	0	0
Out of the remaining classes	0	2	2
All conversions gained	2	292	294

of a scope's membership is shared by both paths and is not tested by it, so the claim is supported on 6 applications and on fixtures, not proved. On 5 of them the library's own function returns a larger set than either, which is the iteration difference and not a disagreement about the edge set.

Table 5.8 gives the recovery by the loss category each construct held in the final baseline and by the modality its own witness earns. Of the recoveries from the attribution categories, 2 earn must and 290 earn may. The loss categories are not independent causes: a shared component can be assigned to attribution even when a missing target node prevents its recovery.

Table 5.9 classifies each construct that stays lost.

The two tables close the attribution categories between them: 292 recovered plus 162 remaining is the 454 the rules assigned, which the script that produces them asserts before it writes either table. All but 2 of the remaining constructs carry a reason; those 2 have a scope and a destination the graph holds, and the diagnosis names no reason they stay lost. The reasons are limitations of the implemented analysis and limits of the source population.

- Some components have no recovered route or template path to a screen. This is

Table 5.9: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	8	11	5	18	1	43
In a service or guard, not placed	0	6	4	4	8	7	29
Scope found, target screen has no node	0	0	23	2	6	3	34
Scope found, target fully dynamic	0	0	2	37	2	13	54
Scope found, not explained	0	0	0	0	2	0	2

Table 5.10: Recorded scope predictions of an earlier graph against the rendered interface, on the 5 applications the harness could serve. These predictions were not recomputed for the final call graph. *Judged* counts screens that rendered themselves rather than redirecting to a sign-in page; the rest cannot be judged from what rendered, because a redirected screen shows another screen's links. *Not predicted* is the only column that counts against the analysis; the other direction is reported in the text, because it can be judged only where the graph names a destination exactly.

Application	Screens	Judged	Pairs rendered	Predicted by the scope	Not predicted	Raw anchors
realworld	10	7	25	25	0	59
alfresco	26	3	0	0	0	0
k8s dashboard	67	1	0	0	0	0
bitwarden	98	18	8	4	4	8
mifos	605	1	0	0	0	0
All	806	30	33	29	4	67

an empty computed scope, not proof that the application never renders them.

- Some sites lie beyond the member-level call graph's supported receiver and provider forms.
- Other sites have a scope but a dynamic destination, or a target for which the graph has no node.

5.3.1 Checking the scopes against the running interface

Everything above measures what attribution recovers. Whether the *set of screens* it computes for a site is the set that site fires from is a different question, and the contribution depends on it. The rendered-interface check of Section 4.3 answers one direction: it navigates to each screen the graph holds, reads the in-app links the page carries, and compares them with the edges the graph's per-screen query yields there. Table 5.10 retains the recorded observations of an earlier graph. The browser experiment was not repeated and its predictions were not recomputed for the final call graph, so this section documents what that earlier graph missed.

A scope claim can be too small or too large. Too small means the application renders a link the graph does not attribute to that screen, which the rendered interface

decides. Too large means the graph attributes a site to a screen the application never fires it from, which the interface decides only where the graph claims a destination exactly. The two are reported separately.

Scopes too small. Of the 33 (screen, target pattern) pairs that the running applications render on the judged screens, 29 are predicted by the computed scope (87.9 %). Behind these predictions stand 67 raw anchors. On realworld, which renders most of its screens without a backend, every rendered pair on every judged screen is predicted. The pairs that are not predicted are all bitwarden's.

The unit of the check is the screen and the destination, not the site: the graph's edges leaving a screen are pooled and set against the links the screen renders. A confirmed pair therefore shows that the screen's outgoing destinations cover what it renders, not that each site behind them was attributed to that screen. An unpredicted pair is not affected by the pooling, so the 4 are direct evidence of scopes that are too small.

The evidence is limited in size. It covers 30 screens of 806, because 776 redirect to a sign-in page when there is no backend behind them, and only 5 of the six applications are measured. ghostfolio is left out: it builds a separate bundle per locale, each with a base path of its own, and the harness serves a single directory at the server root. The check settles one direction of a question, on part of the corpus.

The 4 unpredicted links share one cause, and all four are in bitwarden. Its `/login` screen renders links to `/hint` and `/signup`, and two further screens render a link to `/signup`. The graph holds those edges, with the right witness (`libs/auth/src/angular/login/login.component.html`), but anchors them at an origin node rather than at a screen. The component that renders them lives in a workspace library outside the application directory, and attribution computed the empty scope for it. The failure is a scope the analysis declined to compute, which the running application shows to be non-empty. These sites are the *unplaced* class of Table 5.17.

Scopes too large. This direction does not support a single figure. 80 predictions run this way: the graph holds the edge and the page showed no link. Two of the three causes mixed in that count are not defects. The harness signs in to nothing, so realworld's header, wrapped in `@if (authState === 'authenticated')`, hides *New Article*, *Settings* and the profile link (interface state). A tag page navigated to with a placeholder segment lists no articles (data). The third cause is target over-approximation, a property of the destination and not of the scope: realworld's `[routerLink]="['/profile', article.author.username]"` resolves to `/profile/:username` and to `/profile/:username/favorites`, and the deeper route is recorded as a may edge at half confidence, which is the graph stating that it does not claim the address.

Only predictions that name a destination exactly can be refuted by a page not linking there. There are 7 of them across the judged screens, against 102 over-approximated, and 0 of the 7 were not rendered. A denominator of 7 supports no rate. The over-attribution direction was therefore not refuted on this corpus, because the graph almost never claims an exact destination at a screen this check can observe.

Table 5.11: The computed attribution against assigning every site to every screen. A site is a source construct a witness cites; an edge anchored at a scope is counted over the screens that scope stands for, since counting it at its anchor would understate the analysis's breadth. *Mean scope* is the attributed pairs per site.

Application	Screens	Sites	Attributed pairs	Mean scope	All-screens pairs
realworld	10	39	188	4.82	390
alfresco	26	19	117	6.16	494
ghostfolio	84	189	4104	21.71	15876
k8sdash	67	62	2450	39.52	4154
bitwarden	98	167	1785	10.69	16366
mifos	605	1312	43059	32.82	793760

The measure is reported as several numbers, not a precision figure, so that the two directions stay separate.

5.3.2 What the attribution discriminates

The rendered-interface check cannot say whether the scopes are too large, so this subsection asks how much narrower the computed attribution is than the widest one there is. Assigning every navigation site to every screen recovers every construct, needs no analysis, and is useless: the per-screen query would return the whole site list on every screen. The ratio between the two measures what the analysis contributes (Table 5.11).

The analysis is between 1.7 and 18.4 times narrower than that baseline, and the low end of the range is the figure that qualifies RQ3. On k8sdash the average site is attributed to 39.5 of 67 screens, which is 59 % of the application. The per-screen query there excludes about two screens in five and admits the rest. The widest margin is mifos, where 43059 attributions are computed where the baseline would assert 793760. Across the corpus the mean scope runs from 4.8 to 39.5 screens.

Two things follow, and the second is the less favourable to the analysis.

1. The trivial baseline is not a competitor for the must tier. An unguarded site in a shared component attributed to every screen would yield a must edge from screens that never render it, so Claim 3.1 would fail by construction. That none of the readable must edges attribution added fails the witness check is meaningful only where the attribution is narrow. Of the 133 must edges it added, 62 are unread, all on k8sdash, where the scopes are widest.

2. On part of this corpus the attribution is not very narrow. A scope that names three fifths of an application is a weak statement about where a site fires, and the attribution recovery RQ3 reports is bought with scopes of that width. Nothing in the evaluation shows those wide scopes to be wrong, because the rendered-interface check could not test the direction in which they would

Table 5.12: The corpus extracted at each attribution level. *Sites* counts the source locations a witness cites that the level places on at least one screen. One construct can have more than one, and a location can lie outside the application directory, so site counts are not comparable with the construct counts of Table 5.4. *pairs* the (site, screen) attributions it asserts. The levels are nested, and the table is produced only if each one's pairs are a subset of the next one's. *Screens* counts the screens at an address a link can name, so the catch-all screens of alfresco and mifos are left out of it and of the pairs, and the column falls short of the census's total by those screens.

		sites / (site, screen) pairs				
Application	Screens	owned	nesting	rooted	outlet	full
realworld	10	17 / 26	25 / 48	38 / 178	38 / 187	39 / 188
alfresco	26	11 / 33	14 / 37	14 / 37	14 / 37	19 / 117
ghostfolio	84	111 / 121	129 / 155	170 / 3599	182 / 3712	189 / 4104
k8sdash	67	11 / 12	59 / 275	59 / 275	59 / 2250	62 / 2450
bitwarden	98	124 / 134	149 / 389	154 / 879	156 / 1448	167 / 1785
mifos	605	1080 / 1431	1243 / 1759	1303 / 38059	1303 / 41532	1312 / 43059
All	890	1354 / 1757	1619 / 2663	1738 / 43027	1752 / 49166	1788 / 51703

be wrong. Nothing shows them to be right either. The recovered share of Table 5.17 should be read together with the widths in this table.

5.3.3 What each mechanism contributes

The width says how far the attribution is from doing nothing, not whether the mechanisms that produce it earn their place. Table 5.12 extracts the corpus at 5 nested levels, each withholding every mechanism above it. The withholding covers the stand-ins of Section 3.3 as well as the implemented relations, because otherwise the lowest level would credit direct ownership with sites that only the stand-ins place.

The levels are not evenly spaced.

- Direct ownership places 1354 sites, 75.7 % of what the full analysis places. Containment adds 265 sites, the largest gain in sites of any step, and brings the total to 1619.
- The shell rule is the largest step in (site, screen) pairs and in reference recall, and this thesis does not add it. *Rooted* places 1738 sites, 97.2 % of the full figure, and carries the pair count from 2663 to 43027.
- Everything above that level raises the site count from 1738 to 1788, that is 50 sites or 2.8 % of the corpus total. Outlet descent (which an ancestor-route closure in earlier work also covers) adds 14 of them. The full level adds 36, 2.0 % of the corpus total. That is the member-level call graph that places a service's navigation, its fallback join and the stand-ins for guards and effects together, all of which only that level carries.

The call graph's own share is smaller. Against the coarser class-level rule it replaced, which remains as the fallback for a member no caller reaches, it places 7

more sites, 0.4 % of the corpus total, and 331 more pairs. The two differ in pairs on 5 of the six applications: 10 pairs only the class join asserts and 341 only the call graph does.

The shell rule is not a mechanism this thesis introduced, so the comparison with earlier work uses two of the levels. Oshima and Kishi assemble their page information from the root component as well as from the components beneath it (Section 2.4), so *rooted*, not *nesting*, stands for what they reach. Rima et al.'s specification also closes a route's components over those of its ancestor routes, which is what *outlet* adds. Both levels reimplement those mechanisms inside this extractor and are not runs of either tool. The comparison measures what the added mechanisms place on the same inputs, and does not show that the tools themselves would place fewer. Against the nearest of them, *outlet*, this thesis adds the *full* step alone: 36 sites, 2.0 % of the corpus total, every one outside a component, in a service, guard or effect. Rima et al.'s following of a handler into same-class helpers and into a `subscribe` callback is already inside *outlet* here, because the extractor places every navigation call in a component's class at that component's screens whichever member holds it.

Outlet descent contributes mostly width: it places 1752 sites against rooted's 1738, the same count on 4 of the six applications, while raising the pair count from 43027 to 49166. A table of counts cannot say whether that width is correct, which the next subsection addresses.

5.3.4 Agreement with a structural reference

For realworld the screens each navigation site fires from were read off the sources by hand (Section 4.3): 39 sites and 188 (site, screen) pairs. Scoring every level against that reading gives a comparison no other method here can make, because it has both directions rather than one. Table 5.13 reports it.

Three limits qualify the reading. It applies the analysis's own relations by hand, so it tests their implementation and not whether they describe what the running application renders (Section 4.3). It is not independent of the graph: its first version was completed with nine header sites the scorer reported from the graph. And it is dominated by the shell: 13 of its 39 sites are placed on every screen and supply 130 of the 188 pairs, which every level from *rooted* upward places trivially. Over the other 58 pairs, recall is 82.8 % for *rooted*, 98.3 % for *outlet* and 100.0 % for *full*.

The table is produced only if each level's pairs are a subset of the next, and the full level asserts no pair outside the reference. No lower level can then assert one either. The levels differ only in recall, and the absence of excess pairs at the lower levels follows from the nesting; it is not a separate result. Direct ownership recovers 13.8 % and containment 25.5 %, both far short. The shell rule, the level that stands for the prior work's reach, recovers 94.7 % at 100.0 % precision and leaves 10 pairs unfound. Precision is measured against a reference whose header sites were added from the graph's output, so it is weakest as evidence exactly where the shell dominates.

Everything above *rooted* reduces 10 missing pairs to 0 and asserts no pair outside the reference. The last row, the class-level join that the call graph refines, asserts 9

Table 5.13: Each attribution level against the hand-read structural reference for realworld. *Missing* is a pair the reference has and the level does not; *excess* is a pair the level has and the reference does not. Precision and recall are agreement with that reference, not with what the running application renders.

Level	Correct	Missing	Excess	Precision	Recall
owned	26	162	0	100.0 %	13.8 %
nesting	48	140	0	100.0 %	25.5 %
rooted	178	10	0	100.0 %	94.7 %
outlet	187	1	0	100.0 %	99.5 %
full	188	0	0	100.0 %	100.0 %
class	188	0	9	95.4 %	100.0 %

pairs outside the reference, all at one site, the `navigate` inside `UserService.logout`, which takes precision to 95.4 %. The class join attributes a navigating service to every screen whose components inject it. Ten components inject this one, the header among them, and the shell renders the header on every screen, so the site was attributed to every screen although its only caller is a button on the settings screen. The call graph follows the call from that button and places the site on the settings screen alone.

The clean row has one limit. The structural reference guided two repairs of the analysis: the call graph, written after the reference exposed the class-level defect, and the rule that places an empty-path child on the screen above it (Section 6.3). The 100.0 % precision and the recall of *full* are scored on the same reference, so they are consistent with the two repairs and test neither independently.

For RQ3 two things follow. First, most of the recovery attributed to this analysis is produced by mechanisms earlier work already had: 94.7 % of the 188 pairs are reached by the shell rule and 99.5 % once the enclosing routes are added. The *full* step adds 1 pair on this application and takes recall to 100.0 %. Second, this is one application of six, the smallest, and the corpus-wide widths of Table 5.11 are not licensed by it. What the ground truth establishes is narrower than soundness: the additional mechanisms raise agreement with the structural reference without a pair outside it, after the one scope it found too large and the one screen it found missed had both been repaired with it in view. It cannot establish that the same holds on an application of six hundred screens.

5.3.5 Agreement with the applications' own suites

The end-to-end check of Section 4.2 puts part of the question to the running applications, with a reference that does not pass through the analysis's relations. The second and third validity checks hold on both applications, and the first on mifos, whose uninstrumented run fails the same tests; for realworld only instrumented runs were recorded, none of them with a failure. A hit is any edge between the two screens, not the edge of the site the test used, so the check does not test the scope of an individual site.

realworld. The suite completes 141 navigations, which reduce to 19 distinct pairs, and the graph contains all 19 of them: 16 through an edge from the source screen or from a scope that names it, and 3 only through the shell scope, which names every screen. This result does not bear on what attribution adds, because the *rooted* level, which stands for earlier work's mechanism, already contains 19 of the pairs, that is all of them. Without attribution the graph contains 13. The suite exercises 19 of the 79 pairs the expanded graph contains, out of 100 possible ordered pairs of screens including self-pairs, so the agreement is with the transitions the suite takes and not with the graph.

mifos. The suite completes 62 navigations, 22 distinct pairs, and the graph contains 12 of them. This count applies the pre-registered rule: a redirected navigation is compared on the address it asked for. 5 pairs were redirected, and 2 of those are hits. The scorer's departure from this rule during development is listed in Table A.1. Without attribution the graph contains 10. None of the matched pairs is a must edge, as predicted. 2 hits, those of the redirect bucket, rest only on a site whose target prefix was over-approximated to many routes, so they show that the target is reachable in the graph and not that the site was resolved. No other pre-registered prediction is reported here.

The mifos misses. Of the 10 misses, 3 fall in the pre-registered redirect class and are not classified further. The other classes were defined after the misses had been read, and are tested in the order redirect, base, dynamic, first match winning. Every one of the 10 misses meets the dynamic test whatever the order. The dynamic count is therefore only what is left after the others, and the test, which fires on any unresolved site in the source component or above it, does not show that the missed navigation is that site. The pre-registered five-class scheme and its prediction that misses would be mostly a scope too narrow or a site not read are not scored.

- 4 misses are classed *dynamic*. They are places the extractor does not read, not destinations only the run time knows. Three sites, in the clients, groups and savings views, open an action screen with a template literal inside a commands array, for example `navigate([`actions/${name}`], {relativeTo})`. The reader treats that element as a variable and records the target as unresolved (Section 6.3), although the text is static up to the action name. In the loan view, `relativeTo` is passed through an options variable the reader does not evaluate, so the relative prefix is matched from the root and names no route.

- 3 misses are classed *base*, and are losses of a declared approximation, not repaired. The client view's template holds three relative links, to the personal data tab and to the forms for a new loan and a new savings account. From the client screen the graph resolves them correctly. The template is rendered with each child tab, though, and attributed to the subtree scope of that screen its links are resolved against the child's address, where they name nothing mounted. The edges lead to the not-found screen, and the graph lacks the edge the suite takes from the general tab. Angular resolves such a link against the

Table 5.14: Earlier work and this thesis on the corpus. The first row applies the earlier method's stated restrictions. The other rows run uigraph at three levels: with the attribution mechanism of Oshima and Kishi (*rooted*), with the enclosing routes Rima et al. add (*outlet*), and with this thesis's full analysis. They are not results of the earlier tools, which were not run. A dash marks a figure not measured. All cells are generated figures.

	rooted	outlet	full
Applications the stated input restrictions admit (Oshima and Kishi)	0 of 6	–	6 of 6
Sites placed beyond the rooted level, corpus	–	14	50
Structural-reference recall, realworld	94.7 %	99.5 %	100.0 %
End-to-end pairs in the graph, realworld	19 of 19	–	19 of 19

route of the component that holds it. This is the base resolution Section 5.6 names as this analysis's gap.

The base misses are losses of a declared approximation and the dynamic misses are forms the extractor does not read; none is a target only the run time knows, and none was repaired. No other application with a runnable suite remains to test a repair, and a repair fitted to this check would be untested by it.

5.3.6 Against earlier work

Table 5.14 gathers, in one place, what the corpus shows about the earlier static extractions of Angular navigation (Section 2.4) and about this thesis. Their tools were not run. The input restrictions are those the authors state, applied to the route tables the independent route parser reads. The other rows reimplement their mechanisms inside uigraph, on uigraph's own node extraction, so they measure the mechanisms and not the tools. Every row is measured on the corpus uigraph was developed on (Section 6.2).

Two things follow. The restrictions Oshima and Kishi state exclude every application in the corpus: all 6 declare a route parameter, and all 6 a nested route tree. Against their method, the difference is first what it accepts. Against the mechanisms of both works, applied inside uigraph, the full analysis adds little: 50 sites over the rooted level, 2.8 % of the corpus, and on realworld the rooted level already holds every observed end-to-end pair. What the corpus does not show is how the earlier tools themselves read these applications, since neither was run; Rima et al. state no restriction of this kind in the sources read (Section 2.4).

5.3.7 What the RQ3 evidence supports

Scope attribution raises measured recovery. Its attributed must tier has 0 detected witness-check failures among 150 edges, with 62 unread destinations. This is limited evidence for RQ3's precision requirement, because unread edges do not pass the comparison and target agreement cannot establish execution on every input (Section 5.5). Three qualifications belong with that answer.

1. The great majority of the gain is on the may tier: attribution mostly adds navigation the graph says is possible, not navigation it says is certain.

2. The scopes are compared in both directions on one application only, against a reference that applies the analysis's own relations. The recorded rendered-interface check exposed 4 missing pairs on its earlier graph, all on bitwarden, and cannot establish whether a scope is too large. The ground truth tests both directions against the structural reference, but for realworld alone, and both repairs were made after it had shown the defects. The end-to-end check tests the missing direction on a second application and only on the transitions a suite exercises: it finds none missing on realworld and 10 of 22 missing on mifos, 3 of them through a relative link resolved against the wrong route.

3. The computed scopes are narrower than the trivial attribution by a factor between 1.7 and 18.4 (Section 5.3.2), and at the low end a scope names most of the application. A site attributed to too many screens can produce edges whose readable witnesses agree, because the construct is at that location and does name that target, so no witness-based method can find it.

That the scopes are not too large is not established, and Section 6.2 states it as the most serious limitation of the work.

The same mutants were also run against the baseline graph, to ask whether attribution changes how sensitive the graph is to a source change. Many mutants cannot be scored against the baseline at all, because the construct they change produces no baseline edge: 22 are excluded there against 6 on the attributed graph. The clearest case is the link-deletion operator on realworld, where every candidate sits in the application shell. Attribution widens the part of the source the graph responds to at all.

The raw detection rates, 67 of 67 (100.0 %) with attribution and 50 of 51 (98.0 %) without, are taken over different mutant sets, so their difference mixes a detection effect with that eligibility effect. The paired comparison isolates the detection effect. Of the 51 mutants both graphs can score, identified by operator, file, line and subject, the attributed graph detects 51 (100.0 %) and the baseline 50 (98.0 %). They disagree on 1, a guard removal on realworld's empty-path `editor` child that the baseline run scores and misses, and Section 5.4 shows that this comes from how the harness matches the guarded screen, not from the graph. Outside the paired set, 16 mutants are scorable only against the attributed graph and 0 only against the baseline. On this corpus, then, attribution leaves the detection rate per scored change essentially unchanged, and its effect on change sensitivity is the widening of what can be scored at all.

5.4 Change sensitivity (RQ4)

Mutation testing requires the application to be extracted again once per mutant, so this part of the evaluation was run on three of the six applications. They are

Table 5.15: Mutation sample. Candidates found, mutants sampled under the per-operator quota, and mutants excluded as confounded with a recovery gap.

Application	Candidates	Sampled	Excluded
realworld	26	25	0
alfresco	25	16	1
bitwarden	221	32	5

realworld and alfresco, which carry the earlier mutation figures, and bitwarden, whose routes carry many guards, guard factories among them. Every figure in this section comes from the attributed graph with harness mutate 1.9.1, a fixed seed and a quota of eight mutants per operator. The corrections made to the mutation harness are listed in Table A.1.

Table 5.15 shows how the sample was drawn. The harness found 272 candidate mutations, sampled 73 under the quota and scored 67. 6 were excluded, all of them guard removals: for 5 no edge of the unmutated graph enters any screen the guarded route yields, and 1 guards a route that declares neither a component nor children. An excluded mutant carries no information about sensitivity. The rule removes exactly the cases in which a removal could not be observed, so 11 of the 17 sampled guard mutants are scored and the guard rate below is conditional on the graph entering the guarded screen. The exclusions are few on the attributed graph but not on the graph without attribution: of the 73 mutants sampled against it, 22 are excluded and 51 remain, the same recovery gap seen from the mutation side.

Table 5.16 gives the detection rate per operator, and Figure 5.3 the outcomes summed over the applications. The rename operator is set aside first, because its detection is close to guaranteed by construction: a rename mutant is excluded unless the unmutated graph already holds a node for the segment it renames, and a screen is identified by its full path (Section 3.1). It tests node extraction and supplies 22 of the 67 scored mutants. The three other operators are detected in 45 of their 45 scored mutants: 14 of 14 deleted `routerLink` bindings, 20 of 20 retargeted `navigate` calls and 11 of 11 removed guards. A deleted `routerLink` is sampled only on a line an edge of the unmutated graph cites, so it is missed only if the extractor misreads its own input. The other two operators are constrained in the same way: a retarget is generated only for a `navigate` call whose first element is a plain absolute string at a line the graph already cites, and writes one declared route as a literal; a guard removal is scored only where an edge already carries that guard and enters the guarded screen. RQ4 therefore tests whether the graph re-reads constructs it already represents, not its sensitivity to harder edits. The samples are small, in single digits per operator and application, so the pattern says more than the rate.

Deleting a `routerLink` moved the witness set rather than the edge set in 5 of its mutants, because a second identical link in the same template still produces the edge, which is why detection is judged on witnesses as well as edges. The operator found no candidates in alfresco, which binds `[routerLink]`. Against the graph without attribution the same sample yields only 7 scored link mutants, of which 7 are detected, because the candidates in the application shells produce no baseline

Table 5.16: Detection rate per mutation operator on the attributed graph, as detected over scored mutants. A dash means the operator found no candidate in that application.

Operator	realworld	alfresco	bitwarden
Rename a route path	6/6	8/8	8/8
Delete a `routerLink`	6/6	–	8/8
Retarget a `navigate`	8/8	4/4	8/8
Remove a guard	5/5	3/3	3/3
All operators	25/25	15/15	27/27
Detection rate	100.0%	100.0%	100.0%

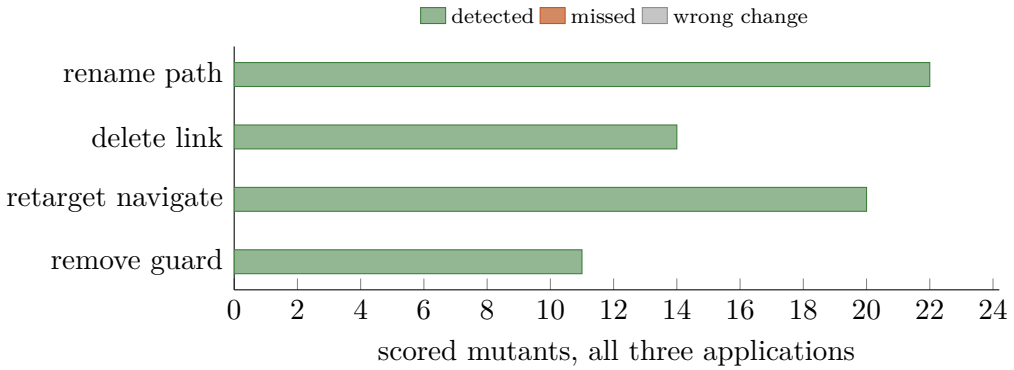


Figure 5.3: Mutation outcomes by operator, summed over realworld, alfresco and bitwarden, on the attributed graph. Excluded mutants are not shown; 6 of the 73 sampled were excluded here against 22 of 73 on the graph without attribution.

edge.

A retargeting mutant replaces the whole commands array of a `navigate` call with a declared route that names no parameter, and is detected when the edited line witnesses an edge to the new destination and none to a destination it reached before. The destination is drawn from the declared routes alone, so both configurations patch each site alike (Appendix A.1 lists the operator's earlier artefacts).

Every guard removal the run can score is detected: 11 of 11, on all three applications, each judged by whether some edge loses a guard the removed list names. A guard is recorded on the edges that enter the guarded screen, so a removal can be observed only where the graph holds that screen and enters it, and all 6 exclusions are that condition failing. The harness matches a class by the file the routing module imports it from, because bitwarden declares two classes named `AccountComponent`. Among the scored guards the empty path makes no difference: 3 of the 3 on a route whose own path is the empty string are detected.

One limit of the same matching remains, and it shows in the run without attribution. realworld's `editor` route has two children, at the empty path and at `:slug`, that load the same component file, so the harness cannot tell their screens apart. Without attribution no edge enters `/editor`, but one enters `/editor/:slug`, so the guard on the empty-path child is scored and missed rather than excluded.

That is the one paired disagreement, and it is a limit of the harness, not of the graph.

The mutation run does not confirm the defect class *guards unreliable in both directions* of Section 5.2 on this instrument, and it does not contradict it either. Every guard it could observe behaved as the model requires, and the 6 it could not observe carry no information about guard representation. The evidence for that class is the pair of defects found by red-team panels and a source survey (Section 5.5.3): a guard carried by a route a wrapper function returns, and a `canActivate` element written as a call rather than as a name. The final extractor reads both.

Of the 67 scored mutants, 67 were detected, and 50 of 51 without attribution. On the 51 mutants both graphs can score, the figures are 51 and 50. In this sample the graph responded to every scored change in the constructs it represents. A construct it does not represent cannot respond, and such mutants are excluded rather than scored, so the rate is conditional on RQ1's recovered share. The mutation harness was corrected during development after the misses of an earlier run were read. The final sample shares 46 of its 73 mutants (same operator, file and line, possibly other destinations) with that run, so the rate is not an independent test.

5.5 What the losses were (RQ5)

The defect classes of Section 5.2 are a diagnosis. RQ5 asks what lies behind each class and what that says about the claim, and it is answered by cause analysis on the final version. For each class the cause was traced in the source of the application it appears in. The same question was asked of each cause: is the navigation unreadable, or is it merely unread? If the final version reads the cause, the information was in the source and the loss was the tool's. The causes it does not read, and the causes beyond a static reader, are listed apart. The tables report the final snapshot used for the main RQ1–RQ3 comparison. The development versions and the corrections between them are listed in Appendix A.1.

5.5.1 What the losses were

This section keeps two words apart: a *class* is a row of Table 5.3, and a *cause* is what the source review found behind a class.

Read by the final version. The information was in the source, so the loss was the tool's.

- *Lazy module boundary* and *constant-valued route paths*: causes in the extractor, read by the final version, and empty in Table 5.4.
- *Guards*: the two unread forms (a guard on a route a wrapper returns, a `canActivate` element written as a call) are read, and the guard rule is the router's own semantics (Section 5.5.3).

- The candidate class *node not linked to its component*: empty on the final version.

Still not read. The final version does not read these causes.

- *Attribution*: 64.3 % of its constructs are recovered. The remainder is in Table 5.9, including sites that no relation the prototype implements reaches. It mixes gaps of the tool with components that no screen may render, and no measurement separates them.
- *Named router outlets*: a construct the model represents and the extractor does not implement. Its navigations are excluded from the denominator (7 sites) and its screens left out of the runtime route oracle's mounted set (64 route entries).
- ghostfolio's spread of computed routes is a construct the extractor could read and does not. 242 of the 251 mounted addresses without a node sit behind it.
- Three readable forms that the end-to-end check exposed: a template literal inside a commands array, `relativeTo` passed through a variable, and the base resolution of a template shared with child tabs (Section 5.3.5).

Beyond a static reader of a route table. No reader of a route table can resolve these.

- ghostfolio's custom `UrlMatcher` computes a route's path in a function. No reader of a route table can resolve it, and both tools report it as a blind spot.
- mifos chooses between two route tables by a value injected at deployment, which the runtime method of Section A.6 finds and the sources leave undecided.

The causes the final version reads fall into three groups, traced here in the source.

The route table was not reached. This is the lazy-boundary cause. An NgModule loaded by `loadChildren` keeps its routes in a separate `*-routing.module` that it imports. A reader that stops at the module file finds nothing, and if the routing module is then scanned at the project root, every screen in it is minted at its own relative path. Four further forms hide a table: a route array held inline by `RouterModule.forChild([...])`, whose elements may name routes (`[ERROR_ROUTE]`) rather than write them; a loader that references its module directly (`loadChildren: () => VaultModule`); a barrel that re-exports the route array; and a wrapper such as mifos's `Route.withShell([...])`, a call whose array argument is skipped whole. A lazy route may also name a module through a barrel that re-exports a feature module, which imports the routing module that holds the table. A reader that follows one hop finds no table and places the screens at the application root (`/breach-report`), where the running router mounts them

deeper (`/reports/breach-report`). Both static readers shared this cause, so their agreement could not expose it and the runtime route oracle did. The final extractor follows the chain to the table, bounded at three hops, refusing at every hop a file the route itself sits in, a file another route loads, and a file whose every table is registered at the application root. A related cause left nodes with no link to the component that renders them. It is the candidate class and is empty on the final version.

The path was named, not written. This is the constant-path cause. In one application every route path is a reference into a constant table, which collapses its graph to a single node when it is not followed. `path: internalRoutes.account.path` is a compile-time string, and so are an enum member and a `$localize` segment. Following the name needs the workspace's `paths` aliases, because the table commonly lives in a library outside the application directory, and the destructuring a route file reaches it through.

The destination was named, not written. A target may be a field, a dotted chain into the route table, an element of a commands array, the return of a builder, or a value in a lookup the component fills. Resolving these is what separates a link from a dead end. A field seeded with a literal is mutable state, not a constant, and the reader does not follow it, because following it would turn a run-time value into a must edge.

Node extraction is therefore not the bottleneck on the final version: every application is complete against the readers' denominator. An analysis that decides where a navigation can fire needs a node for the screen it fires from, and a construct whose source screen has no node is a loss attribution cannot repair. The gaps that remain are declared rather than inferred. The custom matcher and the deployment-time choice depend on information a static reader does not have. The spread of computed routes does not: its routes are in the source, and the extractor does not yet read them.

The corrections to the independent route parser are listed in Appendix A and Table A.1. The final baseline and attributed runs use the same route denominator (Table 5.2).

5.5.2 Construct accounting

Table 5.17 gives the construct accounting in four parts, because a single ratio cannot separate three different things. A site is *recovered* when it becomes a transition with a destination. It is *dynamic* when the extractor records the destination as unresolved: a value from the backend or a history entry, but also a field it treats as mutable although the route table initialises it, or a template literal inside a commands array (Section 6.3); the graph then records an edge into that screen's `u_` sink with modality unknown, rather than dropping or guessing. It is *unplaced* when the destination is readable but the screen it fires from is not, and the edge is anchored at an `o_` origin node that says so. What is left carries a recorded reason but no transition; the check enforces that a reason is recorded, not that it is correct.

Table 5.17: Construct accounting after the loss list was worked. Percentages are of the in-scope sites.

Application	Sites	Excl.	In scope	Recovered	Dynamic	Unplaced
realworld	40	1	39	100.0 %	0.0 %	0.0 %
alfresco	34	7	27	25.9 %	25.9 %	44.4 %
ghostfolio	198	12	186	47.8 %	43.5 %	7.5 %
k8s dashboard	72	4	68	16.2 %	72.1 %	8.8 %
bitwarden	131	19	112	62.5 %	15.2 %	18.8 %
mifos	1343	0	1343	88.2 %	8.3 %	2.5 %
All	1818	43	1775	78.9 %	15.0 %	4.9 %

43 sites leave the denominator, on three grounds decided from the construct's own text before the graph is consulted, so the set is the same in both runs, and the figures are not produced if the grounds do not sum to the total. 35 pass an empty commands array and so change only the query of the current route, and 1 names no destination; neither is a transition between screens. 7 navigate a secondary outlet. These are transitions that the model represents as a second mount (Section 3.2.3) and the extractor does not implement, so they are a declared recall limitation, and excluding them raises alfresco's ratios. Both runs use the same ledger, so the two columns of Table 5.6 are ratios over the same 1775 sites, and the comparison refuses to print if any construct changes its ground between them.

Of 1775 in-scope sites, 78.9 % are recovered, 15.0 % are accounted as dynamic and 4.9 % as unplaced, so 98.8 % are accounted for in all. k8s dashboard shows why recovered and dynamic must be reported separately: a low recovered fraction can coexist with many sites whose destination depends on runtime data. Its dynamic sites are spread over 43 files, 38 of which contribute a single site, so the column reports a habit of the codebase and not one place to fix. mifos supplies 75.7 % of the in-scope sites on its own and pulls the corpus figures towards its own: without it, recovered is 50.0 % and accounted 97.9 %. Changes to the route denominator are recorded in Appendix A.

The named-outlet restriction is enforced on the route side as well. A route that declares a named `outlet` is reached as `(name:path)` and never at the primary address, so it contributes no screen. Otherwise it could take the primary address from the sibling that really renders there, by being declared first, and the graph would name the wrong component at a real address. bitwarden declares such routes (a login and a registration screen each pair a primary component with a second one on a named outlet at the same empty path), but the rule moves no count there. The restriction reaches a named outlet's *subtree* too. alfresco mounts its viewer as `{ outlet: 'viewer', children: [...] }`, and the routes beneath it were minted at the primary address, `/view/:nodeId` and `/search-libraries/view/:nodeId`, which Angular reaches only as `(viewer:...)`. Both tools read the subtree the same way, so the coverage figure could not expose it, and the runtime route oracle of Section A.6 was needed. Neither tool mints such addresses.

Table 5.18: Final-snapshot witness-check outcomes and must conversion. The must column counts edges whose destination was read and passed; the rest are unread destinations, neither passes nor failures, and no must edge failed. "Not applicable" means the application emits no must edge. *Sites cited* counts the source locations the edges cite, *Judged* counts those less the over-approximated and unread ones, and *Clean of judged* is the share of judged locations whose construct names the edge's destination.

Application	Edges	Sites cited	Judged	Clean of judged	Must read and passed	Must conversion
realworld	65	105	65	92.3 %	9/9	7.7 %
alfresco	55	79	23	43.5 %	n/a	0.0 %
ghostfolio	403	685	20	35.0 %	n/a	0.0 %
k8s dashboard	309	441	71	97.2 %	63/125	2.9 %
bitwarden	529	1282	201	59.2 %	1/1	0.9 %
mifos	10811	14300	1076	78.0 %	15/15	1.4 %
All	12172	16892	1456	75.8 %	88/150	1.4 %

5.5.3 The soundness property of the final tier

A gain in recovered share must be assessed together with the evidence for the emitted edges. Table 5.18 reports the final snapshot's witness check beside RQ1's must conversion. An edge without a detected failure is not necessarily a pass, because the checker can leave its cited destination unread. The corpus total of such edges is 62 of 150.

The rules the must tier follows

The witness check compares the destination a cited construct names with the destination its edge names. It never asks whether the construct runs every time its code does, nor whether the router lets the navigation through, which are the two halves of Claim 3.1 that Section 3.1 assigns to the demotion rules. These rules were written during development, partly in response to the objections of a source review and of red-team panels made with the assistant the declaration at the front of this thesis describes. Every defect those reported was reproduced against the code before it was repaired.

Conditions on the construct. A site is demoted when it does not run every time its code does. The conditions are:

- under a guard on the target, a cap from attribution, a catch-all route or a target with holes;
- under `*ngIf`, `@if`, a component mounted under a template condition or an ICU case;
- in a branch, a callback, a `catch` clause, after an `await` or an early `return`;

- in a member no handler calls unconditionally;
- at an address two declared routes can answer;
- before code that may start another navigation.

A construct under more than one condition is filed under the first the reader finds. The template conditions include realworld's own `*ifAuthenticated` directive.

Guards as the router runs them. For each source and destination the extractor computes the guards the router runs: `canDeactivate` on the levels the navigation leaves, `canMatch` on every level it matches, `canActivate`, `canLoad` and the resolvers on the levels it enters, and `canActivateChild` on the levels above an entered one. A level is kept, and its guards do not run, when source and destination pass through the same route object, the level has no parameter the navigation could change, and it does not ask to run its guards on every navigation. A navigation between two children of a parent guarded by `canActivate` therefore runs the children's guards and not the parent's. Gating it on the parent's would treat every navigation as an entry from outside the application. Further rules complete the reading.

- A resolver the navigation awaits holds the edge at may, as a guard does.
- A guard list the reader cannot name still gates, because refusing to read a guard must never be the reason an edge is called deterministic.
- A guard on a child route with an empty path and no component gates the screen above it, because Angular matches such a child on every navigation to its parent. bitwarden's `redirectGuard()` sits on such a child.
- A guard carried by a route a wrapper function returns is read.

Same-address navigation and modality. Angular ignores a navigation to the address its source already displays, and where the address differs only in a parameter or the query it runs only on some inputs. A shell or anchor edge whose destination is one of the screens it stands for is therefore drawn per screen: must from the others, and a may self-loop at the destination. An edge takes the modality of its most certain construct, which becomes its witness; every other construct it cites records the modality it earns on its own, and the witness check judges the must tier only on the constructs that earn must. Claim 3.1 asks that a must edge correspond to *a* navigation that occurs, not that every construct reaching the same screen does. realworld's tag page shows the point: its Global Feed link to the root runs whenever the page does, while two other links from the page reach the root under `*ifAuthenticated` and inside an `@empty` block.

The witness check opens every cited site, not only the first: 27.9 % of the cited sites are later citations of an edge, 58.7 % in the application where the share is largest. In bitwarden such citations targeted `/vault`, where the running router places a component through a route a helper produces (Section A.6). An extractor that did

not read that route fell back to the catch-all's redirect to the root. Under the present rule these citations are recorded as may, which is what they are evidence for.

The deterministic tier and its evidence

Table A.4 in Appendix A.8 records, for each construct a development version's must tier cites, the condition the final extractor reads. It is produced with the extractor's own readers, so it records their decisions and does not corroborate them.

With these rules the deterministic tier is 150 edges with attribution and 17 without it, resting on 25 constructs with attribution and 21 without, in 4 applications; 2 of the 6 have no must edge. Most of the tier is navigation between screens that share a guarded parent, which reading guards as the router runs them lets through. realworld holds 9 edges, its logo links and its Global Feed link into the root. k8sdash holds 125: the logo link and the Create button of the toolbar its chrome component renders above every screen, drawn from every screen under that component except the root (which a redirect answers), the destination itself, and the screens whose `canDeactivate` guard runs when the user leaves them. mifos holds 15, cancel and back buttons that return from a form to the section that opened it, and the menu of its system page. bitwarden holds 1, a cancel button.

The edge count is a poor measure of how much evidence the tier holds: 2 constructs yield 125 of the 150 edges. Without k8sdash the tier is 25 edges on 23 constructs in 3 applications, which is why RQ1 is answered in constructs. Two assumptions weaken the evidence further.

- The k8sdash edges are only as right as the scope of the chrome component. An over-wide scope there would add must edges from screens that do not render the toolbar, and over-wide scopes are not tested outside realworld (Section 5.3.1).
- The must edges rest on an assumption about the order in which route tables registered apart are matched (Section 6.3): 126 of them, in 2 applications (k8sdash, bitwarden). Without it the tier holds 24 edges on 22 constructs.

Each of the 25 constructs was read against the source for the condition around it, the address Angular resolves it to (for a relative link, the route's consumed segments less one per `../`), and the guards and resolvers its navigation runs. The readings are kept in `results-must/tier-reading.json`, and the census refuses to report a must construct that file does not list. This reading was made once, by the AI assistant the declaration at the front describes, and with the extractor's decisions in view. It is a check that each decision can be defended from the source, not an independent second reading.

The witness check finds 0 failures in the 150 must edges and records 62 edges whose cited destination it cannot read. The unread edges are not passes. All of them are k8s dashboard toolbar edges, the same edges whose source side rests on the chrome component's scope and on the route-order assumption, so these weaknesses are not independent. Outside k8s dashboard every must edge was read and passed. Neither the source reading nor the mechanical check is an independent proof of

Claim 3.1. The demotion rules are also exercised by a mutation sweep (173 mutants at `920e1aa`) in which every one of the 173 deletions fails some test. 2 further deletions are not swept: they were retired as equivalent, each deleting a condition already implied by the shape of a route table or by another check. The sweep measures whether the tests detect deletions of the rules, not whether the rules are a complete account of runtime navigation. These checks do not constitute a proof of soundness.

The tier is not a property of the applications. The extractor never evaluates a guard, so a guard that can refuse a navigation demotes it whatever the guard would decide. Extracting the corpus again with every guard taken to let the navigation through gives 194 must edges in 6 applications instead of 150, resting on 69 constructs. 44 of those edges run a guard, and for 40 of them every guard the navigation runs is one the source screen's own routes carry by name, so a user on that screen has passed it at least once on the way there. 29 of the guarded edges are ghostfolio's, which names its authentication guard on each route rather than once on a parent. The small tier therefore says what the extractor can assert without evaluating guards. Evaluating them would move at most 44 edges from may to must. A reader who expected static extraction to recover a large deterministic graph should take this as part of the answer.

The recovered column of Table 5.17 counts sites witnessed by an edge of any tier and must not be read as deterministic recall.

Over the whole graph

Over the whole graph, the figures are counted per cited source location and not per edge, because that is what the check opens. The 12172 edges of the final corpus graphs cite 16892 locations between them, an edge citing several wherever several constructs yield it. Three figures answer three different questions.

- The must column of Table 5.18 bears on Claim 3.1, in one direction: a must edge whose cited site named another destination would refute the claim. The claim's quantification over inputs is carried by the demotion rules (Section 3.1).
- The whole-graph share: only 6.5 % of the locations are clean, because 88.2 % of them are over-approximations for which the check claims no target agreement by design. It is reported beside the must column because 150 edges of 12172 are a small slice of the graph, and a precision figure for a slice is not one for the graph.
- The judged share: of the 1456 locations the check does judge (neither over-approximated nor among the 535 it cannot read), 75.8 % are clean. This is not a defect rate. The judged locations that are not clean are ones where the construct names another destination or none, and none is cited by a must edge.

The 88.2 % figure needs a caveat, because "by design" reads as a property of the method. The edge count is dominated by one line, and the location rate by

Table 5.19: The route configuration the framework holds, against the extractor's node set, as recorded by the runtime route oracle of 29 and 30 October (Table 4.5). *Routes* counts the route entries the configuration holds once every lazy subtree is expanded, and *mounted* the addresses among them at which the router places a component on the primary outlet. *Not mounted* counts nodes the extractor declares at an address the framework mounts nothing at. *Not minted* counts addresses the framework mounts and the extractor has no node for, and *prefix only* those further addresses the framework's configuration holds where a component is placed but no screen of the address's own is: in 1 the route is a prefix with no empty-path child, and in 2 its empty-path child is a redirect, and in neither case is the extractor wrong to mint no screen there. The column is reported so that *mounted* less *nodes* is accounted for in full. *Incomplete* is the extractor's own statement, from its soundiness notes, that part of the route set was beyond it.

Application	Routes	Mounted	Nodes	Not mounted	Not minted	Prefix only	Matcher only	Declared incomplete
realworld	14	10	10	0	0	0	0	no
alfresco	123	30	27	0	3	0	0	yes
ghostfolio	377	326	84	0	242	0	1	yes
k8s dashboard	140	67	67	0	0	0	0	no
bitwarden	170	99	98	0	1	0	0	yes
mifos	935	614	606	0	5	3	0	yes
All	1759	1146	892	0	251	3	1	

mifos. Of the 12172 edges, 44.8 % (5454) come from a single call site in mifos, `view-transaction.component.ts:135`, which navigates to a template literal whose static prefix is empty:

```
this.router.navigateByUrl(`${accountUrl}/actions/${name}`)
```

With no prefix to narrow on, the site over-approximates to every declared route in an application that declares six hundred of them. mifos supplies 88.8 % of the corpus's edges, and that one line supplies half of those. The line inflates the edge count, but over-approximation remains the large majority of cited locations without it, so the rate is a property of how the method handles non-literal prefixes on this corpus, mostly on mifos. An edge count is the wrong unit for anything the thesis claims. The case also argues for a prefix floor: a site whose static prefix is empty tells the analysis nothing, and the model would lose nothing by recording it as a dynamic sink instead of as 5454 may edges.

5.5.4 Nodes at addresses the application does not mount

Across the corpus the compared graph declares 0 screens that the framework mounts nothing at. Over 6 applications, 1759 route entries and 892 nodes, every address the extractor claims is an address the framework's own configuration mounts. This is external evidence about these six applications that no node is minted at an address the application does not mount. It was measured on an earlier graph with the same node total (Table 4.5), which had been corrected using earlier findings of this check.

It is a precondition of the must-tier property and not part of it. An address behind a guard that always refuses still counts as mounted, so the runtime route oracle could not expose such a node. The two static readers could not produce this evidence, because an address both of them place at the wrong prefix is an address they agree on.

The evidence is bounded by what a census keyed on addresses can represent. An address is counted as mounted when the configuration names it as a literal pattern and places a component there on the primary outlet. 65 route entries place a component and are counted in neither column, because they have no such pattern: 1 decides its address through a `UrlMatcher`, and 64 sit on a named outlet, where the address is `(name:path)`. Table 5.20 shows every route entry in its bucket, and neither table is produced unless the buckets sum to the route total and every exclusion that drops a component-bearing entry is named. The matcher case is the more important: a navigation to `/admin/users` in the running application is answered by that component, so the mounted column is a lower bound by at least one address. How many addresses the matcher claims is not published, because reading its body by hand is not a measurement.

The outlet entries were put to the running applications, because an exclusion that drops a component should be shown to drop something no address reaches. Navigating the primary address each of them sits under (`/personal-files` in alfresco; `/namespace`, `/event` and `/clusterrole` in k8s dashboard; `/login`, `/signup` and `/login-with-device` in bitwarden) is recognised as the primary screen in every case, though the observer reads the primary outlet's chain only, so whether an outlet component is also activated there is not measured. Addressing the outlets in Angular's compound form, for example `/login(secondary:)`, produced no recognition event from a signed-out session, which does not show that no address reaches an outlet. The exclusion therefore rests on the ledger and not on the runtime route oracle. The alfresco viewer's entries are the ones Section A.7 already declares outside the node set, and the rest are outside it for the same reason, a region filled by a navigation that names the outlet. Those navigations are the construct the ledger excludes as addressing a secondary outlet, so the two exclusions are the same fact seen from the two ends.

Two further limits belong with the 0 figure, which is the strongest claim in this chapter and should not be read as more than it is.

- "Mounted" is a property of the configuration and not of what a user sees. Some addresses the extractor mints hold a component that renders only application chrome, and the address's own screen is reached by a redirect or by an empty-path child. The runtime route oracle measures two such addresses directly: in k8s dashboard the root redirects, and in bitwarden the root places chrome and no screen of its own. The same shape elsewhere in the corpus is counted as a loss for one application and as agreement for another, decided by what the extractor happened to mint there.

- Where two routes sit at one address and a `canMatch` guard chooses between them, no static reader can say which answers. bitwarden's `/vault` is such a

Table 5.20: Every route entry of Table 5.19, in the one bucket it falls in. *One address* counts entries that a surviving entry at the same address already answers for. *Named outlet* and *Matcher* are the two exclusions that drop a component the application places (64 and 1 entries, 65 in all), and the rest place no component at any address. This table and Table 5.19 are produced only if the buckets sum to the route total and every exclusion that drops a component-bearing entry is named here.

Application	Routes	Mounted	One address	Redirect	Named outlet	Empty outlet	Matcher	No component
realworld	14	10	1	0	0	0	0	3
alfresco	123	30	3	2	66	0	0	22
ghostfolio	377	326	9	1	0	0	1	40
k8s dashboard	140	67	0	3	29	0	0	41
bitwarden	170	99	7	9	13	0	0	42
mifos	935	614	17	32	0	0	0	272
All	1759	1146	37	47	108	0	1	420

pair: the census keeps the first-declared component and the running router activates the other.

Neither limit moves a published number, because the census counts addresses and both members of that pair are leaves. Both are reasons to read Table 5.19 as a statement about the route configuration and not about the rendered interface.

The runtime route oracle also exposes causes that the two static readers share, which their agreement cannot show. In alfresco it returned `/view/:nodeId` and `/search-libraries/view/:nodeId`, a blind spot the two readers held in common: both assembled the routes beneath a named outlet against the primary path, and node coverage was 100 % over a set that contained them. Section A.7 declares the construct as an open case, but declaring it is not the same as being sound over it, since a consumer reading the graph finds a node at an address the built application answers with its catch-all. The final extractor carries the outlet down the walk and mints no screen for such a subtree, so the construct is a loss and not a claim. The second shared cause, a lazy route reached through a barrel, is described with the other causes of the unreached route table (Section 5.5.1).

5.6 Dead links, found from the source

A site whose target the application mounts nothing at is a dead link. The method finds candidates for such links at no extra cost. The candidates are found statically, and each is then checked against the route configuration the runtime route oracle recorded before it is reported.

The clearest case is mifos. A transactions tab links to a journal entry with

```
[routerLink]="['/', 'journal-entry',
'view', 'S' + transaction.id]"
```

The application does declare that screen, at `/accounting/journal-entry/view/:id`, inside the accounting module. The link omits the parent prefix, so the URL it builds is one no route matches. mifos declares `{ path: '**', component: NotFoundComponent }`, so the link does not fail invisibly. It renders the not-found screen, which a user can mistake for missing data rather than a broken link.

The same shape repeats across the application: links to `/loans-accounts/...` name a subtree declared only under an owner (`/clients/:clientId/`, `/groups/:groupId/`), links to `/savings-accounts/...` one declared only under `/centers/:centerId/`, `/clients/:clientId/` and `/groups/:groupId/`, and further calls name segments the application mounts under other names, such as `/users/create` (mounted as `appusers`) and `/system/scheduler-jobs` (`manage-jobs`). A test that never opens the tabs holding these links would not find them, and a crawler would find them only if it reached those screens. Reading the route configuration finds them without running anything.

The list is not one defect class, and the split matters for what each part is worth.

- 18 of the candidates are an absolute link written against a path the application does not mount, usually the name the server uses rather than the one the router does. Those are the application's.
- 5 come from a *relative* `routerLink`, such as `[routerLink]="'edit'"` or `[routerLink]="['../../transactions']"`. Their target depends on resolving the link against the screen it fires from, and getting that base wrong is this analysis's gap before it is the application's defect.
- 4 are neither, because the result files do not record enough of the construct to say which it is. They are counted and not classified, rather than assigned to the half that would flatter the finding.

The last two groups are reported as candidates and weighted lower. Counted as places in the source rather than as targets, the three groups are 18, 3 and 4. The difference is one relative link in a shared template that resolves against three owning screens, so a single mistake yields three candidates at one place in the source.

A further 69 sites are not published at all. The reason is the boundary of the measurement, not the way the sites were read. They sit outside the directory the measurement is scoped to, in ../../libs, ../aca-shared, ../browser, in the repositories of 2 applications. Their files enter the analysis only because those repositories declare a `paths` alias for them. Most are the source of a *sibling front end*, a browser extension built from the same monorepo, whose own router declares the same addresses a census would have called dead.

An accusation needs two things: that the application mounts nothing at the target, and that the link is reached in the application measured. For a file belonging to another front end the second is not merely unestablished but false. The route set these sites would be judged against is the wrong one, and publishing them would report one application's valid links as another's defects. They are counted here and left out of the census.

5.7 Summary

Node extraction is complete against what a static reader can see, and the one-sided runtime route oracle finds no node where the framework mounts nothing. Scope attribution recovers a majority of the constructs the loss rules assign to it, most of them at the may tier, and gains little in must conversion. Its scopes are narrower than the trivial attribution by a factor that is small on some applications, and the evidence that they are not too large is limited to one application's structural reference, and the end-to-end suites test only the missing direction. In the sampled mutants on three applications the graph responded to every scored change in the constructs it represents; mutants on constructs it does not represent are excluded. The final version reads the causes behind the node-extraction defects except those that Section 5.5.1 lists as not read or beyond a static reader. Its deterministic tier is small, rests on a source reading and a mechanical check, and is not proved. The development versions and the corrections between them are listed in Appendix A.1, and Chapter 6 discusses what the results mean for the claim.

Chapter 6

Discussion

This chapter interprets the results of Chapter 5; it does not repeat them. Section 6.1 reads the findings against the research questions. Section 6.2 sets out what limits them, and Section 6.3 declares what the analysis does not decide. Section 6.4 argues how far the approach would carry to other routers, and Section 6.5 says what should come next. The thread running through the chapter is that the extractor loses transitions in two places, at the nodes and at the edges, and that each level limits what the other can achieve.

6.1 Key findings

Two places where extraction fails. Static extraction of a transition graph from Angular source fails in two places, and the first turned out to be the tool's rather than the method's. Node extraction is complete on all six applications against the readers of the route configuration (Table 5.2), and the runtime route oracle observes no node placed at an address where the framework mounts nothing. That shows that no such error was observed, not that none can occur. The runtime route oracle also bounds the first claim: the route configuration it reads holds 251 mounted addresses with no node, 242 of them in ghostfolio (Section A.6), so only 2 of the six are complete in the stronger sense of a node for every mounted address. Both readings are reported: the weaker is what the loss classification is computed against, and the stronger is what a user of the graph would want. The node defects (constant paths, lazy boundaries) are traced in Section 5.2. The final version reads their causes (Section 5.5), and those losses are empty.

What remains is lost at the edge. The navigation sites exist in the source and the screens they would connect exist in the graph, but the extractor cannot decide which screens a site belongs to. The exceptions are sites that the diagnosis labels as having a target screen with no node (the label also covers targets held in a field), most of them in ghostfolio (Table 5.9). Scope attribution addresses this loss at the edge. Under the order in which the loss rules are applied it is the largest loss category in every application (Table 5.4), but that order is part of the measure (Section 6.2), and the ranking is a fact about an extractor, not a limit of the approach. The two levels are not independent: attribution cannot draw an edge into a screen that has

no node, and a complete set of nodes yields few transitions while the sites outside the component a route renders stay unattributed. Reporting only one level would have described a tool that works better than it does, which is why Chapter 5 reports node coverage beside must conversion. The ranking also depends on the unit of counting. The per-application view says what a developer would hit on a given codebase, and the corpus view says where effort pays off across codebases; one large application can dominate the second.

RQ3 and what the loss rules can delimit. A lost construct is assigned to the first loss category that applies (Table 4.3). A construct inside a shared component that appears only in screens without nodes is therefore counted as an attribution loss, although attribution alone cannot recover it. The converse is small: of the constructs attribution recovered, 292 came out of a construct the rules had already assigned to an attribution category, none out of a node category, and the rest out of the residual category to which no rule applied (Table 5.8). The categories are a diagnostic partition of the losses, not a bound on what attribution recovers. On realworld attribution recovered every one of its attribution losses (Table 5.6), and Table 5.9 records no remaining loss there. Elsewhere, two of the remaining reasons are not attribution problems: a target screen with no node, as the diagnosis labels it (the label also covers targets held in a field), is a node-level loss, and a fully dynamic target lies outside any attribution. Two others are limits of the implemented analysis. An owning component that is never reached gives an empty computed scope that the analysis cannot tell from unreachable code. A site in a service or a guard whose receiver or provider form the call graph does not resolve needs relations (dispatch, the guard relation T) that are specified in Chapter 3 and not implemented; a stand-in replaces T.

No construct that the baseline converted was lost (0 leave the converted categories). The gain does widen the must tier: under the demotion rules (Table A.1, rows 14 to 18) it holds 17 edges without attribution and 150 with it, mostly the links of realworld's and k8sdash's shells placed on every screen the shell renders. Whether the edges attribution added at k8sdash occur on every input is what the demotion rules decide; 125 of them are kept at must. The demotion rules were written during development, in part in response to objections from red-team panels run with the AI assistant. The witness check finds 0 failures among the 150 must edges, but cannot read the destination of 62, all of them k8sdash's toolbar edges, so only about half of that application's must edges are read.

The price of the unimplemented model. Navigation sites in services, guards, interceptors, resolvers and effects are a small loss category overall, but in one application they are almost as large as the shared-component category (Table 5.4). Placing them takes the full level of Section 5.3.3, which adds 36 sites to the outlet level; the call graph places only 7 more than the per-class join. Applications that navigate from services therefore need an injection relation of some form, and the dispatch relation is still not implemented.

RQ4 is conditional, and the condition is RQ1. Leaving aside the rename operator, whose detection is close to guaranteed by construction, the other three operators, each generated or scored only at constructs the graph already represents in the simplest form the extractor reads, are detected in 45 of 45 scored mutants on the attributed graph: every deleted `routerLink`, 20 of 20 retargeted calls and 11 of 11 removed guards (Table 5.16). The mutation harness was corrected during development after its misses had been read (Appendix A.1), so the clean result is not an independent test. It does not show that the remaining operators would survive a larger or differently drawn sample. The final sample shares 46 of its 73 mutants with the run whose misses were read (Section 4.3.2), so the two runs share a seed and not a sample. The three mutated applications use different routing styles, and no operator fails under one style and not the other (Section 5.4); the corpus was not stratified by style, so nothing more general follows.

For guards, what separates a detected mutant from an unobservable one is whether any edge enters a screen the guarded route yields, which is the exclusion rule (Section 5.4). The class "guards unreliable in both directions" of Chapter 5 rests instead on what Section 5.5 measured: a guard carried by a route a wrapper function returns, and a `canActivate` element written as a call. Source review in red-team panels found those defects, not the witness check, and the mutation run neither confirms nor refutes the class.

Detection is also conditional on recovery. Against the attributed graph only 6 of the sampled mutants cannot be scored at all, all of them guards the graph can record on no edge (Table 5.15); against the baseline graph 22 cannot. That distance is where RQ1's recovered share shows up in this experiment, and either count is a recovery result and not a sensitivity one. The conclusion is therefore: given that a construct is represented in the graph, every scored change to it was visible, and the probability that it is represented at all is what RQ1 measures. For change impact analysis this form of claim is appropriate, because an impact query is only asked about the part of the application that the model represents, and the graph states which part through its witnesses.

Two uses of the graph. The first, with measured support, is change impact analysis. Every edge carries the file and line of the construct that produced it, so changed lines map to the edges whose witnesses fall inside them, and from there to the screens those edges leave and enter. This is the screen-level counterpart of Chianti [62], in the impact analysis tradition of Arnold and Bohner [4, 12, 44]. RQ4 shows that the mapping reacts to the injected edits for the constructs the graph represents. That is a property such an analysis needs, not a measure of its accuracy, which would ask whether every affected screen is named and how many are named in excess.

The second, not itself measured, is checking in continuous integration at no execution cost. The graph is recomputed from source on each commit, the extraction is deterministic and carries no state between runs (Section 4.3), and nothing has to be installed, served or crawled. Properties such as "a screen still has an inbound deterministic transition" can then be asserted like a type check. A guard assertion is

the member of that list the graph cannot yet carry: for a screen no edge enters there is nothing to read, and an assertion that passes for the wrong reason is worse than none. A dynamically built model cannot be used this way at the same cost, because it needs a running application, test data and an exploration budget [51, 52, 37].

Where both uses stop. Both stop at runtime dynamic routing: a target computed from a value, a route table assembled at startup from configuration, and an overlay no route describes. There the static graph is silent or over-approximates into the may tier, which carries no soundness claim. Under the assumptions of Claim 3.1 the must tier is a lower bound on the navigation an application can perform; the may tier is not. A query that must not miss a transition therefore cannot be answered from the graph alone, and the graph's witnesses and declared approximations tell a consumer which queries those are. Authentication bounds the deterministic tier in the same way. Without a backend, 776 of the 806 screens the rendered-interface check visited in 5 applications redirect to a sign-in page (Section 5.3.1). A guard decides on state the source does not fix, so a reader that keeps Claim 3.1 demotes it, and the small must tier is in part this fact and not a defect of the extractor. Taking every guard to let the navigation through raises the tier from 150 to 194 edges (Section 5.5). A consumer that knows its user is signed in may read those edges under that assumption, which then belongs to the consumer and not to the graph.

6.2 Limitations and threats to validity

The results are measurements of one tool, on one corpus, with five methods and an end-to-end check on two applications, performed by the person who wrote the tool. Each of those four choices limits what the figures support.

The tool was developed together with this evaluation, on the same corpus. uigraph was built for this thesis and corrected while it was evaluated (Appendix A.1). Every figure measured on the six applications is therefore in-sample: the extractor, the independent route parser, the mutation harness and the structural reference were corrected against the applications they are scored on. The structural reference guided two repairs of the extractor. The end-to-end check comes closest to an out-of-sample test. The extractor was fixed at its final commit before the suites' logs were read, and nothing was repaired after; the archive cannot corroborate that order, since the preregistration and the logs were committed together. No held-out application measures how far the corrections generalise.

The scope is compared in both directions on one application only, and only against the analysis's own relations. Every other method in Chapter 4 checks a node, an edge or a witness; none can be asked whether the set of screens attribution computes for a site is the set it fires from. Three methods bear on it, each partly.

- The hand-read structural reference (Section 5.3.4) answers in both directions, but for one application of six and only as agreement with the analysis's own

relations. The analysis finds 100.0 % of its (site, screen) pairs at 100.0 % precision. The *class* level, the per-class rule the call graph replaced, asserts 9 pairs outside it at one site; the call graph was written after that finding and is scored on the same ground truth, so the clean figure is consistent with the repair but does not test it independently. The levels that stand for earlier work, reimplemented in this extractor, recover 94.7 % (shell rule) and 99.5 % (with the enclosing routes) of the pairs and, being nested inside the full level, hold none outside it.

- The end-to-end check asks the same half of the question of the navigations two applications' own suites perform. On realworld it does not discriminate: the shell-rule level, which stands for earlier work, already contains all 19 observed pairs (19), among 100 possible ordered pairs of which the attributed graph holds 79. On mifos the graph contains 12 of 22 transitions, against 10 without attribution. Of the 10 misses, 3 fall in the preregistered redirect class, where the graph holds the pair on the address the navigation landed on. The other 7 are defects of the tool and none is repaired: 3 are a relative link in a template shared by a screen and its child tabs, resolved against the child's address, and 4 sit at places the extractor fails to read (a template literal inside a commands array, `relativeTo` passed through a variable).
- The rendered-interface check covers 5 of the six applications, answers half of the question, and ran on an earlier graph than the final call graph and on 30 of 806 screens (Section 5.3.1). Of the 33 (screen, target pattern) pairs the applications render on the screens it could judge, 29 are predicted by the edges leaving that screen. That tests the screen's pooled outgoing destinations, not each site's scope. The 4 that are not predicted turn out to be sites attribution declined to place, not sites it placed wrongly.

What remains unvalidated is the other half and the reach. A site attributed to too *many* screens produces edges whose witnesses all pass, because the construct really is there and really names that target. The rendered-interface check cannot convict it, since a predicted link missing from the DOM is equally a link rendered only to a signed-in user. It could convict only predictions that name a destination exactly: 7 on the screens it could judge, against 102 that over-approximate. The direction is unrefuted but close to untestable by this method. Driving the applications far enough to render their interiors (seeding a backend, signing in, visiting a guarded screen) would lift both limits, and would do most to change what the thesis can claim about its own contribution. Without a running application only the width of the scopes can be measured (Section 5.3.2): on k8sdash the average site is attributed to 59 % of the screens, and a scope that wide says little about where a site fires, even if it is sound. The recovered share that attribution gains comes at the cost of that width. The same untested direction bears on the must tier: most of k8sdash's must edges, one per screen its toolbar is attributed to, are as right as that one scope.

Construct validity. The denominator of must conversion (Section 4.2) over-counts, since it contains constructs in code the application never mounts, and it under-counts, as follows, so the figure is not a bound in either direction. The independent route parser recognises navigation by four syntactic patterns: a `routerLink` attribute or binding, and a call to `navigate` or `navigateByUrl` on a receiver whose name

contains *router*. Navigation expressed any other way is counted as neither a loss nor a success. This includes:

- navigation wrapped in a helper such as `this.nav.goHome()`, which contributes one construct at the wrapped call site and none where it is invoked;
- an assignment to `location.href`, and a redirect driven by a server response;
- navigation performed inside a third-party component.

The ratio is therefore recall against an enumerable syntactic proxy for an application's navigation, and the completeness of that proxy is not itself measured. The test-source cross-check is the only method that sees past it, because a URL the application's own end-to-end specifications name is an address its authors expect to exist. It reads the specifications without running them, so a URL there is named and not observed, and it covers one application. The end-to-end check observes completed navigations, but tests whether the graph contains those transitions, not whether the construct count covers the application's navigation.

Node coverage carries a stronger form of the threat, in the opposite direction. Its denominator is the screens the independent route parser can address, so a screen whose path neither tool can read is outside the ratio and not counted against it; refusing to read more raises reported coverage. Screens whose paths are assembled inside a template literal from an exported constant, as three of one application's declared screens are, fall outside the ratio when neither tool reads them, and coverage can then read as complete (Section 5.5). Nothing in the ratio would say so. Every coverage figure is therefore printed beside the count of routes the independent route parser knows are there and cannot address.

The two units stand in a many-to-many relation, which is why the ratio counts constructs and not edges (Section 4.2). Several constructs can yield one edge, the extra ones recorded as merged witnesses. The witness check judges every location an edge cites, so a construct counts towards the ratio only on evidence the check has read; merged citations are 38.1 % of those in realworld's attributed graph and 58.7 % where the share is largest (Section 5.5). The ratio and the precision figure still answer different questions, which is why they are reported separately.

The graph also loses guard information. Constructs merge into one edge only when their guard texts are equal, so a merge never discards a guard. The loss comes earlier: a structural condition in a template, or a condition around a navigation call, demotes the edge to may but is not written into its guard (Section 3.2). A consumer learns which router guards run, not under what condition in the application the navigation is reached.

The loss breakdown has a second construct threat: the precedence order of the rules is part of the measure. A construct outside a route component is filed as attribution whatever its target, and 101 constructs in the attribution categories have a target that the literal test of the loss rules does not accept. Testing the target first would make the target category larger than attribution on 2 applications (alfresco and k8s dashboard). Category sizes are comparable within one application and under one order, which is what Chapter 5 claims, but they are not independent quantities: a different order would move constructs between categories without any change to the tool.

Internal validity. Extraction runs on source trees whose dependencies are not installed, so the TypeScript compiler cannot resolve types from the framework or third-party packages; type-directed steps such as narrowing an injected receiver are weaker, and the figures are properties of that configuration. The mutation experiment has small samples per operator, in single digits for several operator and application pairs, drawn with a seeded quota, so the pattern of which operators are detected is more trustworthy than the rates [58]. The exclusion rule uses the graph itself, which couples the sample to the tool's own recovery; the excluded mutants are reported, but the sample is not independent of the artefact under test. The test-source cross-check measures node existence and nothing finer. Several of the forms the independent route parser reads it learned from the extractor (Section 4.2), which is why the external check rests on the runtime route oracle.

External validity. The corpus is six open-source Angular applications pinned to single commits. Proprietary applications may differ in size, routing style and navigation mechanisms [35, 36]. The selection is tool-driven: `SAP/spartacus` was rejected because its routing is configuration-driven in a way the extractor does not support (Section 4.1), so figures measured on the corpus are likely optimistic for Angular applications at large. The candidate pool also showed an inverse relation between application size and end-to-end test density. The evidence from outside the extractor's own reading is uneven. URLs named in end-to-end test sources (read and not run) come from alfresco alone. The runtime router configuration was read on all 6 applications. Rendered links were observed, on an earlier graph and in part, on 5. The end-to-end suites were observed on realworld and mifos, covering 22 distinct transitions of an application with $\geq$909 route entries. The findings about scale therefore rest mainly on the runtime configuration and the extractor's own reading. The results are Angular results; the graph model and the lattice do not mention Angular, and Section 6.4 argues about other routers, but that argument is not a measurement.

Oracle independence. The independent route parser shares no code with the extractor, which makes it independent and says nothing about whether it is right. The independence is also partial (Table A.1): for the route-path forms added during development (a table behind a freezing call, a path assembled from constants, a module reached through a workspace alias), the two tools apply the same rules. Their agreeing about such a path is weaker evidence than their agreeing about a path each reads its own way. The corrections of the independent route parser are listed in Table A.1. A differential test only shows that two implementations disagree, never which is right, and each disagreement was resolved by reading the source, by the person who wrote both implementations.

Experimenter bias. I wrote the extractor, designed the evaluation, selected the corpus and defined the defect classes. The mitigations applied are these.

- The six selection criteria were fixed before any results, and rejected candidates are reported with their reasons; one rejection was on tool support, a ground outside the criteria, and two retained applications do not meet the test-suite criterion (Section 4.1).
- The witness check is mechanical and re-parses the source at the claimed location.

It checks the witness and not the address: an edge whose target node sits at a path the application does not mount satisfies it.
- The mutation oracle is the authored delta, so it does not consult the tool's output.
- Commits and harness version are pinned, determinism was checked before any measurement, and the result tables are generated from result files (Section 4.3); the two tables compiled by hand are the defect-class table and the table of the witness check and the test-source cross-check.
- The protocol corrections are reported in full, including those that made the figures worse (Appendix A.1).

Several controls were not applied:

- no blinding, no second annotator for the loss classification, and no independent audit of the residual constructs inspected by hand;
- the defect classes were defined after the first observations, not in advance;
- the source-level reading of the final must tier (its 25 constructs) was made by the artificial intelligence (AI) assistant declared at the front, which also helped write the extractor and the independent route parser. It was made with the extractor's decisions in view and no human second reader checked it. Its agreement with the extractor therefore shows consistency with the stated rules, not independent correctness;
- uigraph was developed on, and is measured on, the same six applications, with no held-out application, so the figures are fitted to this corpus.

The requirement of two lines of evidence per defect class is the strongest control on the last point, and the reason one class is still a candidate. It is weaker than it appears: Table 5.3 shows that four of the five accepted classes rest on one named method, the rest of their evidence being observations, so for those four the control is one method plus my own judgements.

The runtime route oracle. The runtime route oracle of Section A.6 removes the threat that mattered most, that two readers of the same route files agree on a wrong address and the agreement is read as a result. It brings in four others.

1. A selection effect: the method needs an application that builds and boots, and a corpus chosen partly for being buildable is not chosen at random.
2. It reads the route configuration and not the rendered page, so a route behind a guard that always refuses, or whose component throws on render, counts as mounted.
3. The build was a development build in each case, because Angular removes the debug surface the method reads in production mode, and a production configuration can differ in ways a route table does not show.
4. Where a workspace application composes the library a measurement is scoped to, the framework's set is a superset of the library's, and part of any difference is scope and not reading. Two of alfresco's three differences are of this kind: one is declared by the shell application and one by a package it installs. The third, `/about`, is declared inside the measured library in a JavaScript Object Notation (JSON) plugin manifest no reader of the code can see.

These threats do not make a mounted address look unmounted; they weaken the

claim in the other direction. The claim is that an address the extractor mints is one the framework mounts, tested by finding the extractor's set inside the framework's. Threats 2 and 4 make the framework's set *larger* than what a visitor can reach, which makes containment easier to satisfy. Threat 3 moves the set in no fixed direction, since a production configuration can add routes as well as drop them. Both results are statements about the development router configuration that was recorded, under the abstraction of Section A.6. The oracle ran on an earlier graph whose node total equals the final one (Section 4.3.3). The compared graph mints 0 such addresses, which is the absence of a failure on that configuration and not a proof of its impossibility.

6.3 Declared approximations

This section lists the constructs the analysis does not decide, so that a consumer of the graph can tell which queries it may ask. The thesis claims soundness only for the deterministic tier and only over the declared route set (Claim 3.1), and pairs that claim with this list in the soundiness position of Section 2.6.2 [48]. The list separates constructs read exactly, where a failure is a defect that Chapter 5 counts, from constructs approximated, where no completeness is claimed. Constructs of the approximated group still count in the construct denominator of Table 4.2, where they appear as dynamic, unplaced or attribution losses. A declared approximation can still cost transitions that the suites take; the end-to-end check measures that cost on mifos (Section 5.3.5). Two of the approximations were found by the runtime route oracle of Section A.6 and not by reading the analysis, and they come first because both are places where the analysis is silent rather than approximate.

A route may be declared outside the language. alfresco's about page is registered by a JSON manifest that names a path and an identifier, which an extension registry binds to a component class at run time. No reader of the application's TypeScript can see such a route, and none raises a note, because nothing in the code is unreadable: the route is not in the code at all. Covering it would require reading the framework's plugin format as a second source language, which is not attempted here.

Which of several tables a prefix mounts may be decided outside the sources. mifos holds two route tables for its users module, one per identity provider, and selects between them with a constant assembled from `window.env`, a value injected into the page at deployment. Where the sources fix the condition of a conditional, the extractor reads it: a flag written in an environment file is resolved and the arm it selects is read. Where they do not, as here, it declares the subtree unread. The loss is declared and is 5 screens wide. That is the price of the rule that decides the question: the table a prefix mounts is the one its route names, or the one the module registers with `forChild`, or the only one in the file, and otherwise none. Reading every table a file holds would have given this prefix the right addresses by accident of the two tables agreeing, and would give any prefix every table a file holds, minting the cross product of prefix and path wherever two tables differ.

Where such a condition *is* fixed by the sources, the value read is the one the

checked-out sources carry. An Angular build may substitute one environment file for another, so a constant a production build fixes differently selects a different arm there. The measurements are taken over the sources as checked out, and the runtime route oracle reads a development build for the same reason, so the two agree about which configuration is measured.

Read exactly are the following, and everything in this group is the analysis's responsibility.

- Route entries whose path is a string literal, together with the component they render.
- `routerLink` bindings whose value is a literal or an array of literals.
- Calls to `Router.navigate` and `Router.navigateByUrl` with a literal target.
- The nesting relation obtained from element selectors resolved within the importing file's scope.
- The primary `<router-outlet>` with the shell rule that follows from it.
- Guards named on a route, as labels on the edges entering that route's subtree.

The defects of Section 5.2 (constant route paths, lazy boundaries, unlinked nodes) lie inside this group. They are bugs with known causes and not approximations to be declared away, which is why the cause analysis of RQ5 can say that the final version reads them (Section 5.5).

Approximated are the following constructs, each with the reason it is approximated and the effect on the graph.

- **Components created programmatically, including dialogs and overlays.** An overlay is a user interface state that no route declares, so it has no screen node, and no edge records that a screen opens or dismisses it. The component implementing it is reached from no template, so its sites have the empty scope. With attribution, each such navigating component is given an unplaced origin that carries its file, and its sites become edges from that origin, held at *may* (Section 3.1). A navigation in a service that only such a component injects is recorded only as a note. The graph lacks the overlay state, the transitions into and out of it, and the screens from which its exits fire, not the destinations of those exits.
- **Named outlets and targets of the form `{outlets: ...}`.** An auxiliary route adds a second active route and a second mount point, encoded in the URL as `(name:path)` [27]. The model of Chapter 3 represents this as a second mount, and the adapter does not implement it. A route that names a non-primary outlet, every route beneath it, and a route whose outlet the reader cannot fix to a string mint no node; the last kind is counted in the note that marks the route set incomplete. A commands array that carries an `{outlets: ...}` element yields no edge and no note. In `navigateByUrl` an auxiliary group is dropped and the edge to the primary path is held at *may*, since the navigation fails when the auxiliary route does not match and that route is not read; a group that also carries the primary outlet yields nothing. A later string element of a `navigate` command is one segment whatever it holds, so an auxiliary URL written there reaches whichever route matches it as one segment. The test-source cross-check shows the auxiliary URLs among its misses.

- **Navigation relative to the active route (`relativeTo`).** The target is a function of the route of the component that holds the site, and attribution is what makes it resolvable: it is resolved once for each screen the edge is emitted from. A template `routerLink`, and a `Router.navigate` call whose options are an object literal with a `relativeTo` key, are resolved against that screen's path, applying a leading run of `.` and `..` segments; a call without `relativeTo`, and `navigateByUrl`, are resolved against the root. A target with more `..` segments than the route has segments is one Angular rejects, so it yields no edge and a note. A `..` that follows another segment, as in `a/../b`, is applied by Angular but kept as a literal segment by the extractor, so the edge falls to the catch-all or to a note, not to a wrong screen. The base is right only when the screen is the route of the component that holds the site, and only then can the edge be must. It is held at may in these cases:
  - a component rendered under a route it does not own, whose own route the analysis does not track;
  - `relativeTo` written in shorthand, passed through an options variable, or given a route other than the component's own such as `this.route.parent`, none of which the reader evaluates (it accepts only a constructor parameter or an `inject` call that yields the component's `ActivatedRoute`);
  - a link that binds its own `[relativeTo]` input;
  - an edge emitted from a node without a path of its own (a scope anchor, the shell or an unplaced origin), where a bare relative target is matched as if absolute.

  Such an edge can name the wrong screen, which the may tier admits and the must tier does not, and a target that matches nothing under the assumed base is recorded with a note that says so.
- **Templates passed as values (`TemplateRef`) and rendered elsewhere.** The analysis does not follow a template value from its declaration to its instantiation. Links inside an `<ng-template>` are read from the declaring template and attributed to the declaring component's screens, and every such link is held at *may*, since nothing says whether or where the template is instantiated. The attribution is right when the template is instantiated within the declaring component's own subtree, as with a template passed to a child's input or to a dialog opened from that screen. When the value is carried to a component rendered on other screens, the edges stay with the declaring screens and not the screens on which the template appears. Recovering the place of instantiation is a dataflow problem, the same obstacle as for JavaScript XML (JSX; Section 6.4).
- **Dispatch through NgRx effects and router event subscriptions.** The dispatch relation is specified in Chapter 3 and not implemented, so the screen that dispatched an action is never recovered. At the levels that keep the call graph or its class join, *full* and *class*, the extractor stands in for it: every navigation call in a file inside the application directory that contains a `createEffect` call is given $\top$ and offered from the shell at *may*, or into the shell's `u_` sink when its target is fully dynamic. This is sound and coarse, since an effect can fire on any screen. At the lower levels the stand-in is not

applied, and an effect outside the application directory, or declared with the older `@Effect` decorator, is treated as any service call. A navigation inside a `Router.events` subscription is attributed to the scope of the class that contains it, as any other call site, and held at *may* because it runs in a callback (Section 5.5).

- **Navigation inside signal effects and observable subscriptions.** The firing condition is a data dependence in a graph the runtime builds, not a control-flow condition in the program text (Sections 2.7.5 and 2.8). The model gives such edges the modality *unknown*, with the read set recorded as the symbolic guard; the extractor implements neither. A navigation inside an `effect` body, a `subscribe` callback or any other function something else calls is attributed to the scope of its class and held at *may*, because it runs only when that function is called (Section 5.5); a fully dynamic target goes to the `u_` sink. The graph records that the screen can be left for that destination, not under what condition.
- **Computed navigation targets.** A target built at run time from values cannot be enumerated statically in general, and the analysis treats it according to the static text it contains.
  - A target with no static part, such as a `routerLink` whose value is an interpolation among them, is an edge of modality *unknown* and confidence 0 into the `u_` sink of the node it fires from. A template literal used as an element of a commands array is read as a variable element.
  - A target with a static leading part (the head of a template literal, the left operand of a concatenation, or the constant elements before a trailing variable element of a commands array) is over-approximated into the *may* tier, with one edge to every declared route that extends that prefix by at least one segment. An empty prefix therefore connects the site to every declared route except the root. The over-approximation is incomplete: a prefix segment matches only an identical route segment, so a route with a parameter in that position is not a candidate, and a prefix that matches no route yields no edge and no sink, only a note that gives the count as zero.
  - A commands array whose variable element is followed by constant elements is read as a pattern and drawn as a single *may* edge to the first declared route that matches it, with the variable element matching only a parameter segment. A route whose segment in that position is a constant is not reached, and no note records the omission.
- **Injection tokens and provider bindings.** The call graph resolves a call on a field through the field's declared type: the class it names, the classes a provider declaration binds it to with `useClass` or `useExisting`, or the subclasses of an abstract class no provider binds. Three things are not read.
  - A binding is taken application-wide and not from the injector that resolves the token, so where two components provide a token differently both implementations are joined, which widens a scope and never narrows it.
  - A factory or value provider and a provider array a function returns from outside the application bind nothing the pass can follow, so a call through

such a token contributes no edge and falls back to the join over every component that injects its class, or to no scope at all if none does.
  - A receiver is resolved when it is a field of the class, an `inject(T)` call, or a local variable or parameter initialised with `inject(T)` or declared of an application type; any other receiver is not resolved.

  Over the corpus the pass resolved 5574 calls on application types to the class the type names, 4937 through a provider binding and 18 to subclasses, and left 1111 unresolved, in 2 of the six applications.
- **Configuration-driven route registration.** A table passed as an array to `RouterModule.forRoot`, `forChild` or `provideRouter` is read wherever the registration runs, including inside a provider factory the application calls, which is how alfresco's folder-rules library mounts `/rules`; a table held in a variable is also found by its shape. A route that exists only once configuration has been evaluated at startup is not recovered. Where a route object is declared and its `path` is supplied by configuration, the path is counted as unreadable, no node is minted, and the route set is declared a lower bound. Where the table is computed by a function and passed to a registration call, provided through the `ROUTES` token, installed with `Router.resetConfig`, or declared in a JSON extension manifest (Section A.6), the routes are absent from the graph and no note is raised, so the route set is not declared a lower bound on their account. The independent route parser behaves identically on each of these forms, so the two readers agree on routes neither can see. The same approximation is why one candidate application was rejected (Section 4.1).
- **Guards that would let the navigation through.** A guard is a function of the application's state, and the extractor does not evaluate it. A guard the navigation runs holds the edge at *may*, even when the guard would always pass. The guards a navigation runs are the `canMatch` of a route its address is matched through, the `canActivate`, `canLoad` or resolver of a route it enters, the `canActivateChild` above those, and the `canDeactivate` of a route it leaves. A route the source and target share is taken to be reused unless its path has a parameter, `runGuardsAndResolvers` asks for more, or the navigation does not keep the segments the route consumes; a parameter that does not in fact change makes this too cautious. A guard is identified by the text of its expression, so two guards that compute the same thing are two guards. Content projected through `<ng-content>` is read as its host template places it, so a link projected into a component that renders it only under a condition is read as unconditional; `mat-menu` and `mat-tab` are the only library containers read as rendering their content lazily. A route component behind a conditional outlet is conditional, but a component nested in it is judged by its own placement, and the application root's outlet is not read.
- **Navigations a later one may cancel.** This item and the one on modifier keys are the execution model of Claim 3.1: the claim holds only on executions they do not describe. A navigation is demoted when code that runs after it, before the event is handled, may start another (Section 3.1). Whether that code may navigate is read through the calls the reader can resolve to a body in the application: a member of the class, a function of the file, a method the

checker resolves on a typed receiver. A call into a library, through a receiver the checker cannot type, or into code loaded at run time is taken not to navigate, so a navigation followed by such a call can be published as must although the call cancels it. The reader does not follow a navigation a handler schedules for later (a timer, a promise it does not await), nor the other listeners one event reaches: a click handler on an ancestor the event bubbles to, or a `(click)` binding that navigates on the same element as a `routerLink`.

- **Children of a route a function returns, mounted at the empty path.** The first empty-path child under a screen, and the first below that one, render on the screen's own address, and their links are drawn from it. Where the empty-path route is written inside a function that returns it, as `withShell([...])` does, the walk does not place the component the function's children name under the screen above, and its links stay on an unplaced origin at *may*. The graph loses those edges and asserts nothing false about them.
- **Activation with a modifier key.** A `routerLink` on an anchor leaves a click made with a modifier key held, or with a non-primary button, to the browser, which opens the address in a new browsing context; so does an anchor whose `target` names another context. The model reads every activation of a link as a navigation of the application, so a must edge from an anchor holds for a plain activation and not for these.
- **The order in which Angular matches routes.** The router takes the first route, in declaration order, that matches an address; the matcher prefers an exact path to a parameter route and consults a redirect only when no route matches. Wherever two declared routes, or a route and a redirect, can answer an address, the edge is held at *may* and may name the route Angular does not take. One exception rests on an assumption. Route tables an application registers apart are joined in the order their providers are registered, which the extractor does not read. It takes a catch-all in one such table to be matched after every route of the others, the order Angular's documentation recommends and the one the corpus applications appear to keep, and it names each catch-all it dismissed this way in a note. Extracted without the assumption, the must tier holds 24 edges on 22 constructs instead of 150 on 25: the other 126 edges, on 3 constructs in 2 applications (k8sdash, bitwarden), rest on it.
- **Ambiguous selectors in the nesting relation.** When a selector belongs to more than one component, the candidates are narrowed to those the rendering file imports; where none is imported, the analysis joins over all of them and records a note, which can attribute a site to screens on which its component is never rendered. Attribute selectors are matched as element selectors are. When the question is whether a component renders an outlet through a component it places, an ambiguous placement counts only if every candidate renders one.

Table 6.1 gives the prevalence of the approximated constructs in the three applications that were cloned for the mutation run. The in-scope kinds are not repeated, because the construct denominator of Table 5.1 already counts them with the independent route parser. The counts come from `approx-counts.mjs`, which is part of the harness and runs over the application sources with tests excluded. It counts call sites, which is a different procedure from the independent route parser that

Table 6.1: Prevalence of the approximated constructs in three of the six applications, produced by `approx-counts.mjs` in the harness. The figures are call-site counts over the application sources with tests excluded, taken at the pinned corpus commits. They are a different unit from the construct denominator of Chapter 5, which comes from the independent route parser.

Construct	realworld	alfresco	bitwarden
`dialog.open`	0	3	0
auxiliary outlets	0	7	0
`relativeTo`	1	1	7
`TemplateRef`	2	0	11
`createEffect`	0	49	0

produces the construct denominator of Chapter 5, so the two are not comparable term by term. Three of the six applications were counted, so no corpus-level prevalence is claimed.

Two things follow from the counts. The approximated kinds are few in realworld and bitwarden against their construct denominators (Table 5.1: 40 and 131), so they do not explain the recovery gap of those two. In alfresco the approximated sites outnumber the 34 constructs, almost entirely through `createEffect`, so there they cannot be set aside; the units differ, since the denominator counts navigation constructs and the `createEffect` figure counts effect declarations, whether or not they navigate. The kinds are also concentrated, in different applications, so an approximation that costs nothing on one application can be the dominant one on the next. That is an argument for declaring the list and not reporting a single aggregate. Call-site counts also understate the structural cost of an approximation: a handful of `dialog.open` call sites can open many components, and the inspection of the owner components that attribution never reaches found many opened programmatically. The right unit is the number of components and navigation sites an approximation strands, not the number of calls that strand them.

6.4 Transfer to other frameworks

All measurements in this thesis concern Angular. This section argues how much of the approach would carry over to React with React Router and to Vue with Vue Router [61, 81]. It is a qualitative analysis and not a measurement: it compares the documented behaviour of the three routers with the input relations the analysis needs, and yields hypotheses at the level of evidence of a design argument. Showing that the approach performs as well on React or Vue would require a corpus for each framework and the methods of Chapter 4.

The analysis has a framework-independent part and a framework-specific part. The graph model, the lattice of scopes, the fixpoint, anchored edges and the expansion query do not mention Angular. What is specific to Angular are the four input relations of Section 3.2: component nesting, router outlets, the call graph with

Table 6.2: Constructs that provide the input relations, by router.

Concept	Angular	React Router	Vue Router
Outlet	`<router-outlet>`	`<Outlet />`	`<router-view>`
Several outlets	named outlets, encoded in the URL	none	named views, not in the URL
Declarative link	`routerLink`	`<Link to>`	`<router-link :to>`
Imperative navigation	`Router.navigate`	`useNavigate()`; redirect from a loader	`router.push`
Route guard	per route (`canActivate`, ...)	loader that throws a redirect	per route (`beforeEnter`) and global (`beforeEach`)
Access to services	hierarchical injectors	context and hooks	`provide`/`inject`, composables, plain imports
Reactive trigger	signal effect	`useEffect`	`watch`, `watchEffect`

dependency injection, and dispatch relations. Moving to another framework means deriving the same four relations from that framework's constructs. Table 6.2 lists the corresponding constructs.

Outlets transfer directly: in all three routers a parent route renders a placeholder where the matched child is inserted. The components above the placeholder form a shell that is present on every child screen, so the scopes $\top$ and $\mathit{subtree}(r)$ have the same meaning. Vue's named views do not appear in the URL, so they add a second mount point but no auxiliary route, and React Router has no equivalent.

The call graph is easier in React and Vue. The hardest Angular-specific step is resolving an injected service through the injector hierarchy. In React, a navigation function is usually obtained from `useNavigate()`, and a hook can only be called while a component renders, either directly or through other hooks. Every such navigation site therefore has an owning component that can be found by following hook calls, and no injector needs to be resolved. Vue composables behave the same way inside `setup`. The case that remains is code that imports the router instance directly, which Vue applications often do in stores and HTTP interceptors. This is the analogue of Angular's service case: the site has no component of its own, and its scope is the join over its callers. Since the link is a plain module import, it can be resolved with ordinary import resolution, as in the JavaScript call graphs of Feldthaus et al. [23].

Redirects do not transfer, and the asymmetry was found by porting the model's reading. Angular's validator refuses a route that declares `redirectTo` beside `children` or a `component`, so a redirecting Angular route has neither a layout nor a component to lose and the model's step from "redirects" to "renders nothing" costs nothing. Vue Router applies a record's redirect only when that record is the

last one matched, so a redirecting parent with children does render its component for a matching child. Carrying the Angular reading into the Vue extractor dropped those components; the extractor now keeps the component of a redirecting record that has children. It declares one residue as a note at the parent rather than modelling it as an edge: navigating to the parent's own address lands on the redirect target with that component unmounted. The same porting exposed a second Vue-only hazard: a table that mixes the Vue 2 catch-all `'*'` with the Vue 3 form `'/:pathMatch(.*)*'` folds both to one address, and the extractor keeps the record declared first. Where that is the Vue 2 form, which Vue Router 4 rejects, the surviving component is the wrong one. The collision is declared, but which record to keep is not something the table alone determines.

Guards move to different scopes. A redirect thrown or returned from a React Router loader runs when its route is entered and again when its params or search change; a parent whose match is unchanged is by default not revalidated, the analogue of Angular's route reuse. It plays the role of `canActivate`, and as for an Angular entry guard its source is the screen being left, not the loader's route: a navigation from `/public` to `/admin` that the loader redirects to `/login` leaves `/public`. Its scope is therefore the join of the origins of the navigations that may run the loader, as the relation T gives for entry guards, and not $\mathit{subtree}(r)$; where those origins are not recovered, the conservative scope is $\top$. Vue's global `beforeEach` guard corresponds most closely to a `canActivateChild` on a root route with no component; a redirect from it is anchored at $\top$, which the lattice already contains.

Component nesting is harder in React. Angular and Vue keep templates in a separate language, so the nesting relation can be read from element names. In React, components are created in JSX, which is ordinary JavaScript. A component can be stored in a variable, passed as a property or as `children`, and rendered under any condition the language can express. Recovering the nesting relation then becomes a data flow problem closer to points-to analysis than to reading a template, which is the main place where a React version would need new work.

The reactive boundary is the same. A navigation inside `useEffect` or a Vue watcher fires when its dependencies change, so, like a signal effect in Angular, it would be given modality *unknown*.

The lattice, the fixpoint and the anchored representation transfer unchanged. Of the relations, outlets and guards transfer with small changes, the call graph becomes simpler, and component nesting becomes harder in React. These are expectations derived from the routers' documented behaviour, and testing them on real applications is left for future work (Section 6.5).

6.5 Future work

This section looks forward; the development versions and the corrections between them are listed in Appendix A.1. The topics are listed in the order of how much each would change what the thesis can claim.

An oracle for the screen set on a large application. This is the cheapest addition that would strengthen the thesis most. The hand-built ground truth of Section 5.3.4 covers only the smallest application, and reading every site by hand is not practical on an application of hundreds of screens, where most of the corpus's (site, screen) attributions are made (Table 5.11). Two routes lead there: driving such an application into a signed-in state and reading its rendered links, or annotating a stratified sample of its sites instead of all of them. Recording, for each attributed site, the relation that supplied its scope would narrow the verdict further, since it would show which *rule* made a site wrong without re-extracting once per rule.

The remaining node-level losses. The final version reads the causes of the node defects of Section 5.2. Some attribution losses that remain, most of them in ghostfolio (Table 5.9), are constructs whose scope was found but whose target screen has no node. Those behind the spread of computed routes could be recovered by reading more of the route configuration, with no change to the attribution pass; this has not been tried. Any behind the custom `UrlMatcher` could not, because no static reader of a route table can address that screen.

The unimplemented part of the model. Chapter 3 specifies two things the prototype leaves out.

- The dispatch relations that carry NgRx effects and router event subscriptions.
- The injector hierarchy that the implemented call graph reads past, since it takes every provider binding as applying application-wide (Section 6.3). Reading bindings relative to the injector that resolves them would narrow the scopes, and the fixpoint and the complexity bound already cover it. The call graph follows class hierarchy analysis in the style the framework-modelling literature recommends [17, 68, 3].

Guards that return a `UrlTree` belong to the same group, since a redirecting guard is itself a navigation site and not only a label. The wider treatment of guards in Chapter 3, where the guard of an edge is the conjunction of the conditions on the containment path with the guards the router runs, is the more substantial piece of work; its test is the guard-removal operator, which on this corpus reaches only guards the graph already records on an edge (Section 5.4).

Dialogs and overlays as interface states. They are the largest declared approximation in structural terms, and they strand entire components rather than single call sites. The model needs only a second kind of node together with edges that open and dismiss it. The route-derived definition of the screen set would have to be widened, which is why this is not a small change to the adapter.

A runtime tier. The runtime tier is the complement to the whole static approach and not an extension of it. The model already admits observation identifiers as witnesses beside source locations. Setting that tier against the dynamic lineage, where the state flow graph comes from firing events on a running application [51, 52, 75],

would answer the question this thesis cannot: how much of what a crawler finds the static graph already contains, and how much of what the static graph contains a crawler never reaches. The Android work combines static analysis with runtime traces in the same way [46, 40].

For applications used signed in, such a combination is also the way to complete the graph. The static side supplies what a crawler lacks: every declared screen, the may-tier candidates with a witness each, and the guards they wait on, so it can tell an explorer which screens remain unvisited and which state each needs. A signed-in run against a seeded backend supplies what the static reader lacks: which may edges occur, which addresses computed routes mount, and which links a guarded screen renders. An observed transition would gain an observation witness beside its source witness. That is evidence that the may edge exists, not a promotion to must, since a must edge asserts the transition on every execution the claim ranges over and one run shows it on one. A candidate never observed stays may. The combination can therefore add evidence without weakening Claim 3.1. This thesis neither builds nor measures it.

A larger and broader corpus. An automated search over a larger pool of repositories, with the published quality filters applied mechanically [35, 36], would weaken the tool-driven selection bias of Section 6.2, provided that applications the extractor handles badly are kept and not rejected. A case study on a proprietary industrial application, reporting only aggregate figures, is the most direct answer to the external validity threat. A minimum density of user interface tests should become a selection criterion whenever execution-based ground truth is needed (criterion (4) of Section 4.1 asked only for test configuration), since otherwise a study ends up with its oracle and its scale on different applications.

React and Vue. A React or Vue adapter, evaluated with the same harness on a corpus of the same shape, would turn Section 6.4 from an argument into a result. The predictions of Section 6.4 are specific enough to be falsified.

A second line of evidence for the candidate class. The defect class "node not linked to its component" has one confirmation, from the comparison between the independent route parser and the graph, and the rule adopted in Chapter 5 admits a class only with two. The class is empty on the final version (Table 5.4), so its standing matters only for the record of the development versions (Appendix A.1). A second method would have to reach it independently, and the test-source cross-check is the natural candidate: a screen whose node exists but carries no component link should still be visited by tests while contributing no outbound edges. Until then the class stays a candidate, and its share of the starting corpus total stays provisional.

The use that motivated the thesis. That use is still unmeasured. Mobile testing agents pass the current screen to a model and replay the action it returns [47, 73], with application knowledge obtained by dynamic exploration [74, 72]; coding agents navigate a repository through an interface whose design measurably changes

their performance [32, 77, 82]. Whether a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge, improves either kind of agent, or improves automated test generation, is a controlled experiment that this thesis does not attempt. The graph and the harness are the artefacts such an experiment would need, and the defects quantified in Chapter 5 are what it would have to control for.

6.6 Summary

The findings of Chapter 5 are conditional: node extraction is complete only relative to what a route table can be read to declare, the loss ranking depends on the order of the loss rules, and change detection holds only for constructs the graph represents. The limits and the declared approximations say which queries the graph can and cannot answer. The transfer argument is a hypothesis, not a measurement. The most valuable next steps are an oracle for the screen set on a large application and a runtime tier that completes the graph for signed-in use.

Chapter 7

Conclusion

This thesis measured how much of the navigation structure of an Angular application can be recovered from its source code without running it. It explained where the recovery fails, and it proposed an analysis for the largest loss. This chapter summarises the answers to the five research questions, the contributions, and what remains open. The development versions of uigraph and the corrections between them are listed in Appendix A.1.

RQ1: how much is recovered. Loss happens at two levels. At the level of nodes, every screen the static readers find declared has a node in all 6 applications. Against the routes the framework mounts at run time, 2 of the six are complete, and most of the missing addresses lie behind one construct the extractor declares unread. At the level of transitions, almost no navigation construct becomes a deterministic transition: 25 constructs yield 150 must edges, and 125 of those edges come from one application's toolbar. Most of the tier rests on an assumption about route order the extractor does not read (126 edges; 24 without it), the toolbar's scope is untested, and the witness check cannot read the destination of 62 edges.

RQ2: where extraction fails. The failures fall into five defect classes and one candidate class (Table 5.3). Every lost construct is assigned to the first loss category whose rule matches. Under the order in which the rules are applied, attribution is the largest loss category in every application. The ranking depends on that order: if the target were tested before the owner, the target category would be the larger one on alfresco and k8s dashboard (Section 5.2).

RQ3: what scope attribution adds. Scope attribution recovers 292 of the 454 constructs in its loss categories (64.3 %), almost all as possible rather than certain transitions. Most of them are on mifos: without it the share is 37.3 %, and 4 of the six applications recover under half. No construct the baseline recovered was lost. On the one application whose screen sets were read by hand, the analysis finds 100.0 % of the reference pairs and none outside them; that reference was partly completed from the graph's output and guided two repairs. A reimplementation of the mechanisms of earlier work reaches 99.5 %, so the gain is small. The computed scopes are also wide. RQ3 is therefore answered only in part: the recovered share rises, but whether the

added scopes are too wide was tested on one application only. The applications' own end-to-end suites test the missing direction on two applications. The graph holds every observed transition on realworld, where the check does not discriminate, and 12 of 22 on mifos, against 10 without attribution. Of the mifos misses, 7 are unrepaired defects of the tool and 3 fall in the preregistered redirect class (Section 5.3.5).

RQ4: whether the graph responds to change. On three applications, the graph detects 45 of 45 injected link deletions, retargets and guard removals. Every operator is scored only at constructs the graph already represents, in the simplest form the extractor reads, so detection is close to guaranteed for each. The mutation harness was corrected during development after its misses had been read, and the final sample shares 46 of its 73 mutants with the run whose misses were read, so the rate is not an independent test; the samples are small. Without attribution fewer changes can be scored at all. On the 51 mutants both graphs can score, the attributed graph detects 51 and the baseline 50. The experiment shows that the graph responds to change; it does not show how well the graph would select tests.

RQ5: whose the losses are. The defect classes name failures of extraction, so the answer lies in their causes, traced in the source. Almost every cause is a construct a static reader can read, so the information was in the source and the loss was the tool's and not the application's. The final version reads most of them. This is a statement about classes, not about every lost construct: destinations only the run time knows are counted apart (Table 5.4). The causes still not read are named router outlets, which the model represents as a second mount and the extractor does not implement, and one application's spread of computed routes, which the extractor declares unread. The end-to-end check adds target forms the extractor does not read (Section 5.3.5). For the attribution class the answer holds for the part the analysis recovers; the unrecovered remainder (Table 5.9) mixes gaps of the tool with components that no screen may render, and no measurement separates them. Two further causes, which no defect class names, lie outside the reach of any static reader of a route table: a route path computed in a custom `UrlMatcher`, and a choice between two route tables made at deployment. Every site that still yields no transition carries a recorded label, and 2 of them say that no reason was found. A check enforces that a label is recorded, but not that it is correct.

Contributions. The thesis contributes four things. The first is the extractor, uigraph, built in this work. It accepts nested and parameterised route trees, which the stated restrictions of an earlier Angular extraction exclude: they admit 0 of the 6 corpus applications (Section 5.3.6), and uigraph extracts all of them. The second is the measurement itself: a corpus of six pinned applications, five evaluation methods with different blind spots, four further checks of the scopes, and a classification of every lost construct. The third is the apparatus that makes it checkable: the result tables are generated from the harness's result files, and the methods that read the sources re-run from pinned commits. The runtime route oracle re-runs only in part, the realworld end-to-end check depends on an unpinned public backend, and

superseded figures cannot be regenerated. The fourth is scope attribution, specified as a dataflow analysis over screen sets and implemented over three of its four input relations (Section 3.2).

What remains open. The dispatch relation is specified but not implemented, the guard relation is replaced by a stand-in, and the call graph ignores the injector hierarchy. The screen sets were checked in both directions on one application only, against a reference that applies the analysis's own relations, and uigraph was developed on the same corpus it is measured on. The end-to-end check on mifos found transitions lost to a declared approximation, a relative link in a template shared with child tabs resolved against the child's address, and to target forms the extractor does not read; none is repaired (Section 5.3.5). The readings of the must tier were made by the AI assistant, and two readings by one model measure consistency rather than correctness. The rendered-interface check judged only the screens that are reachable without signing in; driving the applications into a signed-in state is the work that would test the rest (Section 6.5). Finally, whether such a graph helps the consumers that motivated the work, such as agents that drive a user interface or automated test generation, is not measured here.

What the measurements establish is that a declared route table is a usable starting point for static navigation extraction, but not a sufficient one.

Appendix A

Validating the measurement

This appendix holds the records of how the measuring instruments were checked and corrected. Nothing in it is a result of the analysis. It is the record that the figures of Chapter 5 come from instruments whose own failures were looked for and found. Section A.1 lists every correction in one table. The sections after it give the detail behind the corrections that need more than a table row: the disagreement between the two route readers, the movement of the denominators, the reasons for the remaining losses, the dead-link corrections, the runtime route oracle, and the must tier.

A.1 History of the corrections

uigraph was developed within this thesis, and three of its versions are named in this appendix, each by one label.

- The *early development version*: commit f17a302, before the corrections the evaluation prompted. On it k8s dashboard had nodes and no edges (Table 5.3).
- The *intermediate version*: extractor eb9dab80, with its 20-edge must tier and the tables as printed at 7d109c7.
- The *final version*: extractor commit `c42f220`, harness 1.9.0 and mutation harness 1.9.1. The source of the Angular adapter is identical at `920e1aa`, where the sweep of the demotion rules was recorded.

The chapters present only the final version, whose tables of Section 5.5 were regenerated at its commit. Of the intermediate version only the must-tier figures are cited (Table A.1), as printed at 7d109c7; the runs behind the printed tables were lost, and an archived re-run of the same extractor gave a different loss ledger. The figures of the early development version and of the intermediate must tier are historical: they were measured with `eb9dab80` and before and are read from git at `f17a302` and `7d109c7`. The first node audit (Table A.2) is historical too: harness 1.2.0, recorded extractor `4505fd2a`, absent from the repository, committed table only.

The results of Chapter 5 are those of the corrected instrument. Table A.1 records how the instrument got there: what was wrong, how it was found, what was

changed, and what the change did to the reported figures. The figures of superseded instruments cannot be regenerated with the harness this thesis ships. Each is either read from a named commit's own generated results, or taken from the output of an earlier run that was not kept. Where an earlier figure cannot be regenerated, the row describes the effect in words. The rows follow the order in which the chapters record the corrections; they are not a dated log. The reviews and red-team panels named in the table were made with the AI assistant that the declaration at the front of this thesis describes, and every defect they reported was reproduced against the code before it was repaired.

Table A.1: The corrections made to the instruments, in the order the chapters record them. *Found by* gives the method as the main text names it.

No.	What was wrong	Found by	What changed	Effect on the figures
Protocol and harness (Section 4.3.2)				
1	Test-source cross-check: a URL parameter named in a test could match a literal route segment, which gave matches that were not real screens.	Seen as the recall figure of one application falling when the rule was tightened.	A parameter no longer matches a literal segment. A negative test, which names a URL that must not exist, is excluded.	The earlier recall figure cannot be regenerated. The end-to-end recall of that application is now 71.4 %.
2	The first witness check reported a low precision for one application.	Inspection of the flagged edges. All were may edges of confidence one half, produced by one over-approximated call site connected to every declared route.	No extractor change is recorded. Over-approximated sites are reported apart, since the check claims no target agreement for them.	The low figure described the measurement, not the extractor. The must tier of the same run had no failing edge.
3	Mutation testing: a mutant whose target is absent from the baseline graph repeats a recovery gap already counted elsewhere.	Not stated beyond the misses of one application.	Such mutants moved to a separate category.	Most of that application's misses left the detection rate.
4	The first construct denominator counted the substring `routerLink`, which also matches `routerLinkActive` and its options.	A construct count that tripled, a figure that looked implausible.	Constructs are counted by the independent route parser that also produces the loss breakdown, so ratio and breakdown share one definition.	For alfresco the independent route parser finds 34. The substring count was about three times larger.

No.	What was wrong	Found by	What changed	Effect on the figures
5	The first independent route parser did not look inside wrapper calls, did not follow a lazily loaded module to the routing module it imports, and walked lazy tables without their prefix.	Coverage that reached 100 per cent: every declared screen of mifos had a node.	The independent route parser reads wrapper calls, follows the imported routing module, and keeps the prefix.	mifos has 610 declared screens, of which 58 have a node. The extractor was the same in both runs.
6	Must conversion was first defined as must edges divided by constructs.	A ratio that rose implausibly after attribution (mifos).	The ratio is computed per construct, and edges whose screen set is a whole subtree are stored once (Section 3.2).	The rise came from how edges were stored, one per screen, and not from evidence. The ratio is defined in Section 4.2.
7	The mutation harness patched the first textual match of a construct, judged detection on node and edge sets alone, and retargeted only the head of a commands array.	A review of the harness, and a detection rate that moved by more than twenty points.	Patches are made at the construct's offset. Detection is judged on the witness set as well. A retargeting mutant points at another declared route, a guard mutant on a screen no edge enters is excluded, and a rename counts when the renamed route appears.	The detection rate for realworld rose sharply. The corrected rule was applied with the same seed but not the same sample: 70 mutants in the earlier run, 73 in the final one, 46 at the same location (operator, file and line; destinations may differ). Of the 9 earlier misses, 6 are not in the final sample. The earlier run showed six paired disagreements in favour of attribution, five of them retarget mutants whose operator replaced only the head of the commands array; with the operator corrected the baseline detects those five too.
8	The guard exclusion rule found a route by the nearest path literal before `canActivate`, which fails for routes with children, empty paths, constants and template literals.	Reading the graph at the sites the rule had let through.	The route is found by matching the braces of the enclosing object over source with comments and strings blanked. A route yielding no screen is excluded.	Four guard mutants that cannot be observed had been scored as misses, and the discussion had explained them as a defect in how the graph represents a guard. They are now excluded. The superseded run sampled 70 mutants.

No.	What was wrong	Found by	What changed	Effect on the figures
9	Removed guards were judged wrongly in two ways: an edge between two children of a guarded parent no longer carried the parent's guard, and a comparison of sets read one of three identical guard texts as unchanged.	A change in the extractor (it gated an edge only on the guards its navigation runs), not a figure.	A removal is detected when some edge loses a guard, and excluded when no baseline edge carries it. Guards are counted, not collected.	Which mutants are sampled is unchanged. How a sampled mutant is scored changes.
10	The construct denominator took a construct from any line its pattern matched, including a commented-out link and a sentence of a code comment.	A table script stopped: the loss classifier found no reason for a commented-out link in mifos.	Comments are blanked before parsing, with the compiler's own scanner for TypeScript.	One construct leaves the denominator in each of bitwarden and mifos, and nothing else changes.
The extractor and the deterministic tier (Section 5.5)				
11	A guard on a route that a wrapper function returns was not read, so everything below it was published as unconditional.	Red-team panels reading the graph and the source after commit f17a302. The witness check did not find it.	The guard set of a screen also reads routes a function returns.	At f17a302 the tier held 1024 must edges with 26 witness failures, read from that commit's own results. The failures were target mismatches on edges the unread guard left on the deterministic tier. Reading the guard cut mifos's tier to 1 on the intermediate version.
12	A `canActivate` element that is a call, such as a guard factory, was skipped, so a route guarded only by one read as unguarded.	Red-team panels, as in row 11.	An element the reader cannot name gates the edge anyway.	bitwarden's deterministic edges into its root no longer read as unconditional.
13	The witness check opened only the first of several constructs that yield one edge.	Correcting the check itself, so that it judged every cited site. This found edges in bitwarden whose later citations targeted `/vault`, which the extractor had not read.	The check opens every cited site. An edge took the weakest modality over all constructs citing it, a rule replaced in row 18.	Later citations are 27.9 % of cited sites, and 58.7 % in the largest application. The intermediate tier then held 20 must edges, with 20/20 passing and 0 failures.

No.	What was wrong	Found by	What changed	Effect on the figures
14	The demotion rules were incomplete. A link under `*ngIf`, inside `@if`, in a callback, in a `catch` clause, or after `await` or an early return was emitted as must.	A review of every construct cited by the intermediate tier's 20 edges (30 distinct constructs, cited 36 times), then red-team panels. The witness check cannot find this defect: it compares destinations and does not ask whether a construct runs every time.	A site is demoted under every condition of Section 3.1. The panels extended the rules to members no handler calls unconditionally, components mounted under a template condition, ICU cases and addresses two routes can answer; a later review extended them to code after a navigation that may start another.	32 of the 36 citations ran under such a condition. Table A.4 records the condition read at each construct.
15	A guard on an empty-path child without a component, which Angular matches on every navigation to the parent, was not read. `canMatch`, `canActivateChild` and `canLoad` were not read at all.	The review of row 14.	Such a guard gates the screen above it, and the other three guard kinds are read.	10 of the intermediate tier's edges, all of bitwarden's, are guarded.
16	The repair of row 15 gated an edge on every guard of its destination, which demotes navigations the router lets through untouched.	A red-team panel.	The extractor computes, for each source and destination, the guards the router runs: levels left, matched, entered and kept.	Navigations between screens that share a guarded parent are let through, and mifos holds 15 must edges again.
17	A navigation to the address its source already displays was treated as one that occurs on every execution.	The review of row 14.	A shell or anchor edge whose destination is one of its screens is drawn per screen: must from the others and a may self-loop at the destination.	1 of the intermediate tier's edges is such a self-loop.
18	The weakest-modality rule of row 13 discarded a certainty the edge had. realworld's Global Feed link runs whenever its page does, yet two other links to the same root do not.	Reading the edges that survived.	An edge takes the modality of its most certain construct, which becomes its witness. Every other construct records the modality it earns itself, and the witness check judges the must tier only on constructs that earn must.	The final tier is 150 edges resting on 25 constructs. It is a different set from the intermediate tier, not that tier made smaller. Superseded tables counted every construct a must edge cites as converted; the current ones count a construct only where it earns must itself.

No.	What was wrong	Found by	What changed	Effect on the figures
The independent route parser and the denominators (Sections A.2 and A.3)				
19	The independent route parser did not read a path written through a constant, an enum or `$localize`, a first-party import through `baseUrl`, or a route array named rather than written.	Investigating the disagreements of the first node audit (Section A.2). Each gap had scored the extractor against paths the independent route parser itself had not read.	The independent route parser reads all three forms.	Before the last of them, the independent route parser saw almost none of kubernetes/dashboard's declared screens and scored nearly every node the extractor minted there as unmatched.
20	The independent route parser read Storybook fixtures as declared screens, read a sibling application's tables reached through a tsconfig alias as this application's, and recorded a path in an `Object.freeze` table as unresolvable.	The dead-link corrections (Section A.5).	The independent route parser sets Storybook files aside, ignores the sibling's tables, and reads the frozen table. Dropping the frozen routes from the denominator as unreadable was considered and rejected, since they were readable one call away.	The declared sets of 5 of the 6 applications moved, and in 1 of them the count stayed the same (Section A.3).
21	A file produced by an interim revision of the independent route parser, dropped into the directory the tables are generated from, was read as the earlier independent route parser's output. The script that produces the tables could not detect this, and an absent file was read as a denominator that did not move.	Fixing the provenance of the comparison.	The earlier independent route parser's own committed output is read from the repository.	The bitwarden decomposition built on that file is withdrawn; Section A.3 gives the corrected one.
22	The first-audit extractor reached route tables without their parent prefix, and minted a node at ∗ for each catch-all.	Resolving the disagreement of the first node audit (Section A.2).	The repairs of the loss list: a table not reached, a path named and not written, a destination named and not written (Section 5.2).	Of 143 nodes with no declared route, 138 were the tail of a declared route and 5 were catch-alls. On the final version 2 nodes match no declared screen, 2 of them at ∗.

No.	What was wrong	Found by	What changed	Effect on the figures
23	The grounds that exclude a construct from the denominator were tested after the graph had been consulted.	Not stated.	The grounds are decided from the construct's own text before the graph is consulted. The comparison refuses to print if a construct changes its ground between the two runs.	The after column had been a ratio over more sites than the before column. Both are now over the same 1775 sites.
Findings of the runtime route oracle (Section A.6)				
24	A route that declares a named outlet could take the primary address from a sibling. The routes beneath a named-outlet route were also minted at the primary address.	The runtime route oracle returned two alfresco addresses that the framework answers with its catch-all. Both static readers had read the subtree alike, so coverage of 100 per cent could not expose them.	Outlet context is carried down the walk, and no screen is minted for a subtree beneath a named outlet.	On the compared graph the extractor declares 0 screens at addresses the framework mounts nothing at. What remains is a limit on recall (Section A.7).
25	A lazy route that names a module through a barrel, which re-exports a feature module that imports the routing module, was followed one hop only. The table was then scanned at the root.	The runtime route oracle, compared with the route set as the static readers held it before. Both readers had arrived at the defect independently.	Both follow the chain to the table, bounded, and refuse at each hop a file the route sits in, a file another route loads, and a file whose tables are registered at the root.	10 addresses in alfresco, bitwarden were wrong, the two alfresco ones of row 24 among them. Node coverage had been 100 per cent against a denominator carrying the same wrong addresses.
26	The test that keeps a lazy route from adopting the application's root table read `RouterModule.forRoot`, so it fired nowhere in the corpus.	Instrumenting the predicate, at commit `d40c8dc`.	Both tools mark root tables per array, taken from the argument of either registration API.	A guard that never fires had been credited with a property nothing showed it to have. The counts are in Section A.7.
The dead-link census (Section A.5)				
27	An earlier correction followed only the redirecting kind of catch-all route.	Attempts to refute the dead-link finding.	The extractor follows both kinds, holds the edge at may, and records where the catch-all sent the URL.	The correction had held for the application it was written against and failed for the two that needed the other kind.

No.	What was wrong	Found by	What changed	Effect on the figures
28	Resolving a target and calling a link dead were conflated. bitwarden's authentication paths went through an unread enum, so the wildcard forwarded every one of them.	Attempts to refute the finding.	The enum and the wrapper call are read. Each application records a note when its route set is a lower bound. Each caveated candidate is put to the framework's own route configuration.	The wildcard had raised that application's recovered figure. Candidates withdrawn: 24 because the framework serves a screen, 19 because the matching route places none, 0 because a redirect answers. 0 keep the caveat.
29	Routes the tools could not read were taken as routes the application never declares: a route array built by a spread, ghostfolio's `matcher` property, and alfresco's tsconfig `paths` alias.	Attempts to refute the finding.	Both tools record the holes they cannot read, and no claim that a path is undeclared is counted inside one.	ghostfolio's `/admin/users` had been reported as a confirmed dead link, and two live links of alfresco as dead. All three constructs are read now.
30	The route-set test read only the first quoted segment of a commands array and so tested a prefix as if it were the target.	Attempts to refute the finding.	The whole array is read as a pattern, computed segments as holes, and matched against the declared routes pattern to pattern.	It cleared alfresco's two candidates and found further sites in mifos. The census now holds 27 candidate sites.
Mutation sweep and end-to-end check				
31	On its first run in the final revision, the sweep of the demotion rules found rules the router fixes had added without a test.	The mutation sweep, at commit c8eec9b.	Tests were written for most survivors. The others were retired as equivalent, since no input can tell them from their absence.	20 of 164 mutants survived; 18 now have tests and 2 were retired. At 920e1aa every one of 173 deletions fails some test. Writing the tests exposed a relative link that climbs above the root.
32	The end-to-end scorer compared a redirected navigation on the landed address, against the preregistration.	A review panel (4 Nov), checking the scorer against the preregistration; fixed at commit `0172867`.	The scorer applies the preregistered rule for redirected navigations.	mifos now scores 12 of 22 pairs; 5 pairs were redirected, and 2 of them hit. The earlier figure, scored on the landed target, was 15 of 22.

Five of the protocol corrections (rows 2, 4, 5, 6 and 7) were found by a figure that looked implausibly good or bad. Each turned out to be a property of the measurement and not of the extractor. A surprising figure is therefore checked against its inputs before it is written down.

Table A.2: Why each unmatched node has no declared screen, as the first node audit measured it (Table 4.5). A node is placed in the first row that applies, and the categories are exclusive.

Why the node has no declared screen	rw	alf	gf	k8s	bw	mif	Total
No route declared at that path	0	1	1	1	23	117	143
Parser could not follow the route	0	3	0	37	2	1	43
Route is a prefix or a redirect, so outside S	1	2	0	0	6	2	11
Declared screen, so a join artefact	0	0	0	0	0	0	0
Unmatched nodes	1	6	1	38	31	120	197

A.2 Nodes the two static readers disagreed about

Table 5.2 also reports the other direction of the differential comparison: the nodes the extractor emits for which the independent route parser holds no declared screen. This section records that direction as the first node audit measured it, with both tools as they stood before the repairs of Table A.1. Section 5.5 repeats the comparison with both tools corrected, and the figures change considerably.

In the first audit the independent route parser held no declared screen for 197 of the 338 nodes in the corpus. Node extraction was therefore inaccurate as well as incomplete. For k8s dashboard the figure is almost entirely the independent route parser's own limit: 37 of its 38 unmatched nodes are lazily loaded entries the independent route parser cannot follow. Mifos is not explained that way, since it has 1 blind spot and 120 unmatched nodes. Table A.2 decomposes all 197 by the kind of route the independent route parser found at the node's path, joined on the independent route parser's own normalisation.

Three groups come out of it.

- For 43 nodes the independent route parser could not follow the route that would have declared them. These are the independent route parser's limit and not the extractor's, and 37 of them are k8s dashboard.
- For 11 nodes the independent route parser found a route, but one it classifies as a path prefix or a redirect. Section 3.1 admits only component-bearing routes into the screen set, so the node set holds elements the model excludes: 1 in realworld, 2 in alfresco, 6 in bitwarden and 2 in mifos. An example is `/profile` in realworld, whose route declares `loadChildren` and no component. These nodes make $screen(r)$ answerable for a route the model treats as a path prefix only.
- The remaining 143 have no declared route of any kind at the node's path. They are concentrated in mifos (117, against 1 parser blind spot) and in bitwarden (23).

The third group is the substantial one. Either the extractor emits nodes for paths it assembles differently from the independent route parser, or the independent route

parser misses route tables without reporting them as blind spots. The two methods could not adjudicate, and this was the largest unexplained behaviour the first audit found.

The later repairs resolved it, and the first explanation held. 138 of the 143 sit at the tail of a route the independent route parser had already declared in full. The extractor had reached the table without its parent prefix and minted `/premium` where the independent route parser declared `/settings/subscription/premium`. The other 5 are catch-all routes. The first-audit extractor minted a node at the path `*` for the catch-all of each application concerned, whether the catch-all renders a component (alfresco, mifos) or redirects (ghostfolio, k8s dashboard, bitwarden). The independent route parser's policy keeps it out of the screen set, so there the two tools disagree by convention and not over a table either missed.

The direction matters because it decides whether the measuring instrument or the tool was wrong. The repairs of Table A.1 are accordingly almost all in the extractor. The independent route parser had the smaller share: a route array behind a module it refused to follow, a path assembled from a constant it did not resolve, and a lazily loaded table walked without its parent. Reporting the disagreement rather than attributing it was the right decision at the time, and investigating it produced most of the repairs.

On the instrument of Chapter 5, 2 nodes remain that no declared screen matches, and 2 of them sit at `*`. The independent route parser declares a screen for every other node the extractor mints, and the runtime route oracle confirms that the framework mounts a component at every address the extractor claims. This section therefore records a past state and not a standing defect: the state the differential test was in when it was the only check available, and the reason a third, independent reader was added.

A.3 How the declared-screen set moved

Table 5.2 gives node coverage. 6 of the 6 applications now hold a node for every declared screen. The denominator of those ratios needs separate treatment, because the declared-screen *count* is a poor summary of the declared-screen *set*. Between the independent route parser before the repairs and the independent route parser reported here, the declared sets of 5 applications changed, and in 1 of them the count stood still. A moving denominator can distort a coverage figure unseen, and one that keeps its size can do so too. Each change is accounted for below from the generated subtraction.

ghostfolio. The count did not move: 84 declared screens before and 84 after. Every address changed. 84 paths left the set and 84 arrived. The earlier independent route parser recorded the verbatim expression of a route's path, `/internalRoutes.account.path`, where the application mounts `/about` and its siblings. A comparison of counts would have shown no change.

k8s dashboard. This is the largest move: from 2 declared screens to 67, with 65 arriving and none leaving. The earlier independent route parser reported 37 blind spots for this application, each a `loadChildren` target it could not resolve. It therefore declared only the two screens of its two eagerly registered tables (`/login` and the application root) and nothing beneath a lazy module. The independent route parser reported here declares the whole tree with no blind spot. The earlier denominator was near-vacuous, so coverage against it was 100 % of almost nothing, which is a fact about the instrument and not about the application.

bitwarden. The count went from 97 declared screens to 98.

- 3 were Storybook fixtures, throwaway tables such as `/vault` and `/sm/:organizationId` that exist only to render one component in isolation. Neither tool reads them now.
- 40 were inside the application. 26 of those left for ghostfolio's reason: they were recorded under a verbatim expression, such as `/AuthRoute.Login`, because the enum behind it was unreadable.
- The rest are the mifos defect in miniature. The earlier run read their path segments correctly but placed them at the project root, as `/password` and `/session-timeout`, because it could not follow the lazy table that carries them. They are now declared under `/settings/security/` and `/vault/`.
- 44 arrived from two causes. A route table wrapped in `Object.freeze` is now read by both tools. The report and member subtrees were declared at the application root until Section A.6 showed the running router mounting them elsewhere.

The moved screens are the same screens at a different address, which a diff of the set shows and a count does not. Against the 98 screens that remain, the graph holds a node for every one.

The comparison is made against the earlier independent route parser's own committed output, read from the repository and not from the directory a run writes into. An earlier comparison read a file from an interim revision of the independent route parser as though it were the earlier independent route parser's output. The script that produces the tables could not detect this, because every revision's output is internally consistent, and an absent file was silently read as a denominator that did not move (Table A.1, row 21).

alfresco and mifos. The remaining two moves are small and are printed for the same reason. alfresco went from 25 declared screens to 26: 0 left and 1 arrived, because of the workspace-alias repair. mifos went from 610 to 605, with 8 paths leaving and 3 arriving. The routes of its reporting module, which the earlier run placed at the project root because it would not follow a `baseUrl` import, now sit at `/reports` and beneath it. The net change is too small to move any percentage in Chapter 5. The only defence that generalises is to generate every part of the subtraction so that it can be checked.

Witnesses outside the application directory. One caveat of Chapter 4 can now be quantified. The independent route parser and the extractor agree about an edge only where both read the same source. 636 edges across 3 applications (527 of them in bitwarden, 0 of those on the deterministic tier) have their only witness in a file outside the application directory. The check records that the witness lies outside, not how the file came to be read, so the figure is reported without an explanation. The files are of the kind a monorepo puts beside an application: a shared library and, in bitwarden, the repository's other front ends. Both tools read them, so their agreeing there is not independent evidence. The witness check reports the figure instead of excluding the edges, because a precision figure is a claim about the edges the tool emitted, wherever it read them. A reader who wants the figure over the measured directory alone can subtract the count. The edges are not kept on the ground that they are clicks in the application measured: the files in bitwarden belong to a sibling front end built from the same repository, and whether its components are reached in the application under measurement is not established here. The dead-link census of Section 5.6 decides the other way. Calling a link dead asserts something about an application, so the link must be reached in the application being measured, and there the boundary decides.

A coverage ratio cannot see a screen nobody can address. Some of bitwarden's declared screens were for a time outside the denominator, because their paths are assembled in a template literal from an exported constant that neither tool resolved. Coverage was then 100 %, and the figure gave no sign of the error. The only defence is to report what was set aside, which is why the blind-spots column is printed beside every coverage figure in Chapter 5. For the same reason, that 6 applications reach every declared screen is a weaker result than it sounds: the blind-spots column counts the routes a tool knows are there and cannot address, so reaching every screen one can see shows that the walk finds the route tables, not that the graph is complete.

A.4 Every remaining site names a reason

21 sites are neither recovered nor accounted as dynamic or unplaced, and Table A.3 gives the label recorded for each. A script produces the reasons and exits non-zero if any site reaches its end without one, so a site the analysis cannot explain is a failing state and not a silent remainder. The script enforces that a label is recorded, not that it is correct. The check covers these sites only. The remaining attribution losses of Table 5.9 include 2 recorded as having a scope and a destination with no reason found, and that label is accepted rather than failed.

The largest group, 10 of the 21, reflects the attribution gap of Chapter 3. Each is a class the extractor parsed and found a navigation site in but could place under no screen, because no relation the implementation builds reaches it. Either the call graph of Section 3.2.3 resolves no call into it and no component it reads injects it, or it is an effect the stand-in for the unimplemented dispatch relation does not cover (Section 6.3). The reason given is the extractor's own note and not the harness's

Table A.3: Why each remaining site is not a transition. The script that produces this table fails if any site has no reason.

Sites	Reason the site is not a transition
10	A class the extractor could not place under any screen
6	Over-approximated at the site
4	The target names no route the application declares
1	A service; whether anything injects it is not established
21	All, none of them without a reason

inference. The distinction matters: saying that nothing in the application injects the class would be a claim about the application, whereas the correct claim is about the tool.

A second group concerns a target the extractor read and could not place: 4 sites whose target matched no route. They are split by what can be claimed of each.

- A relative target with no base to resolve against: 0.
- A target inside a subtree one of the tools could not read: 0.
- A target in an application whose route set is a lower bound: none.
- A target that nothing accounts for but the application: 4.

Each is decided by testing the target against the independent route parser's route set, pattern against pattern. The extractor's own note is not repeated, since it says that no node of the extractor's matched, which is a claim about the extractor.

A.5 The four dead-link corrections

Four corrections belong with the dead-link finding, and all four came from attempts to refute it (Table A.1, rows 27 to 30). Three are the same mistake made in three places: from a navigation site, a route neither tool can read looks like a route the application never declares.

Catch-all routes. A target that no route declares does not necessarily go nowhere. A `**` route catches every unmatched URL, either by forwarding it with `redirectTo` or by rendering a `component`. The extractor follows both and holds the edge at may. It records a note saying that the target matched nothing and where the catch-all sent it. An earlier correction followed only the redirecting kind. It held for the application it was written against and failed for the two that needed the other kind.

Resolving a target is not calling it dead. bitwarden declares its authentication paths through an enum the extractor had failed to read. Targets such as `/login`

matched nothing for a reason that was the extractor's, and the wildcard then forwarded every one of them. That raised the application's recovered figure. The enum is read now, and so is the wrapper call that holds it. What is left in bitwarden is a boundary and not a path, and the note says which of the two it is, because a dropped subtree and an unreadable path are different losses. Angular does forward those URLs, so the edge is drawn either way, and what had to change was the claim attached to it. The extractor counts the declared paths it could not read and records one note per application saying the route set is a lower bound.

A single unreadable declaration used to void every claim of this kind across the whole application. That was deliberate, because the alternative is a census that reports an extractor's blind spot as an application's defect. It also asks a harder question than the census needs. Whether the whole route set was read is one question; whether *this* target is served is another. The runtime route oracle of Section A.6 answers the second directly, because the configuration it reads is the one the framework holds. Each caveated candidate is therefore put to that configuration. 24 are withdrawn because the framework serves a screen at the target, which makes them a measurement of what the extractor failed to read. 19 are withdrawn because the target is matched by a route that places no screen of its own, and 0 because a redirect answers it. 0 sites keep the caveat. A matcher route leaves undecided only the addresses under its own prefix: ghostfolio's matcher sits at `/admin` and reaches nothing above it, so reading it as leaving the whole application undecided withheld findings on a ground the harness's own navigation refutes.

Holes counted as absence. A route array built by a spread (`children: [...adminRoutes]`) was not followed, so the routes it carries were minted at the project root and links to them matched nothing. ghostfolio mounts its user administration screen with Angular's `matcher` property and not `path`:

```
{ component:  GfAdminUsersComponent,
matcher:  usersMatcher, ...}
```

Neither tool read the `matcher` property, so `/admin/users` appeared in no route set. An earlier state of the census reported it as a dead link confirmed by hand, but the hand check had searched for `path:  'users'`, which is not how that route is spelled. The recogniser places a component at that address, so the sites naming it are withdrawn. alfresco loads the children of `nodes/:nodeId` through a tsconfig `paths` alias that neither tool resolved, so two live links to `/nodes/:nodeId/rules` were reported as dead. All three are read now. One rule generalises: both tools record the holes they cannot read (a matcher route, an unresolvable lazy boundary, a path that is not a literal), and no claim that a path is undeclared is counted inside one.

A prefix tested as the target. The route-set test read only the first quoted segment of a commands array, so `['/nodes', id, 'rules']` was tested as `/nodes`. The whole array is now read as a pattern, its computed segments as holes, and matched against the declared routes pattern to pattern.

What the candidates are. After the four corrections, the 167 sites that a catch-all route answers divide as follows.

- From an application whose route set is a lower bound, so that no measurement can decide the address: 0.
- A target the framework serves with a screen, which is a defect but the extractor's: 24.
- A target matched by a route that places no screen, which could never have had a node: 19.
- A target the framework answers with a redirect: 0.
- A route the application declares as a screen and the extractor minted no node for: none.
- A route the application declares as a *path prefix*: none.
- A subtree the independent route parser could not open: 0.
- A pattern with a hole where the first segment should be, which names no single address: 2.
- A relative target the analysis had no base to resolve: 18.
- The application root: 12.
- A record that names no target at all, so nothing is established about where it points: 0.
- A site outside the directory the measurement is scoped to (Section A.3): 69.
- Remaining: 23.

Of the 4 unexplained sites whose target named no route, 0 are a relative target with no base, 0 address an unreadable subtree, and none come from an application whose route set is a lower bound. 4 remain.

Those two residuals are the dead-link candidates: 27 sites at 25 places in the source, 23 distinct URLs, in 2 applications (bitwarden, mifos). The count is of sites and not of defects. One wrong prefix written into a shared template is one mistake and several sites. Every category above except the last of each list is a limit of one of the two tools and not a defect of an application. The residual is what is left once the tools no longer blame the applications for their own blind spots.

One of those categories used to be unsettled and is now measured. A route declared as a path prefix answers no URL of its own unless it also declares an empty-path child. Every one of the 19 sites excused on that ground has a target the framework matches with a route that places no screen of its own. A navigation to mifos's `/collections` activates a last route carrying no component, so the address

answers with the application's chrome and an empty outlet. The exclusion claimed this from the start, and it is now a measurement and not an inference.

The result is a candidate list and not a verdict. The four corrections show that the tools' blind spots are not yet all known, and a route that both tools fail to read looks, from outside, like a route that does not exist. Each candidate was checked by hand against the application's own route files, which is how the `appusers`, `manage-jobs` and `configurations` mismatches of Section 5.6 are quoted. No candidate was checked by navigating to it in a running application, which is the only check that would settle one.

Half of that objection has been answered, and not by the route files. Every candidate's target is put to the route configuration the framework holds, read from the running application by the check of Section A.6. The premise that the application mounts nothing at that address therefore no longer rests on two readings of the same files. The classification used for that premise was itself held against what the framework does, on 41 navigated addresses, and agrees on 41. That keeps the census from being empty: both applications it draws from declare part of their route set unread, which by the rule above would suppress every candidate they carry. Still unchecked is the other premise, that the link is reached at run time at all. A target is also matched against the framework's patterns and not navigated to one address at a time, so a candidate's own URL has not been through the recogniser. What the method contributes is the list: a static transition graph puts every such candidate on one page, with a file and a line each, instead of leaving them to be met at run time.

A.6 Asking the running framework

Every result so far rests on two readings of the same route files. The witness check tests a must edge's destination against the independent route parser's route set, and Section 4.3 states the condition under which that test is vacuous: where both tools read a route table at the wrong prefix, they agree on a wrong address and the check passes. Agreement between two readers of one text is evidence only to the extent that the readers can differ. This section reports a third reading, which does not read the text at all.

A.6.1 How the framework was asked

Each application was built in its development configuration and loaded in a headless browser. Once it had bootstrapped, the live `Router` was found through Angular's debug surface and its route configuration read directly. Every lazily loaded subtree was expanded by calling the application's own `loadChildren` and taking the routes out of whatever it resolved to. Nothing in the procedure parses a route file. The answer is the configuration the framework holds, produced by the application's own code, so a disagreement with both static readers cannot be an artefact of a reading they share.

No server was run. The method needs the application to boot, not to work. An

address is counted as mounted when the router's configuration places a component there on the primary outlet, whether or not a guard would admit a visitor or a resolver would find data. All 6 applications bootstrapped and 1759 route entries were read. 0 lazily loaded subtrees were left unexpanded, 0 applications raised an uncaught error, and 2 logged errors to the console, mostly failed requests to the server that is not there. This does not bear on the route configuration, which is assembled before any of those requests is made. 1 route in the corpus decides its address through a `UrlMatcher`. That route and anything beneath it have no address any of the three readers can resolve to a string, so they are outside both sides of the comparison. Table 5.19 reports the result.

Reading a configuration is itself a reading, and the one this comparison depends on is which entries answer an address with a screen. That reading is checked against what the framework does. The harness navigates a running application to an address and records the recogniser's verdict, the event Angular emits when a URL has matched. That event fires before any `canActivate` guard runs, so a guard that sends every visitor to a sign-in page does not hide what was matched. The `canLoad` and `canMatch` guards run during matching, and one that refuses removes its route from the match, so the verdict is the match those guards allow.

Over 6 applications and 42 addresses, chosen to include every address this chapter names, the classification taken from the configuration agrees with the navigation on 41 of the 41 it decides. It declines to decide 1, the address the `UrlMatcher` route claims. The comparison of Table 5.19 decides for itself which entries answer an address, so the same navigations are held against its own list on the two implications that bind it: an address the framework serves with a screen it must hold, and an address answered only by the catch-all it must not. That agrees on 31 of 31. The addresses are those the text reasons about and not a sample of the corpus. The 251 screens the extractor does not mint are not individually checked, and are not claimed to be. The comparison turns on two answers.

- An address whose route carries children places no screen of its own. A navigation to mifos's `/collections` activates a last route with no component at all, so the address answers with the application's chrome and an empty outlet.
- An application with a catch-all answers every address, so the configuration mentioning an address is no evidence that anything is mounted there. In mifos a URL that matches nothing reaches a not-found component, and in bitwarden and kubernetes/dashboard it is redirected to a screen.

A.6.2 What the framework mounts and the extractor does not

The two directions are not symmetric. A screen the extractor does not mint costs recall, since the graph says less than it could. An address it mints that nothing mounts would be a false claim about an application. The corpus holds 251 of the first and 0 of the second.

Of the 251, 242 are one construct in ghostfolio. Its product comparison pages are not written down: a list of competitor records is mapped to routes and spread into the table, and each path is a template string over a field of the record. The extractor mints the parent and declares the spread a route source it could not read. The graph is not missing those screens by accident. The pages behind the spread are generated entries that differ only in a path segment and a data key. It does show that the cost of a declared blind spot can be far larger than the number of blind spots suggests, which a count of notes cannot show.

5 are in mifos, and they are one construct as well. Its users module holds two route tables, one for each identity provider the deployment may use, and chooses between them with `!environment.OIDC.oidcServerEnabled`. That constant is computed from `window.env`, injected into the page at deployment time, so no reader of the code can say which table the application mounts, and the extractor declares the subtree unread. The two tables happen to declare the same five addresses with different components. The extractor used to name those addresses by walking a table the module does not register under the prefix, which was an accident of this application. The addresses are lost here and the loss is declared, which is the trade-off this analysis is designed to make.

3 are in alfresco, and no two of them share a cause. Two are outside the directory the measurement scopes to. `/login` is declared by the shell application that composes the measured library. `/view/authentication-confirmation` is declared by an installed package that the shell provides, and the extractor does not follow a dependency into a package directory, because a package declares no screen of the application under measurement. The third, `/about`, is inside the directory but is not declared in TypeScript at all. The library ships `about.plugin.json`, whose single entry is

```
{ "id":  "app.about", "path":  "about", "layout":  "app.layout.main",
  "component":  "app.about.component" }
```

and an extension registry binds the identifier `app.about.component` to a class at run time. The route exists in data, and no reader of the application's code can see it. The runtime route oracle sees it because the framework assembles the table from the manifest before the runtime route oracle reads the router. This construct and mifos's runtime-selected table are both recorded in the approximations of Section 6.3. alfresco's route set is a declared lower bound on other grounds, so no claim of completeness is made over it either way.

The remaining 1 is bitwarden's vault, whose route is produced by a helper that chooses between two components by feature flag.

A.6.3 Nothing is missing that was not declared

0 applications hold a screen the framework mounts and the extractor has no node for without the extractor having said that part of its route set was beyond it. Every application in Table 5.19 that has a missing screen is an application that declared one. The correspondence is per application and not per cause, which is the weaker

reading and the only one measured here. alfresco declares that it could not read ten spreads, but one of its 3 missing screens, `/about`, has nothing to do with a spread: it is declared in a JSON manifest and in no route file. The measurement supports the claim that no application is missing a screen while asserting that its route set is whole. It does not support the claim that every missing screen is behind the construct its application named.

On this corpus the correspondence also holds the other way: 0 applications declare part of the route set unread and turn out to be missing nothing, and 2 declare nothing and are missing nothing. That is not a property of the analysis and is not offered as one. The direction that matters is that a screen missing with no declaration would be a tool asserting a completeness it does not have, and there are none of those.

This changes what the dead-link census of Section 5.6 can rest on. The census used to suppress every candidate from an application whose route set is a declared lower bound, and a per-application licence lifted the suppression where this comparison found the declared set complete. On the figures above the licence lifts it for 0 applications: both applications the census draws from are missing a screen of their own (mifos 5 and bitwarden 1), so neither can be licensed. A licence of that shape asks whether a whole route set is complete, which is harder than the census needs, and it loses the census of any application that misses one screen anywhere. Section 5.6 therefore asks the question per candidate, of the same framework configuration. A link whose target the framework serves nothing at is a dead link whether or not the rest of the route set was read, and one whose target it does serve is not a dead link however the extractor read it.

When this check was first run, the correspondence failed. alfresco, whose document viewer is assembled by a helper call spread into the table, and bitwarden, whose vault route is built the same way, were asserting complete route sets while holding unread route sources. Each spread was skipped silently, and bitwarden's silence had a cost: with its caveat absent, that application's sites entered the dead-link census on a route set that was missing a screen. Both readers now record a spread they cannot follow as a route source they did not read. The extractor declares the spread whose expression it cannot resolve to an array, and the independent route parser declares every spread, since it follows none. The second is the more conservative, and neither is silent, which is what the property requires.

A.7 What a route table's evidence does not establish

One assumption underlies every approximation in the reader of route tables. A route table that no site in the application claims is read as a table the application mounts at its root. A table is unclaimed when no `loadChildren` names it, no spread or `children` carries it, and no registration call takes it. The assumption is what lets either tool find an application's routes however they are declared. It is an over-approximation: a table registered nowhere the compiled build reaches is

mounted at no address at all, and reading it at the root puts its entries into the graph as screens. It is taken deliberately, because the alternative loses the routes of every application whose root registration neither tool recognises. The cases below are the places where it has been narrowed and not abandoned.

Three further approximations in that reader share one shape, and each was found by attacking a repair of the previous one. All three are declared in the tool's own notes. None was closed, because closing it would refuse a construct the corpus actually uses.

Which file owns a lazily loaded route array. A module loaded through `loadChildren` usually keeps its routes in a separate routing module that it imports, so the reader follows one import to find them. Electing that owner no longer trusts the *shape* of an array: a menu or breadcrumb configuration carrying `path` properties is never elected. However, a `Routes`-typed array shows only that its author declared it to some router, not that *this* application mounts it. A sibling application's typed table, reached through a workspace alias and used as data, is therefore still elected and still minted under the lazy prefix, with a deterministic edge into it. Requiring the elected owner to lie inside the application directory closes that case, but it also refuses a route array that legitimately lives in a shared library outside the directory, which is a pattern the corpus does use. No corpus application distinguishes the two, so the gap is left open and declared.

The loader's specifier. The `loadChildren` specifier itself is not bounded, so a loader whose import resolves into another project's alias root mounts that project's table under this application's prefix. The obvious guard would require the loaded file to belong to the same project root as the route that loads it. It was applied experimentally. It refuses a lazily loaded module that one corpus application keeps in an alias root, turns several of that application's declared screens into declared blind spots, and lowers its reported coverage. By path alone the analysis cannot tell a shared lazy library from another application's route table, and that is the limit.

A routing module imported by two lazy modules. This one is a reading and not a defect. A routing module imported by two lazily loaded modules and loaded by no route is read as a table of the project root and not as children of both importers. The alternative matches Angular more closely but lets one route adopt another's table, which is how the first approximation was reached. The conservative reading is taken, and it costs the affected routes their prefix.

The test for a root table. A fourth case was found by asking what the test that keeps a lazy route from adopting the application's root table actually read. It is reported as a repair because the question produced a measurement that no reading of the code would have given. The test looked for the candidate file's own call to `RouterModule.forRoot`, Angular's discriminator for a root table. That made it sound where it fired, and it is also why it fired nowhere. An application that registers its root with the standalone `provideRouter` API is not recognised at all, and two of

the six have no `forRoot` call anywhere. An application that declares its root array in one file and registers it in another, as ghostfolio and kubernetes/dashboard both do, is recognised in the registering file, which holds no route, and not in the file a lazily loaded module could adopt.

Instrumenting the predicate turned the limit into a measurement. When this repair was made (commit `d40c8dc`, before the final extractor, and not re-measured on it), the test was asked 698 times across the six applications in the independent route parser and 2798 times in the extractor, and answered yes on none of them. These counts were read from an instrumented run at that commit whose output was not committed, so they cannot be regenerated. The independent route parser's own source comment gives 2798 for its predicate, so which count belongs to which tool is not settled. A guard that never fires is worse than no guard, because a commit message and a chapter credit it with a property nothing has shown it to have. Both tools now mark root tables per *array*, taken from the argument of either registration API and resolved through the name it is nearly always written as.

Repairs that stand on fixtures only. Five repairs of the same reader stand on constructed fixtures and on nothing else: a file holding a menu beside an imported routing module, a `forChild` given a table declared in another file, a conditional resolved inside a registration call, a root table a spread elsewhere also carries, and a wrapper that takes its own path as a parameter. Each produced an address the application does not mount, or a declared loss where nothing was lost, and each is now refused. None occurs in this corpus, so every figure in Chapter 5 is identical with and without them, and no result here is evidence that they work. Their evidence is the fixtures, which fail when the rule each pins is removed.

The same holds of the refusals in the independent route parser. The test that stops a lazy route adopting a table another route owns was asked, at the same commit and in the same uncommitted run, 174 times in the extractor and 3512 in the independent route parser, and answers yes in neither. For those refusals the corpus is evidence about the reading the two tools share and not about the guards that bound it. The per-array root-table test is the exception: at that commit it was asked a few hundred times and answered yes eight times in each reader, and one of those answers is what mints alfresco's `/rules`. A rule the corpus exercises and a rule only a fixture exercises are different kinds of claim. The reverse also holds: a corpus of six applications is not a sample that makes an analysis sound, and five constructs it happens not to contain were enough to break two of the properties this thesis claims.

One further case came with it. A table that a feature library both hands to `provideRouter` and exports for lazy loading is mounted at the root *and* under the prefix that loads it, so a claim on its file may not cover both mounts. The root scan now mints a root-registered array whatever claims its file, and alfresco's `/rules` is such a case.

A test specification as the only loader. This is the mirror of the first approximation, a case in which a conservative refusal becomes an over-approximation. When

the only site that loads a route table lies in a test specification, the analysis declines the claim and scans the table from the project root, so its routes are reported at the application's root address. The guard establishes that a site the walk cannot reach may not justify suppressing the root scan. It does not establish that the application mounts the table there. This is a further instance of the unclaimed-`Routes` over-approximation declared above, and it occurs nowhere in this corpus.

The named outlet. The history of the named outlet shows the difference between declaring an approximation and being sound over it. A route carrying `outlet: 'name'` is reached as `(name:path)`, so it contributes no screen on the primary outlet. The extractor refused such a route from the start, but only the route that carries the property. Where a named outlet opened a `children` subtree, the routes beneath it were still assembled against the primary path and minted there. alfresco mounts its document viewer that way, and the graph held `/view/:nodeId` and `/search-libraries/view/:nodeId`, two addresses that application serves only through the viewer outlet. The independent route parser read the subtree the same way, so coverage was 100 % over a set holding both, and no comparison between the two tools could show the gap. The case was written down as open, on the reasoning that an approximation stated is an approximation a consumer can price.

That reasoning was wrong, as the runtime route oracle of Section A.6 showed. A consumer reading the graph finds a node at `/view/:nodeId`, and a navigation to that address in the built application is answered by the catch-all route. The declaration did not make the node true, and the must-tier property admits no exception on the strength of a written caveat. An outlet is inherited, so nothing below a named-outlet route has a primary address either. The extractor now carries outlet context down the walk and mints no screen for such a subtree. What remains is a limit on what the graph contains: a screen reached only through a named outlet is not an element of the node set, and the transitions into it are not in the edge set. That is a statement about recall, and it is the kind of approximation a declaration can legitimately cover.

A filename heuristic. One attribution of a blind spot to the component it hides is a filename heuristic and is labelled as one. Where a route prefix the analysis could not follow has literal segments, a component whose path carries all of them as whole directory names is reported as hidden behind that prefix. The heuristic answers the question a reader asks, which is which screens are missing because of this hole. It is wrong in two directions that the harness states. A route prefix does not determine a directory name, so components behind a hole whose directory is named differently are not attributed to it and are reported only as having no node. A prefix made entirely of parameters has no literal segment, so it is attributed to no component at all.

The attribution reaches the published reason table in neither form. It is asked of all 21 unaccounted rows in the corpus and declines every one of them on its own first two conditions. For 10 the owning file is not a component, and for the rest the graph already witnesses an edge from that file. The comparison itself is never reached on this corpus, so what is recorded here is a property of the rule and not of any figure

in Chapter 5.

A.8 The intermediate must tier, construct by construct

Table A.4 lists every construct that the 20 must edges of the intermediate version cite, with the edge that cites it and the condition the current extractor reads at it (Section 5.5). A construct appears once for each edge that cites it. The condition is read by the extractor's own readers, so the table records the extractor's decisions, and a reader checks it by opening each construct at the line given.

Table A.4: The constructs behind the intermediate must tier and the condition the extractor reads at each. *none* means the construct runs whenever its template or handler does.

Application	Edge	Construct	Condition read
realworld	`/` → `/`	`src/app/features/article/pages/home/home.component.html:23`	structural directive
realworld	`/` → `/`	`src/app/features/article/pages/home/home.component.html:35`	none
realworld	`/` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`/tag/:tag` → `/`	`src/app/features/article/pages/home/home.component.html:23`	structural directive
realworld	`/tag/:tag` → `/`	`src/app/features/article/pages/home/home.component.html:35`	none
realworld	`/tag/:tag` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`/login` → `/`	`src/app/core/auth/auth.component.ts:76`	in a callback
realworld	`/register` → `/`	`src/app/core/auth/auth.component.ts:76`	in a callback
realworld	`/editor` → `/`	`src/app/features/article/pages/editor/editor.component.ts:52`	under a branch
realworld	`/editor/:slug` → `/`	`src/app/features/article/pages/editor/editor.component.ts:52`	under a branch
realworld	`/article/:slug` → `/`	`src/app/features/article/pages/article/article.component.ts:117`	in a callback
realworld	`/profile/:username/favorites` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/footer.component.html:3`	none
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:3`	none
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:10`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:27`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:59`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:88`	control-flow block
bitwarden	`/verify-email` → `/`	`src/app/auth/verify-email-token.component.ts:53`	superseded by a later navigation
bitwarden	`/verify-email` → `/`	`src/app/auth/verify-email-token.component.ts:66`	after await

Application	Edge	Construct	Condition read
bitwarden	/verify-recover-delete-org → /	src/app/admin-console/organizations/manage/verify-recover-delete-org.component.ts:43	under a branch
bitwarden	/verify-recover-delete-org → /	src/app/admin-console/organizations/manage/verify-recover-delete-org.component.ts:58	after early exit
bitwarden	/recover-delete → /	src/app/auth/recover-delete.component.ts:66	after early exit
bitwarden	/verify-recover-delete → /	src/app/auth/verify-recover-delete.component.ts:58	under a branch
bitwarden	/verify-recover-delete → /	src/app/auth/verify-recover-delete.component.ts:71	after await
bitwarden	/request-sm-access → /	src/app/secrets-manager/secrets-manager-landing/request-sm-access.component.ts:76	after early exit
bitwarden	/settings/sponsored-families → /	src/app/billing/settings/sponsored-families.component.ts:194	under a branch
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:86	superseded by a later navigation
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:112	under a branch
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:187	after await
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:214	under a branch
bitwarden	/organizations/:organizationId/settings/account → /	src/app/admin-console/organizations/settings/account.component.ts:260	under a branch
bitwarden	/confirm-key-connector-domain → /	../../libs/key-management-ui/src/key-connector/confirm-key-connector-domain.component.ts:116	after await
bitwarden	/settings/security/password → /	../../libs/angular/src/auth/password-management/change-password/change-password.component.ts:193	under a branch
mifos	/callback → /login	src/app/zitadel/callback/callback.component.ts:39	under a branch
mifos	/callback → /login	src/app/zitadel/callback/callback.component.ts:47	in a catch clause

Bibliography

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Effects. `https://angular.dev/guide/signals/effect`, 2026. Accessed November 2026.

[27] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[28] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[29] May Haydar, Alexandre Petrenko, and Houari Sahraoui. Formal verification of web applications modeled by communicating automata. In *Formal Techniques for Networked and Distributed Systems (FORTE)*, volume 3235 of *Lecture Notes in Computer Science*, pages 115–132, 2004.

[30] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[31] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[33] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[34] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[36] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[37] Iva Kertusha, Gebremariam Assres, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry. *Science of Computer Programming*, 253:103473, 2026.

[38] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[39] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[40] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[41] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[42] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[43] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[44] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[45] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[46] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[47] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[48] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[49] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[50] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[51] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[52] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[54] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[55] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[56] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[57] Naito Oshima and Tomoji Kishi. Model checking method for SPA page transition based on component-based framework. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)*. KSI Research, 2018.

[58] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[59] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[60] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[61] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[62] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[63] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[64] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[65] Bachar Rima, Anas Shatnawi, and Abdelhak-Djamel Seriai. E2E test generation over a source-code-derived workflow space: An Angular case study. In *Software Engineering and Advanced Applications*, Lecture Notes in Computer Science, pages 334–351, Cham, 2026. Springer Nature Switzerland. Published online 27 August 2026; print volume dated 2027. The method is described in this thesis from the publisher abstract and the replication package; the full text was not available.

[66] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[67] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[68] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[69] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[70] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[71] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. *Proceedings of the ACM on Software Engineering*, 2(FSE):1001–1022, 2025.

[72] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[73] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[74] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[75] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 49(3):1086–1101, 2023.

[76] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[77] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[78] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[79] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[80] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[81] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[82] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, Singapore, 2023. Association for Computational Linguistics.

[83] Tao Zhang, Fuzhong Hao, Yunfan Wang, Bo Zhang, and Guangwei Xie. VeriFlow: A framework for the static verification of web application access control via policy-graph consistency. *Electronics*, 14(18):3742, 2025.

[84] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.